Basic Knowledge

Everything you need to understand
the Cardano track and Shelfie

TUM Blockchain & AI Hackathon 2026
Track: **Agentic Commerce on Cardano**
Team *Los Curiosos*: Helmut · Isabel · Fernando · Lorenz

How to read this guide. Part I explains blockchains and Cardano from zero. Part II explains the payment and agent technologies of the track: HTTP 402, x402, AI agents, Masumi and Sokosumi, and what "on-chain" really means. Part III explains our own idea, Shelfie, and how all the pieces fit together on hackathon day. Colour code of the boxes:

concept · example · everyday analogy · pitfall · how Shelfie uses it · takeaways · self-check

Study material, prepared before the build window. Facts about tools were taken from the team's research notes (`investigacion/`, `diseno/`); anything marked *verify Friday* is still unconfirmed.

Contents

I Foundations 4

1 Blockchain from zero 5
- 1.1 What is a ledger, and why share it? . . . 5
- 1.2 Hashes: digital fingerprints . . . 6
- 1.3 Keys and digital signatures . . . 7
- 1.4 Wallets, addresses and the seed phrase . . . 7
- 1.5 Transactions . . . 9
- 1.6 Blocks and the chain . . . 9
- 1.7 Consensus: how the network agrees . . . 10
- 1.8 Finality and confirmations . . . 11
- 1.9 Fees . . . 11
- 1.10 Tokens and stablecoins . . . 12
- 1.11 Smart contracts (a first look) . . . 12
- 1.12 Public explorers . . . 12
- 1.13 Testnets and mainnet . . . 13
- 1.14 Why use a blockchain for payments between machines? . . . 13

2 Cardano 15
- 2.1 What is Cardano? . . . 15
- 2.2 ADA and lovelace . . . 15
- 2.3 Ouroboros and stake pools . . . 16
- 2.4 The eUTXO model, step by step . . . 16
- 2.5 Extended UTXO: datum, redeemer, validator . . . 19
- 2.6 Native tokens . . . 21
- 2.7 Fees and the minimum ADA . . . 21
- 2.8 Metadata . . . 22
- 2.9 Networks: mainnet, preprod, preview . . . 22
- 2.10 Getting test ADA . . . 23
- 2.11 Blockfrost: reading and submitting without running a node . . . 23
- 2.12 Cardanoscan: reading a transaction . . . 24
- 2.13 Wallet setup for the team . . . 25
- 2.14 Cardano in our project . . . 25

II Payments and agents 28

3 HTTP 402 and x402 29
- 3.1 HTTP from zero . . . 29
- 3.2 The history of `402 Payment Required` . . . 30
- 3.3 Why paying per request matters for machines . . . 31
- 3.4 The x402 protocol . . . 31
- 3.5 x402 on Cardano . . . 34
- 3.6 Worked example: the Shelfie verifier . . . 35
- 3.7 Pitfalls . . . 37
- 3.8 x402 versus the alternatives . . . 37

4 AI agents, agentic commerce, Masumi and Sokosumi **39**
4.1 Large language models in one page . . . 39
4.2 From model to agent . . . 40
4.3 Agentic commerce . . . 41
4.4 The track brief, explained . . . 42
4.5 Masumi Network . . . 43
4.6 Sokosumi . . . 45
4.7 What already exists: the competitive landscape . . . 47
4.8 How Shelfie uses each piece . . . 48

5 On-chain versus off-chain **50**
5.1 Two zones . . . 50
5.2 Hash commitments: proving without publishing . . . 51
5.3 Escrow from zero . . . 52
5.4 Oracles and the oracle problem . . . 54
5.5 Trust model: who must trust whom? . . . 54
5.6 What can go wrong: failure scenarios . . . 55
5.7 What is on-chain and off-chain in Shelfie . . . 56

III Our idea: Shelfie **58**

6 Shelfie: the idea **59**
6.1 The problem: nobody can see the shelf . . . 59
6.2 Our fictional customer: Katharina Brandl . . . 60
6.3 The three payment gates . . . 60
6.4 The demo moment . . . 62
6.5 Why it fits the judging rubric . . . 63
6.6 Business model (illustrative pricing) . . . 63
6.7 Limits we admit . . . 64
6.8 Alternatives we considered . . . 64

7 Architecture and domain model **66**
7.1 The big idea: a verified micro-task engine . . . 66
7.2 Components and owners . . . 67
7.3 One task, end to end . . . 68
7.4 Entities and ID prefixes . . . 68
7.5 State machines . . . 70
7.6 Idempotency and the payout queue, from zero . . . 71
7.7 Wallets and who pays whom . . . 73
7.8 Key API endpoints . . . 73
7.9 Important decisions (ADRs) in plain language . . . 74

8 The verifier **77**
8.1 Design principle: a deterministic gate around a probabilistic reader . . . 77
8.2 The challenge code . . . 77
8.3 The verification pipeline and gates G1 to G7 . . . 79
8.4 The vision model contract . . . 80
8.5 Thresholds . . . 81
8.6 The 20-photo evaluation . . . 81
8.7 Manual override: a stage safety net, clearly flagged . . . 82
8.8 Privacy . . . 83
8.9 What the verifier cannot prove . . . 83

9 Hackathon day **85**
9.1 Official key times and the no-code rule 85
9.2 The milestone ladder M1 to M6 85
9.3 Top risks 87
9.4 The evidence log 88
9.5 Submission checklist: the essentials 88
9.6 Pitch phrases, verbatim 88
9.7 Top Q&A to know by heart 89

A Glossary **91**

Part I

Foundations

CHAPTER 1

Blockchain from zero

Why this matters for us

Shelfie pays real people (“workers”) for photographing price tags, and the AI agents in our system pay each other for services. All of these payments happen on a blockchain. You do not need to be a cryptography expert to build Shelfie, but you must understand what a blockchain *is*, what a *transaction* is, why a payment is “final”, and why nobody can secretly change a receipt. This chapter explains the ideas one by one, with no prior knowledge assumed. Chapter 2 then narrows everything down to Cardano.

1.1 What is a ledger, and why share it?

A **ledger** is simply a list of records that says who owns what. A bank’s database is a ledger: “Anna has 120 EUR, Ben has 40 EUR.” Every payment adds a line, for example “Anna pays Ben 20 EUR”.

Normally one organisation owns the ledger (the bank). You must *trust* that organisation: that it does not lose your line, does not change it, and does not refuse to show it to you.

A **distributed ledger** is a ledger that many independent computers keep *at the same time*, each with its own full copy. A **blockchain** is the most common kind of distributed ledger. The computers follow shared rules to agree on which new lines are valid, so no single owner is needed.



Figure 1.1: Left: a bank owns the only ledger. Right: many computers (nodes) each keep a full copy of the same ledger.

Everyday analogy: The village notebook

Imagine a village where every sale is announced in the market square and *every* villager writes it into their own notebook. If one villager tries to cheat by writing a different amount, the other 99 notebooks disagree with him and he is ignored. The ledger is “true” because the majority holds the same story, not because a boss says so.

1.1.1 What a shared ledger gives us

- **Transparency:** anyone can read the ledger (on public blockchains).
- **Tamper resistance:** changing history would require changing most copies at once.
- **No gatekeeper:** nobody can block you from sending a valid transaction.
- **Shared truth between strangers:** two parties who do not trust each other can still agree on "this payment happened".

1.2 Hashes: digital fingerprints

The most important tool in a blockchain is the **hash function**. It takes any data (a word, a photo, a whole book) and produces a fixed-length "fingerprint" called a **hash** or **digest**. The popular function SHA-256 always produces 256 bits, usually written as 64 hexadecimal characters.

A hash function has four properties that matter to us:

1. **Deterministic:** the same input always gives the same hash.
2. **Fixed size:** a 3-byte input and a 3-gigabyte input both give 64 hex characters.
3. **One-way:** given a hash, you cannot work backwards to find the input.
4. **Avalanche effect:** changing even one character of the input changes the hash completely and unpredictably.

Example: A tiny change, a totally different hash

Real SHA-256 values (computed with the standard `sha256sum` tool):

Input	SHA-256 (64 hex characters)
`Alice pays Bob 10`	`69ad18dfd1ac4b3d711b02ee62caa00a82f0f67cc11ebacdd0c0ede0d8b6eaed`
`Alice pays Bob 11`	`83e970b5a467ce2dea0a573ee54380252a1d193b2625e2c29ad43ad6600990aa`
`alice pays bob 10`	`be26845c48086d4f8edae714f8e48e32d806b4bcd4c054a0c19eb634b1f292a6`

We changed one digit (10 to 11) or only the capital letters, and the result shares nothing recognisable with the original. If someone alters an old payment from 10 to 11, its hash no longer matches the one everybody stored, so the fraud is visible immediately.

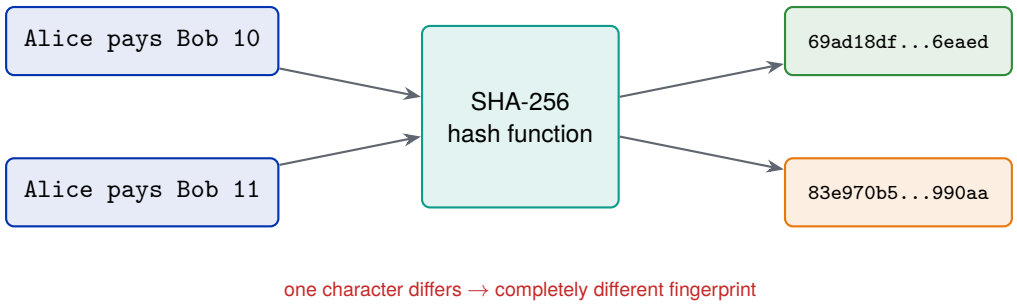


Figure 1.2: A hash function turns any input into a fixed-size fingerprint. Tiny input change, huge output change.

Everyday analogy: The tamper seal

A hash works like a tamper-evident seal on a jar: you cannot tell what is inside from the seal, but if anybody opens the jar and closes it again, the seal no longer matches.

In Shelfie: Hashes appear everywhere

Every Cardano transaction is identified by a hash (its *tx hash*), which we will paste into an explorer as proof of payment. Shelfie also uses hash-like identifiers (idempotency keys, hashes of photos) to make sure a photo is paid for only once. See the verifier chapters later.

1.3 Keys and digital signatures

1.3.1 Public and private keys

Hashes alone cannot prove *who* authorised a payment. For that we use **public-key cryptography**. You create a pair of mathematically linked numbers:

- the **private key** (also called secret key or signing key): kept secret; it is your power to authorise;
- the **public key**: can be shared with everybody; it lets others *check* what the private key did.

You can derive the public key from the private key, but not the other way round (similar to the one-way property of hashes).

Everyday analogy: Mailbox and master key

Your public key is like your mailbox address: everyone may know it and drop mail in. Your private key is the only key that opens the box and, in our case, also the only pen that can create your signature.

1.3.2 Digital signatures

A **digital signature** is a short piece of data created from (a) a message and (b) your private key. Anybody who has your public key can verify that the signature is genuine and that the message was not changed afterwards. The private key is never revealed.

Why is this powerful? The network does not need to know *you*. It only needs to check that the signature matches the public key that owns the funds. Whoever controls the private key controls the money. This is the reason for the blockchain saying: *"not your keys, not your coins"*.

✗ Common pitfall: Signatures protect the content, not your judgement

A valid signature only proves that the key holder approved *exactly this* message. If a bug in our software builds the wrong payment and signs it, the signature is perfectly valid and the money is gone. Blockchain payments cannot be "called back". This is why Shelfie validates a photo *before* it triggers a payout.

1.4 Wallets, addresses and the seed phrase

A **wallet** is the software (or device) that stores your private keys, creates signatures, and shows your balance. Strictly speaking, the wallet does not "contain coins"; the coins live on the blockchain. The wallet contains the *keys* that unlock them.

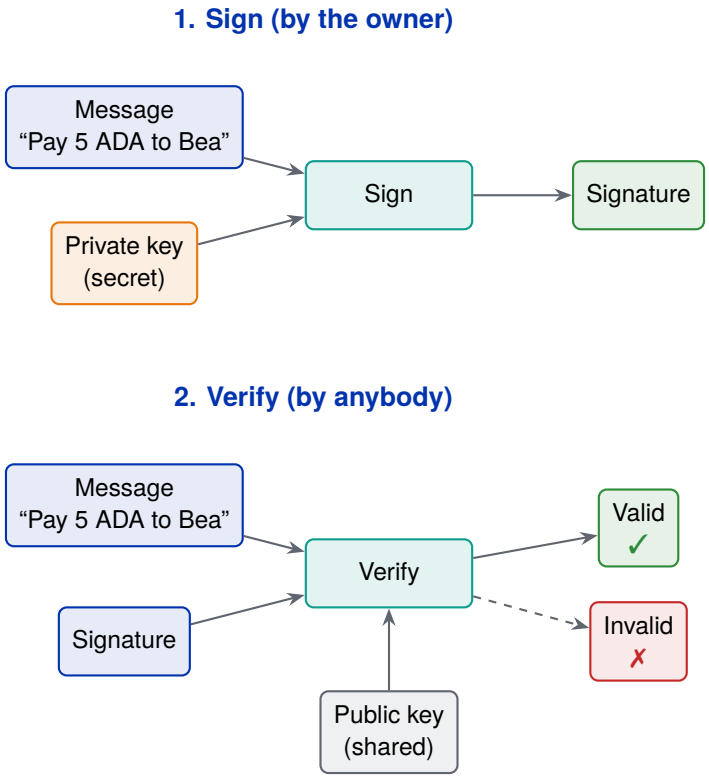


Figure 1.3: Signing needs the private key; verifying needs only the public key. If the message or signature is altered, verification fails.

An **address** is the text string others use to send you funds. It is derived from your public key (via hashing), so it is safe to share. On Cardano preprod, test addresses start with `addr_test1...`; on mainnet they start with `addr1...` (details in Chapter 2).

1.4.1 The seed phrase (mnemonic)

Writing down a long random private key is error-prone. Wallets therefore generate a list of normal words, the **mnemonic** or **seed phrase**, typically 12, 15 or 24 words. From this list the wallet can regenerate *all* your private keys and addresses, on any device.

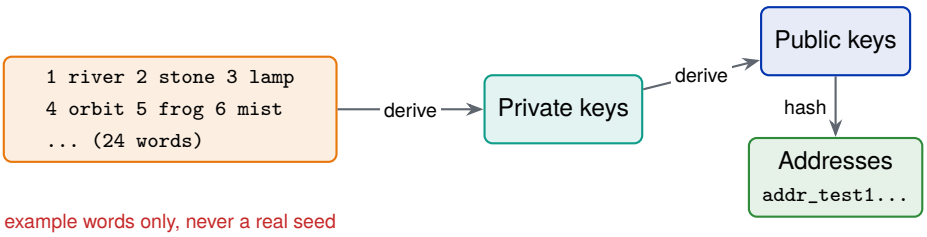


Figure 1.4: One mnemonic generates all keys and addresses. Everything flows one way: you cannot go from an address back to the mnemonic.

✗ Common pitfall: Never share your seed phrase

Anyone who sees your seed phrase can recreate your keys and take everything, on every address, forever. No bank can reverse it. Rules for our team:

- Never paste a mnemonic into chat, a ticket, a screenshot, or a Git commit.
- Keep it only in a local `.env` file that is listed in `.gitignore`, or in a password manager.

- For the hackathon, use *throwaway test wallets* that hold only worthless test funds.
- Nobody legitimate (not a mentor, not a "support agent") will ever ask for it.

In Shelfie: Our wallets are server-side

In Shelfie, our backend signs payments itself. The research notes describe a headless signer that reads a 24-word mnemonic from an environment variable (`.env`) instead of a browser wallet. That is convenient for agents, and it is exactly why the mnemonic must be a test-only throwaway wallet that never reaches Git.

1.5 Transactions

A **transaction** ("tx") is a signed instruction to change the ledger: "move these funds from here to there". It contains who spends what, who receives how much, a fee, and the signature(s) of the owner(s). The network checks the rules and, if valid, accepts it. Then the transaction gets its **transaction hash** (tx hash), which acts as its unique ID and receipt.

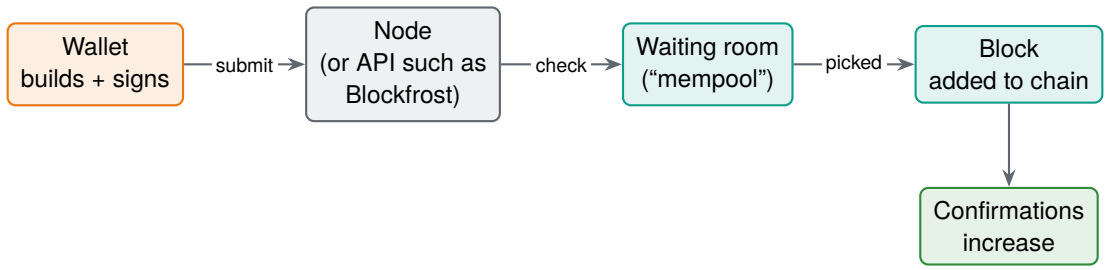


Figure 1.5: Life of a transaction: build and sign, submit, wait, be included in a block, then gain confirmations.

Example: A simple payment

Mia wants to pay worker Noor 4 ADA for a photo. Her wallet builds a transaction: "Mia's funds $\rightarrow$ 4 ADA to Noor's address, fee ≈ 0.17 ADA". Mia's wallet signs it with her private key and sends it. A few seconds later it sits in a block. The tx hash can now be given to Noor, or to a judge, as proof.

1.6 Blocks and the chain

Transactions are not added to the ledger one by one. Nodes collect many valid transactions and group them into a **block**. Each block contains:

- a list of transactions,
- a timestamp (or slot number),
- the **hash of the previous block**,
- its own hash.

Because each block includes the previous block's hash, the blocks form a **chain**. This is where the name "blockchain" comes from.

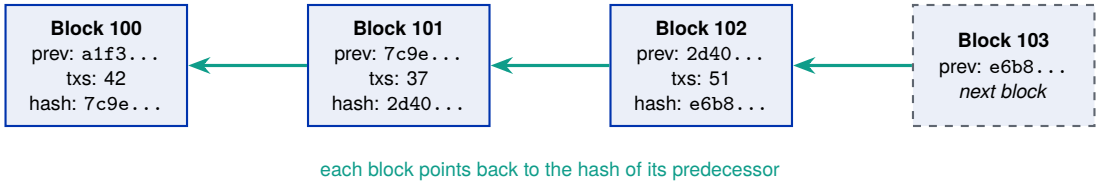


Figure 1.6: A chain of blocks. The hashes shown are shortened examples.

1.6.1 Why chaining makes history hard to change

Suppose an attacker alters one old payment in Block 101. Block 101's content changes, so its hash changes. But Block 102 stored the *old* hash of 101, so the link breaks. To hide the fraud, the attacker must also recompute Block 102, 103, and every later block, and then convince the majority of the network to accept this alternative history. The more blocks sit on top of a transaction, the more effort that takes.

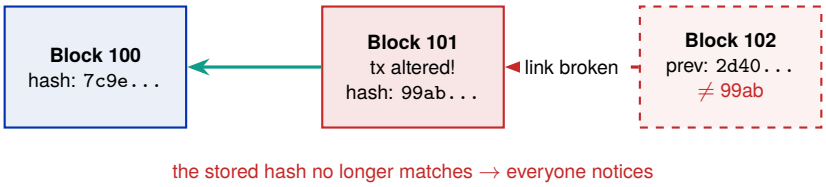


Figure 1.7: Tampering with one block breaks the link in the next block.

1.7 Consensus: how the network agrees

If thousands of computers keep a copy, who decides which block comes next? The rules for agreeing are called **consensus**. Two major families exist.

1.7.1 Proof of Work (PoW)

Used by Bitcoin. Miners compete to solve a hard puzzle (guess a number so that the block's hash starts with many zeros). The winner adds the next block and gets a reward. The puzzle needs massive computing power and electricity, which makes cheating expensive.

1.7.2 Proof of Stake (PoS)

Used by Cardano (and Ethereum today). Instead of burning electricity, participants *lock up* coins as a deposit, their **stake**. The protocol selects who creates the next block, with probability proportional to stake. Cheating risks the deposit and reputation, and honest work earns rewards.

	Proof of Work	**Proof of Stake**
Scarce resource	computing power and electricity	locked coins (stake)
Who adds blocks	miners who win a puzzle	validators chosen by stake
Energy use	very high	low
Entry barrier	special hardware	coins (or delegate to a pool)
Example	Bitcoin	Cardano, Ethereum
Typical block time	Bitcoin: about 10 minutes	Cardano: about 20 seconds

Table 1.1: PoW versus PoS in one view. Block times are approximate.

Everyday analogy: Lottery with deposits

PoS is like a raffle where every ticket is a coin you have locked as a deposit. The more tickets you hold, the more often you are drawn to write the next page of the notebook. If you cheat, your deposit is at risk.

1.7.3 Forks

Sometimes two valid blocks appear almost at the same time and the chain briefly splits into two branches (a **fork**). The protocol's rule (for example, "follow the longest valid chain") quickly makes everybody prefer one branch. Transactions only in the abandoned branch go back to the waiting room and are normally included again. This is why a freshly included transaction is not yet "final".

1.8 Finality and confirmations

A transaction is **confirmed** once it is in a block, and each new block built on top is another **confirmation**. The deeper it is buried, the less likely it is that a fork will remove it. **Finality** means the point at which reversal is practically impossible.

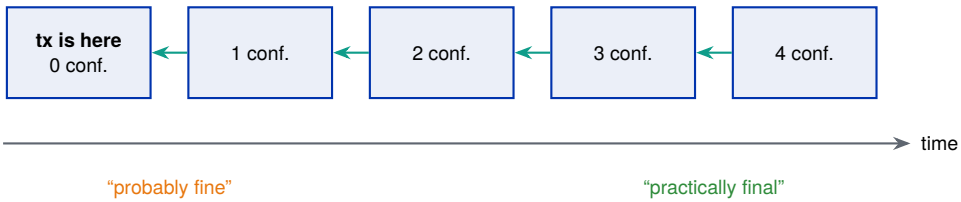


Figure 1.8: Each block added on top of the transaction's block adds one confirmation.

✗ Common pitfall: "Submitted" is not "confirmed"

A node accepting your transaction only means it entered the waiting room. For a payment you must wait until it is in a block, and ideally a few blocks deep. Our research notes also warn: if a payment call times out or the connection drops after "sending", the payment may *still* settle later. Always check the explorer before sending again, otherwise you may pay twice.

In Shelfie: Why our payout queue reconciles

The design stores a metadata tag on every payout transaction and, after a crash or timeout, looks the chain up for that tag before ever retrying. Exactly one payout per task is the rule. The mechanism and its states are described in the Shelfie architecture chapters.

1.9 Fees

Using the network costs a small **transaction fee**. Fees pay the people who run the infrastructure and discourage spam. On Cardano the fee depends mostly on the transaction's *size in bytes* (plus script execution if scripts are used), and is typically a fraction of an ADA; the team's runbook mentions about 0.17 ADA per payment. The sender pays the fee on top of the amount sent.

Different chains price fees differently: some are almost free, some become very expensive when busy. Predictable, low fees are what makes small machine payments (a few cents) practical.

1.10 Tokens and stablecoins

The chain's own currency (for Cardano: ADA) is the **native coin**. Besides it, chains can carry other **tokens**: digital items that represent value, points, access rights, or a real-world currency.

A **stablecoin** is a token designed to keep a stable value, usually 1 token ≈ 1 US dollar. This is important for commerce: a shop does not want its price to move 10 % between the order and the payment.

	Native coin (ADA)	**Stablecoin (e.g. USDM)**
Value	floats with the market	aims at a fixed peg (dollar)
Needed for fees	✓ always	✗ fees still need ADA
Good for	fees, staking, security	stable prices
On testnet	tADA, free	test version (tUSDM), free

Table 1.2: Coin versus stablecoin.

In Shelfie: We price in tADA

The design decision (ADR-03) is to price the verifier fee and worker payouts in tADA, not in a test stablecoin. The reason is that two incompatible test-stablecoin tokens exist (the SDK one and the one from the Masumi dispenser), so a wallet funded with one cannot pay a route priced in the other. Plain tADA avoids this trap.

1.11 Smart contracts (a first look)

A **smart contract** is a program stored on the blockchain that runs automatically when its conditions are met and enforces rules without a middleman. Typical rules: "release the money only when both parties signed" or "refund the buyer if no result arrives within a time limit". Because the program's code and the ledger are shared, nobody can quietly change the rules afterwards.

Everyday analogy: The vending machine

A vending machine is a contract: put in the coins, press the button, and the product drops. No cashier is needed, and the machine follows the same rule for everybody. A smart contract is a vending machine for agreements.

This is only the intro. How Cardano runs contracts (validators, datum, redeemer) is explained in Chapter 2. For Shelfie, an important point: Masumi's *escrow* (money locked until the job result is delivered) is a smart-contract mechanism that we use through the Masumi service; we do not write our own contract for the MVP.

1.12 Public explorers

A **block explorer** is a website that lets anyone search the public ledger: by transaction hash, address, or block. It is the "receipt viewer" of a blockchain. Because chains are public, anyone (a judge, a worker, a teammate) can open the same page and see the same facts, without having to trust our app.

On Cardano the explorer we use is Cardanoscan (preprod version); Chapter 2 shows how to read a transaction page.

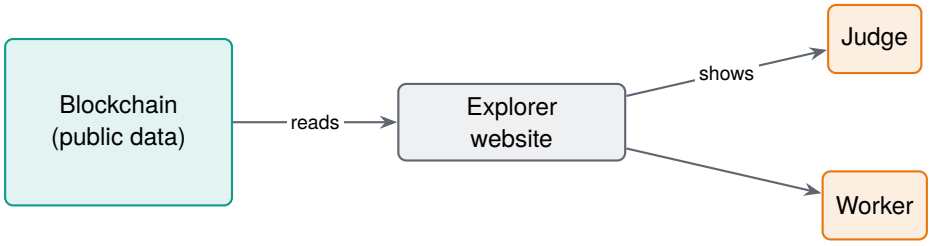


Figure 1.9: An explorer turns raw ledger data into pages anyone can open and share as a link.

1.13 Testnets and mainnet

The real network, where coins have real money value, is the **mainnet**. A **testnet** is a copy of the technology with worthless test coins, built for developers to experiment without risking money. You can get test coins for free from a **faucet**, a website that hands them out.

	Mainnet	**Testnet**
Coins have value	✓	✗
Mistakes cost money	✓	✗
Faucet available	✗	✓
Used for	real products	development, demos, hackathons
Example addresses (Cardano)	`addr1...`	`addr_test1...`

Table 1.3: Mainnet versus testnet.

✗ Common pitfall: Check which network you are on

Mixing networks is a classic mistake: a testnet address cannot receive mainnet money, and tools sometimes default to mainnet. The runbooks warn that the Sokosumi MCP server defaults to mainnet, so always confirm the network is *preprod* before paying. All hackathon money is play money, and we state our amounts as tADA.

1.14 Why use a blockchain for payments between machines?

Imagine a software agent that needs to buy a service from another agent, at 3 a.m., for 0.2 EUR, with an unknown counterpart across the world. Compare this with normal card or bank payments.

	Card or bank transfer	**Blockchain payment**
Needs a bank account	✓ (a human or company must open it)	✗ a key pair is enough
Programmable by software	limited, via bank APIs and approvals	✓ build, sign, submit by code
Works for tiny amounts	often not (fixed fees)	✓ if fees are small
Availability	business hours, borders	24/7, global
Public receipt	✗ private to the parties	✓ tx hash anybody can verify
Reversible (chargeback)	✓	✗ final
Escrow with machine rules	hard	✓ via smart contracts

Table 1.4: Why machines (and Shelfie) prefer on-chain payments.

The trade-off is real: finality means mistakes cannot be reversed, and tools are younger. But for software agents paying each other, **no account, programmable, public receipt** is exactly what is needed.

In Shelfie: What we gain in the demo

Judges will not need to trust our slides. Every verifier fee and every worker payout is a real tADA transaction on Cardano preprod that they can open with a link on Cardanoscan. That public receipt is our strongest piece of evidence.

★ Key takeaways

- A **ledger** lists who owns what; a **blockchain** is a ledger copied across many computers with no single owner.
- A **hash** is a fixed-size fingerprint; changing the input by one character changes it completely.
- A **private key** signs; the matching **public key** verifies. Whoever has the private key (or the seed phrase) controls the funds.
- **Never share a seed phrase**; use throwaway wallets with test funds only.
- A **transaction** is a signed change to the ledger, identified by its tx hash; blocks are linked by hashes, so history is hard to rewrite.
- Consensus is how the network agrees: **PoW** uses computing power, **PoS** (Cardano) uses staked coins.
- Confirmations grow with each new block; "submitted" does not mean "final".
- Fees are small and paid in the native coin; stablecoins keep a steady price; smart contracts enforce rules automatically.
- Explorers make the ledger readable; testnets use free, worthless coins; mainnet is real money.
- Machines like blockchains: no bank account, programmable, public receipts.

? Check yourself

1. You change a single letter in a message and hash it again. What happens to the hash, and why does this help to detect fraud?
2. Who can create a valid signature, and who can verify it?
3. Why can a stranger who finds your 24-word mnemonic empty all your addresses?
4. Why does altering an old block break all blocks after it?
5. Name one advantage of Proof of Stake over Proof of Work, and what Cardano uses.
6. Your payout call timed out. Why should you check the explorer before sending again?

Answers. (1) It changes completely and unpredictably; stored hashes then no longer match. (2) Only the holder of the private key signs; anybody with the public key verifies. (3) The mnemonic regenerates every private key. (4) The next block stores the old block's hash, so the link no longer matches. (5) Far lower energy use; Cardano uses Proof of Stake. (6) The first payment may still settle, and a retry would pay twice.

CHAPTER 2

Cardano

Why this matters for us

The track is called "Agentic Commerce on Cardano". Every payment in Shelfie (the verifier fee, the worker payout, the escrow around the brand's job) is a Cardano transaction. If the team understands Cardano's *UTXO* model, two of the most common bugs ("why does my wallet say no funds?" and "why can't two payments leave one wallet at once?") become obvious instead of mysterious. This chapter explains what Cardano is, how its money works, which tools we use to talk to it, and finally exactly which parts we touch in our project.

2.1 What is Cardano?

Cardano is a public, proof-of-stake blockchain (see Chapter 1) with its own coin, **ADA**. It was built with a research-driven approach and is known for formal methods, low and predictable fees, and a ledger model called *extended UTXO* that we explain in detail below.

Property	On Cardano
Consensus	Ouroboros, a Proof-of-Stake protocol
Native coin	ADA (smallest unit: lovelace)
Ledger model	extended UTXO (eUTXO)
Smart contract languages	Plutus (Haskell-based), Aiken, and others that compile to the same on-chain code
Native tokens	built into the ledger; no contract needed to create or send them
TypeScript tooling	Mesh, Lucid Evolution, Evolution SDK (used by the x402 template)

Table 2.1: Cardano at a glance.

2.2 ADA and lovelace

The coin is called **ADA**. Like cents are to euros, the smallest unit is called a **lovelace**:

$$1 \text{ ADA} = 1{,}000{,}000 \text{ lovelace}.$$

Software works with whole numbers of lovelace (no decimals, no rounding errors); humans read ADA. Our design contracts use lovelace, and our prose uses ADA.

Example: Converting between ADA and lovelace

ADA		lovelace
1 ADA	=	1,000,000
2 ADA (a typical x402 price in the runbook)	=	2,000,000
0.17 ADA (typical fee)	=	170,000
0.5 ADA	=	500,000
5 ADA	=	5,000,000

To go from ADA to lovelace, multiply by one million; to go back, divide by one million. In a code field such as `amount: "5000000"` that means *5 ADA*, not five million ADA.

✗ Common pitfall: Off by a factor of a million

A classic bug: writing `5` when you meant 5 ADA sends 5 lovelace (0.000005 ADA), which is below the minimum and will be rejected. Always check the unit in a field name or doc before using a number.

In Shelfie: tADA, not ADA

On the preprod test network, the coin is named **tADA** (test ADA). It behaves exactly like ADA but has no value. All Shelfie prices and payouts are in tADA. Any euro value in the pitch is illustrative only.

2.3 Ouroboros and stake pools

Ouroboros is Cardano's Proof-of-Stake protocol. Time is cut into short **slots** (one second each) grouped into **epochs** (five days). For each slot a **stake pool** is chosen at random, weighted by the amount of ADA staked with it, and that pool creates the block. A pool is run by a **stake pool operator**.

2.3.1 Delegation

You do not need to run a pool to take part. You can **delegate** your stake to a pool: your coins stay in your wallet, under your control, and are never sent anywhere; you only point their voting weight to a pool and receive a share of the rewards. Delegation is a mainnet topic and does not matter for the hackathon; it is mentioned here so that the words are familiar.

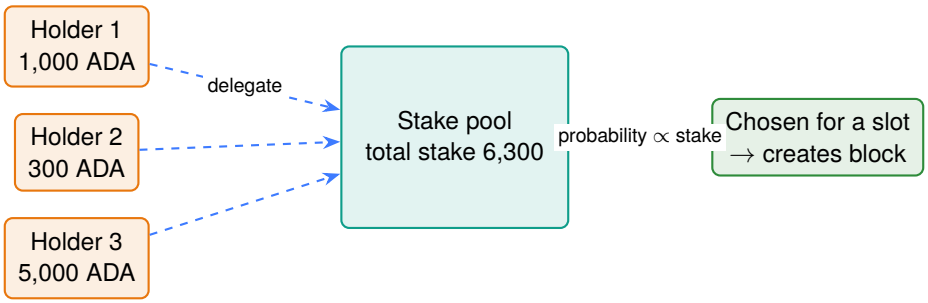


Figure 2.1: Delegation: holders keep their coins and lend their weight to a pool. The numbers are made up.

2.4 The eUTXO model, step by step

This is the most important concept of the chapter. Take your time.

2.4.1 Two ways to keep a ledger

There are two common ways to record who owns what:

Account model. Each address has a balance, like a bank account: "Mia: 15 ADA". A payment subtracts from one balance and adds to another. Ethereum works this way.

UTXO model. There are no balances. The ledger holds a set of individual coin "chunks" called **UTXOs**. Your "balance" is the sum of the chunks you can spend. Bitcoin and Cardano work this way.

UTXO stands for *Unspent Transaction Output*. An *output* is created by a transaction and sits on the ledger, owned by an address, until another transaction *spends* it. A UTXO can only be spent *whole and once*.

Everyday analogy: Banknotes and coins in a wallet

Your leather wallet has a 10-euro note and a 5-euro note. You do not own "15 euros in an account"; you own two physical items that add up to 15. To pay 4 euros you hand over the 10-euro note. The shop gives you 6 euros back as *change*. The note you gave is gone (spent); you now hold new items: the 5-euro note and the 6 euros change. UTXOs work exactly like this.

2.4.2 Inputs and outputs

Every transaction has:

- **Inputs:** the UTXOs it *consumes* (destroys). Each must be owned by someone who signs, or locked by a script that agrees.
- **Outputs:** the new UTXOs it *creates*, each with an address and an amount.
- A **fee** and optional extras (metadata, scripts).

The golden rule:

$$\sum \text{inputs} = \sum \text{outputs} + \text{fee}.$$

Nothing is created and nothing vanishes; it is only re-cut into new pieces.

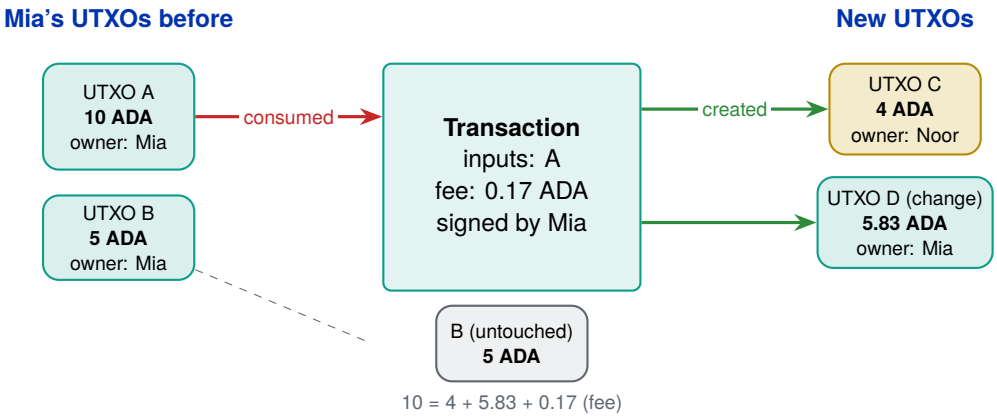


Figure 2.2: Mia pays Noor 4 ADA. Input A (10 ADA) is destroyed; two new outputs appear: 4 ADA for Noor and 5.83 ADA change back to Mia; 0.17 ADA is the fee. UTXO B is not used.

2.4.3 Why there is always a change output

Because a UTXO can only be spent whole, you cannot "take 4 out of the 10". Instead you consume all 10 and create *two* outputs: one for the recipient and one that sends the rest back to yourself (the **change output**). A wallet does this automatically. If you forget to add a change output in your own code, the leftover ADA is swallowed by the fee.

✗ Common pitfall: Forgetting the change

If the inputs add up to 10 ADA but your outputs only add up to 4 ADA, the transaction is invalid (fee too high) or, in a hand-built transaction, you would donate the remaining 5.83 ADA to the fee. Always check that inputs, outputs and fee balance.

2.4.4 Worked numeric example

Example: Mia pays Noor, twice

Start. Mia's wallet holds two UTXOs: A = 10 ADA and B = 5 ADA. Her "balance" is 15 ADA. Fee for each payment: 0.17 ADA (illustrative).

Payment 1: 4 ADA to Noor. The wallet picks A (it is big enough).

	ADA	
Input A	10.00	
Output to Noor (new UTXO C)	4.00	
Change to Mia (new UTXO D)	5.83	
Fee	0.17	
Check: 4.00 + 5.83 + 0.17	10.00	equals the input

Mia now owns B (5.00) and D (5.83): balance 10.83 ADA.

Payment 2: 10 ADA to Noor. Neither B (5.00) nor D (5.83) is large enough on its own, so the wallet uses *both* as inputs.

	ADA	
Inputs B + D	10.83	
Output to Noor	10.00	
Change to Mia	0.66	
Fee	0.17	
Check: 10.00 + 0.66 + 0.17	10.83	equals the inputs

Mia keeps one UTXO of 0.66 ADA. (Whether 0.66 ADA is allowed as a lone output depends on the minimum-ADA rule, see below; the example only illustrates the arithmetic.)

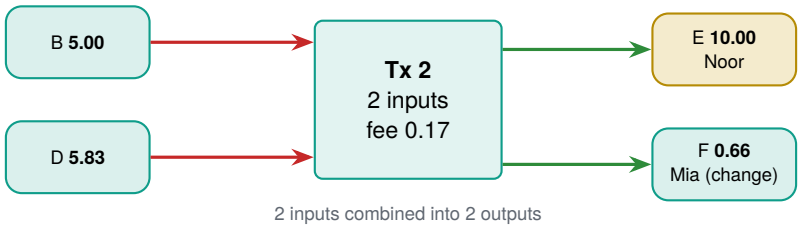


Figure 2.3: Combining small UTXOs: Payment 2 consumes B and D together.

2.4.5 The ledger is a set of unspent outputs

It helps to picture the whole chain as a big shelf of UTXOs. A transaction removes some items from the shelf (inputs) and puts new ones on it (outputs). An item that has been removed can never be used again. This is why double spending is impossible: the second attempt to spend the same UTXO finds it already gone.

2.4.6 Account model versus eUTXO

In Shelfie: One wallet cannot pay twice at the same time

The x402 runbook warns: "one wallet cannot pay concurrently (the first UTxO is the nonce)". Two parallel payments from the same wallet may pick the same UTXO as input, and the second one fails because that UTXO is already spent. This is why our design uses *separate* wallets (W-VBUY, W-PAY, W-SELL) and a *serial queue per wallet* (one send at a time, ADR-08). The throughput limit becomes the confirmation latency, about 20–60 seconds per check.

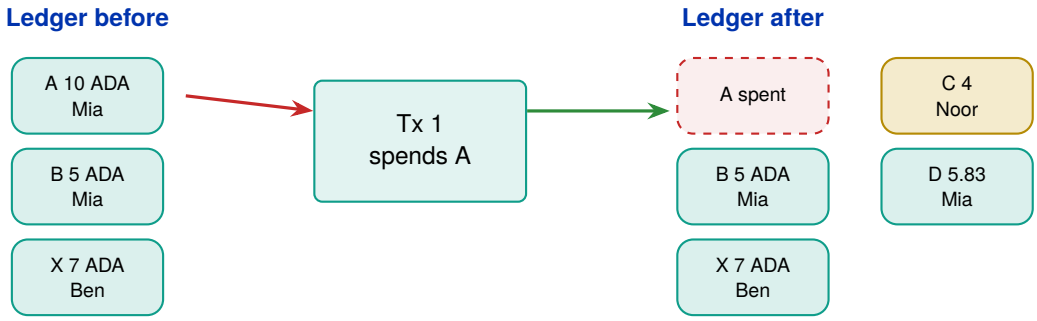


Figure 2.4: Each transaction rewrites a small part of the shelf of unspent outputs. A spent UTXO cannot be used again.

	Account model (e.g. Ethereum)	**(e)UTXO model (Cardano)**
State is	balance per account	set of unspent outputs
A payment	subtracts and adds balances	consumes inputs, creates outputs
Change output	not needed	needed
Two payments at once from one wallet	usually ok (ordered by a nonce)	can conflict if they pick the same UTXO
Fee known before sending	only approximately	exact (depends on tx size)
Failed tx still costs fee	often yes	mostly no: invalid txs are rejected before inclusion
Smart contract state	stored inside the contract	carried in each UTXO's datum
Mental picture	bank account	banknotes in a wallet

Table 2.2: Account model versus eUTXO.

2.5 Extended UTXO: datum, redeemer, validator

So far, every UTXO was owned by a key: only its owner's signature can spend it. The "extended" part of eUTXO allows a UTXO to be locked by a *script* instead. Three ingredients appear.

Validator (script). A small program attached to an address (a **script address**). It decides whether a transaction may spend a UTXO sitting at that address. It answers only *yes* or *no*.

Datum. Data stored *inside the locked UTXO*, such as "buyer = Bea, seller = Sam, deadline = Friday 18:00". It is the state of the agreement.

Redeemer. Data provided *by the spender* in the new transaction, such as "action = Release" or "action = Refund". It is the argument with which one tries to unlock.

The validator receives the datum, the redeemer and the transaction context (who signed, what outputs are created, what time it is) and returns allow or deny.

Everyday analogy: The safety-deposit box

The locked UTXO is a deposit box at the bank. The *datum* is the label with the agreement written on it. The *redeemer* is what you say when you arrive: "I am here to collect". The *validator* is the bank clerk who checks your papers against the label. No clerk can be bribed or changed: it is code that runs the same way for everybody.

2.5.1 Why this design matters

Because the validator runs on a transaction you have already built, you can predict the result before sending: if the check fails, the transaction is simply rejected. No surprises, deterministic costs. This

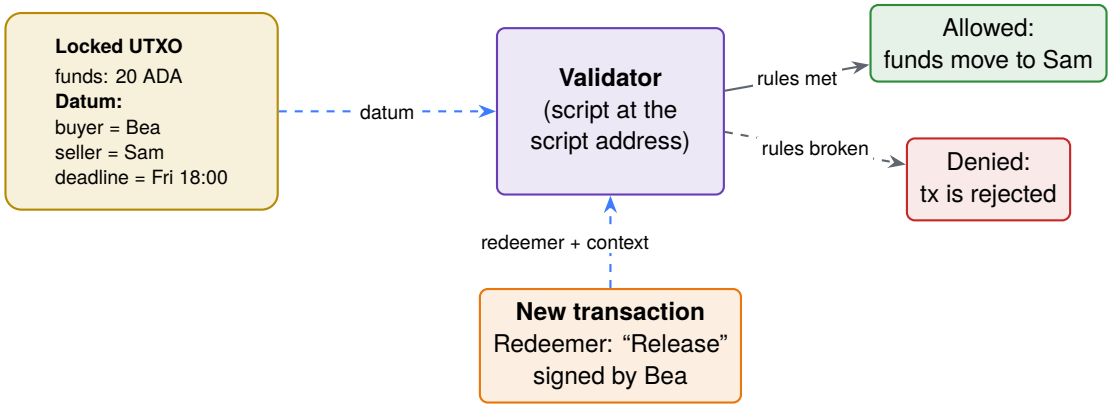


Figure 2.5: A validator checks datum, redeemer and the transaction before allowing the locked UTXO to be spent.

is one of Cardano's selling points.

Example: A tiny escrow

Bea wants to buy a result from Sam for 20 ADA.

1. **Lock:** Bea sends 20 ADA to the script address with the datum {buyer: Bea, seller: Sam, deadline: Friday 18:00}.
2. **Release:** when Sam delivers, a transaction with redeemer *Release*, signed by Bea (or by an agreed party), spends the UTXO and creates an output of 20 ADA to Sam.
3. **Refund:** if the deadline has passed and Sam never delivered, Bea can spend the UTXO with redeemer *Refund* and gets the 20 ADA back.

The script enforces both outcomes. Nobody needs to trust a middleman.

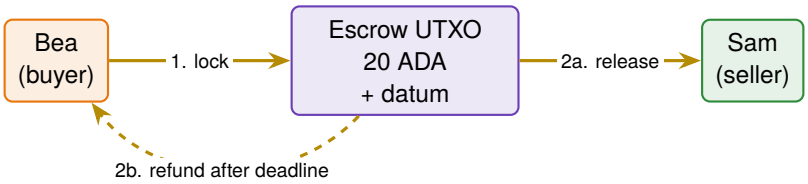


Figure 2.6: Escrow: the money waits in the script until the job is done or the deadline passes.

In Shelfie: Escrow comes from Masumi

Our MVP does not write its own validator. The brand's fee for the lead agent is held in a *Masumi escrow* (a ready-made contract, used through the Masumi Payment Service). After a worker's photo has passed our verifier, the engine pays the worker with a plain transfer from W-PAY. Writing a custom Aiken contract is only a possible stretch idea.

2.5.2 Plutus and Aiken

Validators are written in a smart-contract language and compiled to a low-level on-chain format.

- **Plutus**: Cardano's original contract platform, based on Haskell.
- **Aiken**: a newer, simpler language designed for validators, with its own CLI. The Mesh docs have a "first validator" guide covering datum, redeemer and signatures.

The off-chain code (building and submitting transactions) is usually written in TypeScript with libraries such as Mesh or Lucid Evolution, while the *on-chain* part is the validator. We reuse existing on-chain contracts and write only off-chain TypeScript. The research notes rate writing and debugging our own Aiken validator as a medium-high to high effort, so it is not in the MVP.

2.6 Native tokens

On Cardano, tokens other than ADA are **native assets**: the ledger itself knows how to hold and move them, just like ADA. You do *not* need a smart contract to create, hold or send such a token. A native token is identified by two parts:

- the **policy ID**: a hash identifying the minting rules,
- the **asset name**: a short label within that policy.

A UTXO can carry ADA and several kinds of native tokens at once.

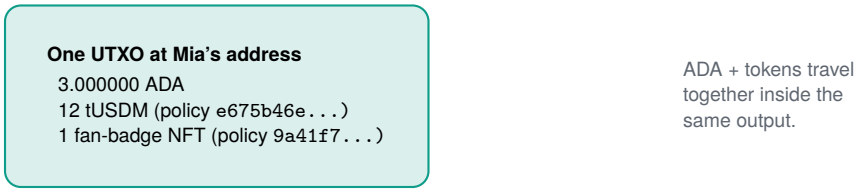


Figure 2.7: A UTXO may hold ADA and any number of native tokens. The second policy is invented for the illustration.

2.6.1 Example: tUSDM

tUSDM is a test version of a US-dollar stablecoin (USDM) used on the testnet. The x402 runbook found a trap: there are *two different* tUSDM tokens, one given by the SDK's test-token page (policy starting `e675b46e`) and one from the Masumi dispenser (policy starting `16a55b2a`). They have different policy IDs, so a wallet funded with one cannot pay a route priced in the other.

In Shelfie: Why we price in tADA

Pricing in lovelace (tADA) avoids the two-token mismatch completely. Decision ADR-03: "price and payouts in tADA, not tUSDM". The Masumi quote for the paid task, however, uses 1 test USDM, which belongs to the Masumi path and is handled by the Masumi service.

2.7 Fees and the minimum ADA

2.7.1 Transaction fees

Each transaction pays a fee, calculated from its size (and from script execution if scripts run). For simple payments the runbook quotes roughly **0.17 ADA per payment**. The fee is exact once the transaction is built, so you can display it before signing.

2.7.2 Minimum ADA per output

Every UTXO that exists on the ledger must carry a minimum amount of ADA, the **min-ADA** (also called min-UTXO). It prevents the ledger from filling up with worthless dust. The minimum grows with the size of the output (tokens, datum). For a simple ADA-only output it is about 1 ADA. The runbook states: "lovelace routes need at least 1 tADA (min-UTxO)".

Example: Why we cannot pay a worker 0.3 tADA

If Shelfie tried to send a tiny payout of 0.3 tADA, the output would hold less than the minimum and the transaction would be rejected. A payout must be *at least the minimum* (about 1 tADA for a simple output). Also, sending a token to someone requires attaching some ADA, which the sender provides. Our default price of 1 tADA per verifier check sits exactly at this floor, and "verify Friday that the facilitator accepts it, else use 2 tADA" is written in decision ADR-02.

2.8 Metadata

A transaction may carry optional **metadata**: a small piece of structured data (text, numbers, lists) attached to the transaction and stored forever on-chain. It does not change who owns what. Typical uses: invoice numbers, tags, references, a note.

In Shelfie: The payout tag

Every worker payout carries metadata such as `shelfie:<payoutId>`. If the app crashes after sending but before recording the hash, the reconcile step searches the chain for that tag and finds the earlier transaction, so that exactly one payout is made. Whether the Evolution SDK can attach metadata to plain transfers is still marked **verify Friday** (Task B, before 13:00).

Metadata is public; never put personal or secret data there.

2.9 Networks: mainnet, preprod, preview

Cardano runs several independent networks. Their ledgers are separate, and coins from one are useless on another.

Network	Coin	Purpose
Mainnet	ADA (real)	the production network with real money
Preprod	tADA (free)	stable test network that imitates mainnet; **this is what the hackathon and Shelfie use**
Preview	tADA (free)	test network where new protocol features arrive first; less stable

Table 2.3: Cardano networks.

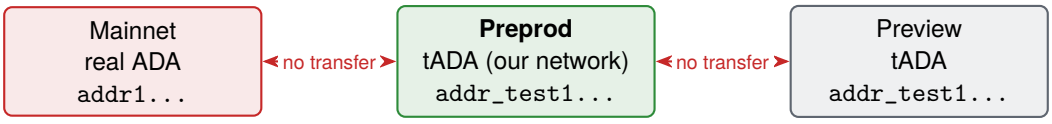


Figure 2.8: Three separate networks. Preprod and preview addresses look alike, so always check the network setting in your tools.

✗ Common pitfall: Sokosumi MCP defaults to mainnet

The Masumi runbook says: "Confirm Preprod; mainnet default" for the Sokosumi MCP server. Also use the `-preprod` flag in the Sokosumi CLI. In code, the network string in the x402 template is `cardano:preprod`.

2.10 Getting test ADA

2.10.1 The Cardano faucet

The official **faucet** (documented under docs.cardano.org, "Cardano testnets") gives free tADA to a testnet address. The limits per claim are not stated in our notes.

2.10.2 The Masumi dispenser

The **Masumi dispenser** (dispenser.masumi.network) hands out test ADA and test USDM for the Masumi path. Facts from the runbooks:

- it needs a **verification code from the Masumi registration email**;
- it offers an optional separate **5 ADA collateral** claim;
- you must confirm it is on **preprod**;
- funding of the Masumi seller wallet may take about 15–30 minutes to show up;
- per-claim amounts and limits are not documented: ask the mentors (open question).

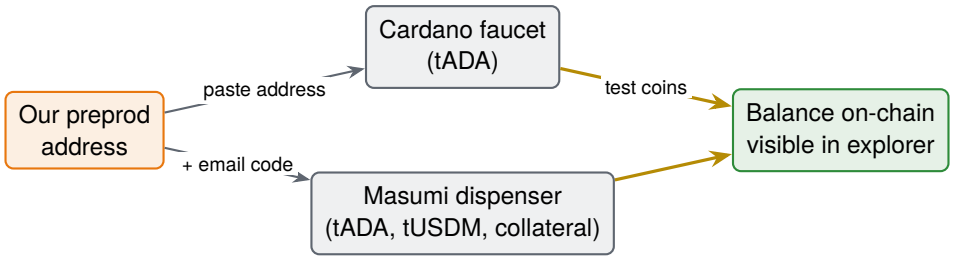


Figure 2.9: Two sources of free test funds. Both send coins to our address; we then see them in the explorer.

2.11 Blockfrost: reading and submitting without running a node

To talk to Cardano you normally need a full **node**: a program that downloads the whole chain (many gigabytes) and stays in sync. That is far too heavy for a hackathon laptop.

Blockfrost is a hosted API service: you send simple HTTP requests (with an API key, called a *project ID*) and Blockfrost answers with chain data, or submits your signed transaction for you. Free accounts exist; creating one is a human step on blockfrost.io, and one project is tied to one network (we need a **preprod** project).

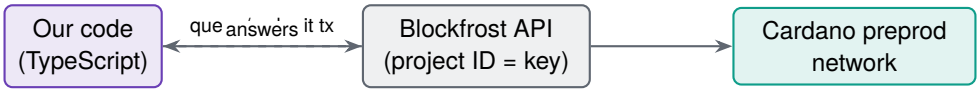


Figure 2.10: With Blockfrost we do not run a node: it relays reads and transaction submissions.

What we use it for: finding UTXOs of a wallet, building and submitting a transaction, checking whether a payment is confirmed. Notes from the runbooks:

- the x402 template needs `BLOCKFROST_PROJECT_ID`; Masumi uses `BLOCKFROST_API_KEY_PREPROD`;
- the facilitator's `/verify` and `/settle` calls use Blockfrost quota;
- **a Blockfrost key in browser code is visible to everyone**, so keep it on the server;

- never commit the key to Git.

2.12 Cardanoscan: reading a transaction

Cardanoscan is a block explorer for Cardano. The preprod site is `preprod.cardanoscan.io`. A transaction page is reached with

`https://preprod.cardanoscan.io/transaction/<tx hash>`

and an address page (inferred from the same host) with

`https://preprod.cardanoscan.io/address/<address>`

The Masumi runbook asks us to paste the receipt's `txHash` into Cardanoscan, inspect the confirmed seller output, and save the resulting URL as evidence.

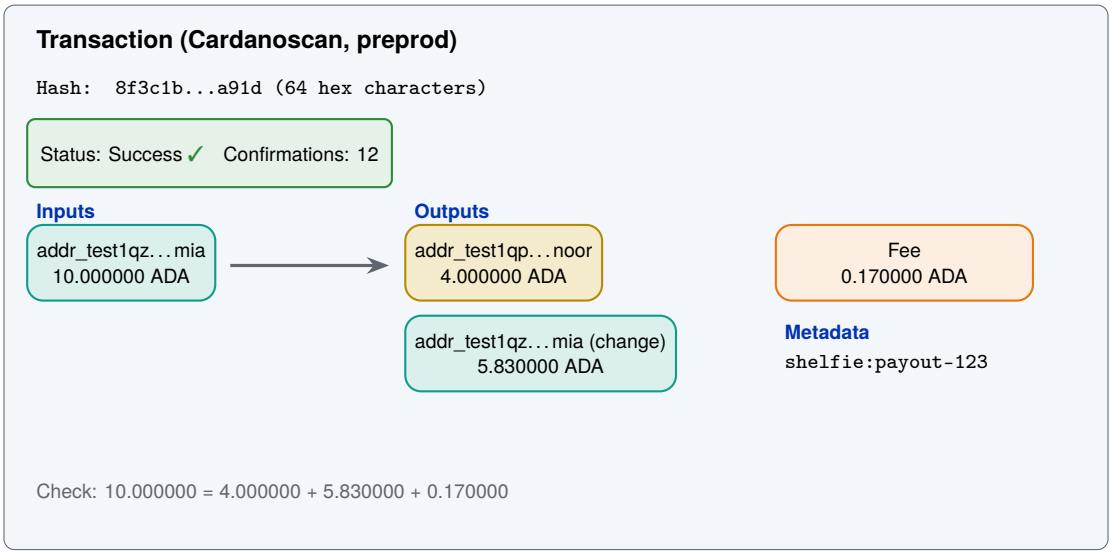


Figure 2.11: A mock-up (not a real screenshot) of what a Cardanoscan transaction page shows. All values are invented.

2.12.1 How to read it, field by field

1. **Hash:** the transaction's ID. Copy it to share or to search.
2. **Status / confirmations:** "Success" and a growing confirmation count tell you the transaction is in the chain and how deep. Wait for several before treating a payment as final.
3. **Inputs:** the UTXOs that were consumed and who owned them.
4. **Outputs:** the new UTXOs: look for the *recipient's address* and the amount. The other output is usually the change going back to the sender.
5. **Fee:** what the network took.
6. **Metadata:** optional tags such as our payout tag.
7. Check the arithmetic: inputs = outputs + fee.

✗ Common pitfall: Pending is not failed

After an HTTP 402 or a dropped connection, a payment may still settle (runbook). Look at the buyer address on Cardanoscan before retrying, so that you do not pay twice.

In Shelfie: Receipts for judges

For every payout and verifier fee, the Shelfie ledger stores the tx hash and a Cardanoscan URL, so each statement in our demo ("worker paid") can be verified by clicking a link.

2.13 Wallet setup for the team

Each teammate who sends or receives funds needs a Cardano wallet on preprod.

1. **Generate a wallet.** The x402 template has a command (`npm run wallet`) that prints a fresh 24-word **mnemonic** and an **address** once. Copy them at once; they are shown only one time.
2. **Store the mnemonic safely.** Put it in a private `.env` (listed in `.gitignore`), never in chat or Git. Use throwaway wallets only (Chapter 1).
3. **Note the address.** It starts with `addr_test1...` on preprod.
4. **Fund it** from the faucet or the Masumi dispenser. Roughly 10 tADA covers many 2-tADA calls plus about 0.17 ADA fee each; the Masumi seller wallet needs about 20 tADA for registration (runbook).
5. **Wait 1–2 minutes**, then check the address on Cardanoscan.

Browser wallets (CIP-30 wallets that sign in a web page) are only needed for the demo UIs in the templates; our headless backend uses the mnemonic-based signer.

✗ Common pitfall: Two kinds of mnemonic prints

The Masumi Payment Service seed command also prints mnemonics. The runbook says: operator only, no recording or screen sharing.

2.14 Cardano in our project

Here is the map of what the team actually touches, and where.

2.14.1 Concepts at a glance

Item	How Shelfie uses it
Network: **preprod**	every wallet, payment and explorer link; no mainnet anywhere
tADA payouts	verifier fee (default 1 tADA per check) and worker payouts, as plain transfers
Blockfrost key	one free preprod project; used by the x402 SDK, the facilitator fallback, and the Masumi Payment Service
Cardanoscan links	saved with each payout/fee as proof; shown on the worker page and in the pitch
Metadata tag	`shelfie:<payoutId>` on payouts, for crash-safe reconciliation
Escrow	Masumi's contract holds the brand's fee until the result is delivered; we do not write a validator
Native tokens	optional: tUSDM exists in the Masumi quote path, but our own prices are tADA
Mnemonics	stored only in private `.env` files; never in Git, chat or screen shares

Table 2.4: Cardano concepts and their role in Shelfie.

2.14.2 The four wallets

The design names wallets by purpose. Keeping them separate avoids UTXO contention and makes money flows easy to read.

Wallet	Role	Explanation
`W-SELL`	verifier income	receives the x402 fee each time the engine buys a photo check from the verifier
`W-VBUY`	verifier buyer	the engine pays the verifier from here; one payment at a time (serial queue)
`W-PAY`	worker payouts	the engine pays workers from here after a photo passes; plain transfer, idempotent queue
`W-MPS`	Masumi seller wallet	used by the Masumi Payment Service (the seller side of the escrow); needs tADA and optional 5 ADA collateral

Table 2.5: The named wallets from the design.

Workers use their own preprod addresses (we do not hold their keys).

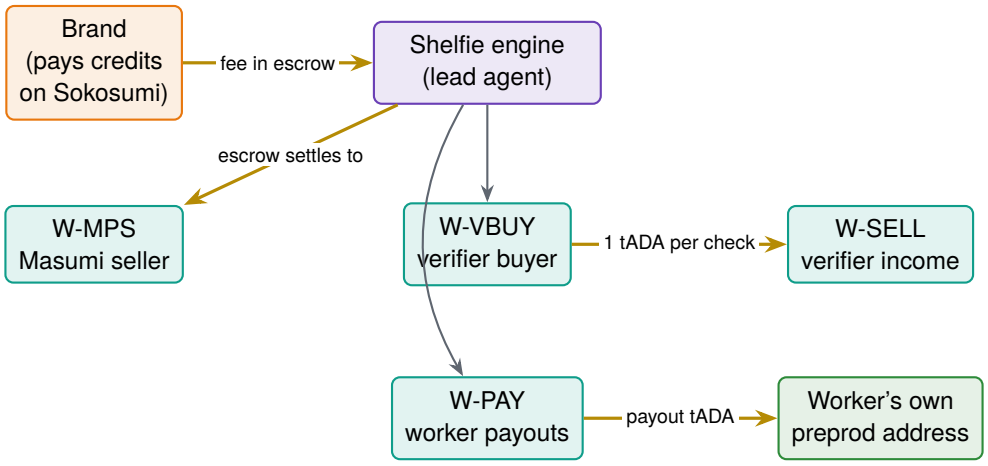


Figure 2.12: Simplified view of money flows between the named wallets. Blue/gold arrows are payments. The exact flow is explained in the architecture chapters.

2.14.3 What stays unverified

- Whether the x402 template's SDK can send plain transfers with metadata (verify Friday, Task B, first 90 minutes). Fallbacks: Lucid or Mesh isolated behind a payout adapter; or manual payout flagged in the demo.
- Whether the hosted facilitator accepts a 1-tADA price (else 2 tADA).
- Faucet and dispenser per-claim limits.
- Which explorer judges prefer (Cexplorer preprod URL form not confirmed).

★ Key takeaways

- **ADA** is the coin; **1 ADA = 1,000,000 lovelace**. On the test network the coin is **tADA**.
- Cardano uses **Ouroboros** (Proof of Stake); holders may **delegate** to stake pools; irrelevant for the hackathon.
- **eUTXO:** the ledger holds unspent outputs; a transaction consumes inputs and creates outputs; inputs = outputs + fee.
- A **change output** sends the rest back to the sender; one wallet cannot make two parallel payments reliably, so we use separate wallets and serial queues.
- A **validator** decides whether a locked UTXO may be spent, using the **datum** (state) and the **redeemer** (action); languages: Plutus, Aiken.

- **Native tokens** need no smart contract; tUSDM has two incompatible versions, so we price in tADA.
- Fees are about 0.17 ADA; every output needs about 1 ADA **min-ADA**; **metadata** lets us tag payouts.
- We use **preprod**; test funds come from the faucet or the Masumi dispenser.
- **Blockfrost** lets us read and submit without a node; **Cardanoscan** shows the receipt (hash, inputs, outputs, fee, confirmations).
- Team wallets: `W-SELL`, `W-VBUY`, `W-PAY`, `W-MPS`; mnemonics stay in private `.env` files.

? Check yourself

1. How many lovelace is 0.17 ADA? How many ADA is 3,500,000 lovelace?
2. Mia owns UTXOs of 8 ADA and 3 ADA. She pays Noor 5 ADA with a fee of 0.2 ADA. Which input does the wallet probably pick, and what is the change output?
3. In an escrow, what is the datum and what is the redeemer?
4. Why can't a Shelfie payout of 0.3 tADA be sent?
5. What does Blockfrost replace, and why must its key not be placed in browser code?
6. Name the four wallets in our design and one purpose each.

Answers. (1) 170,000 lovelace; 3.5 ADA. (2) The 8-ADA UTXO; change $= 8 - 5 - 0.2 = 2.8$ ADA (the 3-ADA UTXO stays untouched). (3) The datum is the stored agreement (buyer, seller, deadline); the redeemer is the spender's action (Release or Refund). (4) It is below the minimum ADA of an output (about 1 ADA). (5) Running your own full node; browser code is visible to everybody, so the key could be stolen. (6) W-SELL receives verifier fees, W-VBUY pays the verifier, W-PAY pays workers, W-MPS is the Masumi seller wallet.

Part II

Payments and agents

CHAPTER 3

HTTP 402 and x402

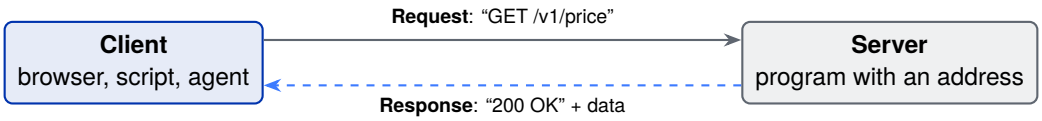
Why this matters for us

Shelfie sells exactly one thing to other software: a *verification* of a shelf photo, for 1 tADA per check. The way we sell it is x402, the Cardano track's "fastest path to a first payment". Our engine (a program) pays our verifier (another program) over plain HTTP, with no accounts, no API keys and no invoices, and every payment leaves a transaction hash that judges can open on Cardanoscan. To understand how that works you need three layers: (1) what HTTP is, (2) what the unused status code `402 Payment Required` was meant for, and (3) how the x402 protocol finally gives it a meaning. This chapter builds those layers from zero and ends with the exact numbers we plan to use.

3.1 HTTP from zero

3.1.1 Client, server, request, response

The web works as a conversation between two programs. The **client** asks (your browser, a phone app, a script, an AI agent). The **server** answers (a program running on some computer, listening at an address such as `shelfie.example`). Each question is called a **request** and each answer a **response**. **HTTP** (HyperText Transfer Protocol) is the agreed format of these messages, so that any client can talk to any server.



One request gets exactly one response. The server never calls the client first.

Everyday analogy: a restaurant

The client is a guest, the server is the kitchen, the request is the order slip and the response is the plate. The guest uses a fixed menu format (HTTP), so every kitchen understands every guest. If the kitchen needs a deposit before cooking, it should say so on the slip it hands back. That missing "deposit" answer is what HTTP 402 was reserved for.

3.1.2 Anatomy of a message

A request has four parts and a response has three:

- **Method**: what the client wants to do (`GET` = read, `POST` = send data to be processed).
- **URL / path**: which resource (`/v1/verify`).
- **Headers**: small labelled extra facts, one per line (`Content-Type: application/json`). x402 lives entirely in headers.
- **Body**: the payload (for us: JSON containing a photo).
- In the response, the **status code** is a three-digit number plus a short text that says how it

went.

method + path headers body

REQUEST

```
POST /v1/verify HTTP/1.1
Host: verifier.example
Content-Type: application/json
Idempotency-Key: verify:sub_01
{ "packId": "shelf-check", ... }
```

status headers body

RESPONSE

```
HTTP/1.1 200 OK
Content-Type: application/json
PAYMENT-RESPONSE: eyJ0cmFuc2...
{ "pass": false, "failedGates": [...] }
```

3.1.3 The status codes you must know

The first digit is the family: 2xx = success, 3xx = look elsewhere, 4xx = the *client* did something wrong or is missing something, 5xx = the *server* failed.

2xx Success	**3xx** Redirect	**4xx** Client side	**5xx** Server side

Code	Name	Meaning in plain words
200	OK	It worked, here is what you asked for.
301	Moved Permanently	The thing now lives at another URL; go there.
400	Bad Request	Your request is malformed (broken JSON, missing field).
401	Unauthorized	"Who are you?" Log in or send credentials.
403	Forbidden	I know who you are, but you may not do this.
404	Not Found	There is nothing at this address.
402	**Payment Required**	**You must pay first.** Reserved since the 1990s, now used by x402.
500	Internal Server Error	The server crashed or failed; not your fault.

Example: reading Shelfie's own error codes

In our verifier contract (`diseno/contracts.md`), errors before payment settles use `400 invalid_request`, `413 image_too_large`, `422 image_not_decodable` and `503 vision_unavailable`. Codes 413, 422 and 503 are standard HTTP codes that we simply reuse: "payload too large", "cannot process this content" and "service temporarily unavailable". The design goal is that these must *not* charge the buyer.

3.2 The history of `402 Payment Required`

When the HTTP status codes were designed in the 1990s, the authors expected money to move on the web soon. They set aside code `402` with the name "Payment Required" and the note "reserved for future use". The idea: a server could answer 402 to say "this resource costs money".

Then nothing happened. No standard defined *how* to say the price, *in which currency*, or *how* to prove payment. Credit-card networks, PayPal and later Stripe built their own checkouts on top of web pages instead. For about thirty years `402` stayed an unused code: browsers show it like any 4xx error, and a few services misuse it informally (for example "your subscription lapsed").



➤ Why 402 is attractive again

Three things changed. (1) Blockchains give a payment method that works without a bank, around the clock, in any amount. (2) Settlement takes seconds to a minute and costs cents or less, so tiny prices make sense. (3) The biggest new customers of the web are no longer humans with credit cards but *software agents*, which cannot fill in a checkout form. x402 is the protocol that turns the dormant status code into a working payment handshake.

3.3 Why paying per request matters for machines

Think about how a program normally buys a service today. A human signs up, enters a credit card, receives an **API key** (a secret password string), and the program sends that key in every request. That works, but it has costs that grow when the buyer is an autonomous agent:

- **Accounts and onboarding.** Someone must register, accept terms, and store the key. An agent that discovers a new service at 3 a.m. cannot do this alone.
- **Subscriptions.** A flat monthly fee is wasteful if you need 12 checks this week and none next week.
- **Minimum charges.** Card fees (often a fixed part of tens of cents plus a percentage) make a price of 1 cent or 10 cents impossible. These are **micropayments**.
- **Secrets management.** A leaked API key lets strangers spend your money.

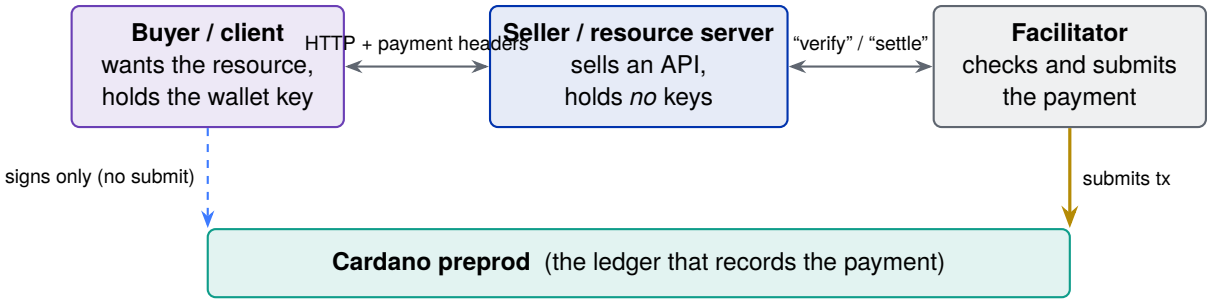
x402 flips this. The *request itself* carries the payment, one request at a time. The seller needs no list of customers and the buyer needs no account.

Everyday analogy: a vending machine versus a gym membership

A gym membership means registration, a contract and a monthly debit whether you use it or not (subscription + API key). A vending machine lets anyone walk up, put coins in and receive exactly one item (pay per request). The machine does not know you and does not need to. x402 turns every API into a vending machine, and the "coins" are a signed blockchain payment.

3.4 The x402 protocol

3.4.1 Roles

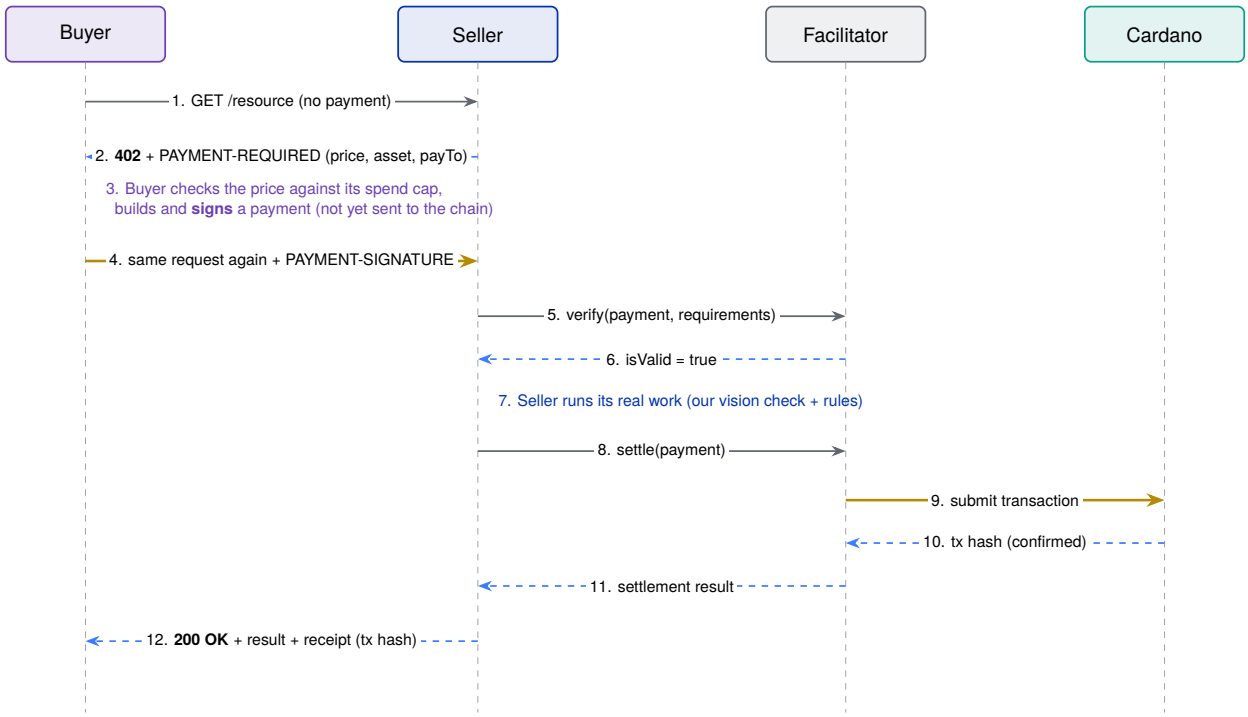


- The **buyer** (client) is the party that wants something. It holds a wallet key and *signs* the payment.
- The **seller** (resource server) owns the API. It only needs a receiving address, never a private key. This is why a seller cannot lose funds to a hack of its server.

- The **facilitator** is a helper service. It checks that a signed payment is valid (**verify**) and submits it to the blockchain (**settle**). It exists so the seller does not need blockchain code or a node connection. On Cardano the hosted preprod facilitator is **non-custodial**: it never holds the buyer's money or keys.

3.4.2 The full flow

The key trick is the *two-step request*. The buyer first asks without paying, learns the price from a 402 answer, signs a payment, and asks again with the payment attached.



Example: the same flow with numbers

A buyer calls `POST /v1/verify`. Step 2 returns: "price 1000000 lovelace, asset lovelace, network cardano:preprod, payTo addr_test1…". The buyer's cap is 2000000 lovelace, so it accepts (1 tADA $\leq$ 2 tADA). It signs a transaction moving 1 tADA to the seller's address and retries (steps 3–4). The facilitator confirms the signature and balance (steps 5–6). The seller does its work, the facilitator submits the transaction (step 9), and after 20–60 s the buyer receives the verdict plus the transaction hash (step 12). Network fee: about 0.17 ADA on top, paid by the buyer.

3.4.3 What is in the payment requirements

The 402 answer carries a small structured document (header `PAYMENT-REQUIRED`) that tells the buyer everything it needs to decide and pay:

Field	Example	Meaning
`scheme`	`exact`	Pay exactly the stated amount (no "up to").
`network`	`cardano:preprod`	Which chain; here the practice network.
`price.amount`	`1000000`	The price in the asset's smallest unit.
`price.asset`	`lovelace`	What to pay with: ADA (in lovelace) or a token.
`payTo`	`addr_test1...`	The seller's receiving address.
`description`	"One shelf-photo verification"	Human-readable purpose.
`mimeType`	`application/json`	Type of the thing you will get back.

Remember: 1 ADA = 1 000 000 lovelace, and on the test network the coin is called *tADA* (worthless test money from a faucet).

3.4.4 The three headers

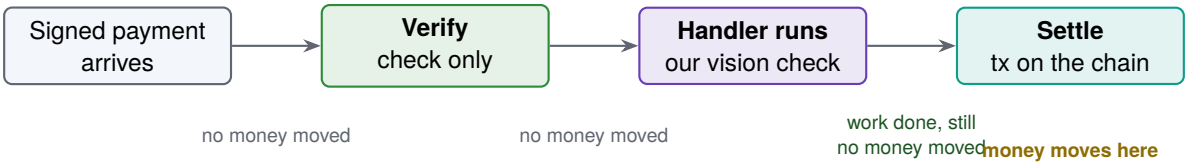
Header	Direction	Content
`PAYMENT-REQUIRED`	seller → buyer (with 402)	The payment requirements above.
`PAYMENT-SIGNATURE`	buyer → seller (retry)	The signed payment, encoded as text.
`PAYMENT-RESPONSE`	seller → buyer (with 200)	The receipt: success flag and the transaction hash.

3.4.5 Verify versus settle

These two facilitator steps are easy to confuse and matter for correctness.

- **Verify** is a *check only*: "Is this signature valid? Does the buyer have the funds? Does the payment match the requirements?" Nothing is sent to the chain, nothing is spent. A failed check arrives as HTTP 200 with `isValid:false` and an `invalidReason`; do not mistake that 200 for "paid".
- **Settle** actually submits the transaction and waits for the chain to confirm it. Only now has money moved.

In the Cardano express template the seller's paid handler runs *after verify and before settle*. This gives the seller a safe "go/no-go" point: it can refuse to do work for an invalid payment, and the buyer is charged only when settlement happens.



✗ Common pitfall: the handler can run more than once

Because retries happen (timeouts, dropped connections), the paid handler may be called again for the same payment. Our verifier therefore stores each verdict under an `Idempotency-Key` and, on a repeat, returns the cached answer without calling the vision model a second time.

3.4.6 What a 402 exchange looks like on the wire

To make this concrete, here is the exchange in simplified form. The header values are really long encoded strings; they are shortened here with dots.

```
--- first request (no payment) ---
POST /v1/verify HTTP/1.1
Idempotency-Key: verify:sub_01

--- first response ---
HTTP/1.1 402 Payment Required
PAYMENT-REQUIRED: eyJzY2hlbWUiOiJleGFjdCIs... (price, asset, payTo, network)

--- second request (same call, now with payment) ---
POST /v1/verify HTTP/1.1
Idempotency-Key: verify:sub_01
PAYMENT-SIGNATURE: eyJwYXlsb2FkIjoi... (signed payment)

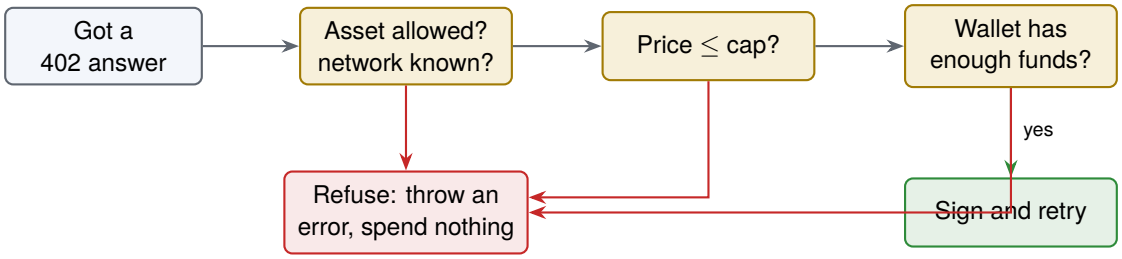
--- second response ---
```

```
HTTP/1.1 200 OK
PAYMENT-RESPONSE: eyJzdWNjZXNzIjp0cnVl... (receipt: transaction hash)
{ "pass": false, "failedGates": ["code_match"] }
```

The values are encoded (base64 text) so they fit safely in a header; the libraries encode and decode them for us. We never write this by hand.

3.4.7 How an agent decides whether to pay

A human reads a price and decides. An agent needs a rule. The buyer wrapper turns the human decision into a short, checkable procedure:



Every "no" ends in refusal rather than payment. This is exactly why the cap and the asset allow-list are not optional extras.

Example: a hostile or buggy seller

Suppose our verifier URL is mistyped and points to a stranger's server that answers 402 asking for 400 tADA. The wrapper compares 400 000 000 lovelace with the cap of 2 000 000, refuses, and the call fails with an error. Without a cap, one wrong string could empty the wallet. On the test network the loss would be only worthless tADA, but the habit has to be right before it ever touches mainnet money.

3.5 x402 on Cardano

The Cardano Foundation provides ready-made parts so we do not implement any of the above by hand. All of these are from the team's runbook (`investigacion/cardano/x402-runbook.md`); nothing has been executed yet, so treat them as *plan*, not as proven.

Part	What it is / how we use it
`@x402/cardano`	The Cardano payment scheme (`exact`), with both a client signer and a server verifier. Pinned at version 2.26.0 together with `@x402/core`, `@x402/express`, `@x402/fetch`.
Hosted preprod facilitator	Public, non-custodial, needs no key. URL in the template's `.env.example`. The public x402.org facilitator does *not* support Cardano. Fallback: run a local facilitator (`npm run facilitator`, about 100 lines, needs a Blockfrost key).
`x402-express` template	A small seller (Express server with payment middleware) plus a buyer script, both in TypeScript. Created with the `giget` command from the Cardano developer portal.
Headless buyer signer	`toClientCardanoSigner` signs from a 24-word *mnemonic* stored in `.env`. No browser wallet is needed; that is what lets a program pay.
`npm run wallet`	Generates a fresh mnemonic and address (shown once).
`npm run demo`	Starts the seller, runs the buyer once, stops. First paid transaction in about 30–60 minutes including keys and faucet (our estimate).
Blockfrost	A free API service that lets our code read and submit to the preprod chain; needs a free project id.

3.5.1 The buyer side: `fetchWithPayment`

In the buyer, the normal `fetch` function is wrapped so that it understands 402. In shape (simplified from the template):

```
const client = new x402Client().setSpendControls({
  allowedAssets: [{ network: "cardano:*", asset: "lovelace",
                    maxAmountPerPayment: "5000000" }] }); // cap
client.register("cardano:*", new ExactCardanoScheme(signer));
const fetchWithPayment = wrapFetchWithPayment(fetch, client);
const res = await fetchWithPayment(url, { method: "GET" });
```

The **per-payment cap** (`maxAmountPerPayment`) is a safety fuse: if a seller suddenly asks for 500 tADA the wrapper refuses to sign. Lovelace is rejected by default (it is not pegged to a currency), so the allow-list entry is mandatory. Our plan sets the cap to 2 000 000 lovelace for a 1 000 000 price.

3.5.2 The seller side

The seller registers one route with its price and receiving address; the middleware does the rest. The seller holds no keys, only `SELLER_ADDRESS`.

```
app.use(paymentMiddleware({
  "POST /v1/verify": { accepts: [{ scheme: "exact",
     network: "cardano:preprod",
     price: { amount: "1000000", asset: "lovelace" },
     payTo }], description: "One shelf-photo verification",
    mimeType: "application/json" } }, resourceServer));
```

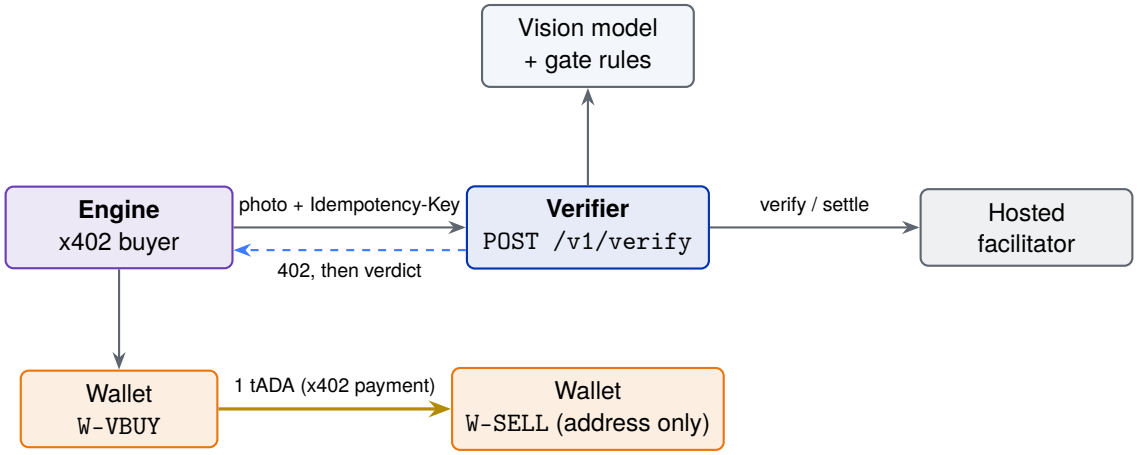
(The template's example route is `GET /api/message`; ours is the contract in `diseno/contracts.md`.)

✗ Common pitfall: limits to remember

Lovelace routes need a price of at least 1 tADA (the chain's minimum output size), so 1 tADA is our floor; if the facilitator rejects it we move to 2 tADA (*verify Friday*). One wallet cannot pay twice at the same moment, because the first coin it spends doubles as a unique nonce; we use one serial queue per wallet. Python has no supported Cardano x402 client, so the paying code stays in TypeScript. Two different test-USDM tokens exist, which is why we price in lovelace.

3.6 Worked example: the Shelfie verifier

Shelfie's engine receives a photo from a human worker. It must decide whether the photo proves the product is on the shelf. The decision is made by a separate *verifier service*, and we deliberately sell it through x402 so "agent pays agent" is visible on-chain.



3.6.1 Endpoint, price and environment

Item	Planned value (from `diseno/contracts.md`)
Paid endpoint	`POST /v1/verify`, protected by x402 seller middleware
Price	1 tADA = `VERIFY_PRICE_LOVELACE=1000000`, *per check, whether the photo passes or fails*
Asset / network	lovelace, `cardano:preprod`
Pay-to	address of wallet `W-SELL`, from env `SELLER_ADDRESS`
Buyer cap	`VERIFY_MAX_PRICE_LOVELACE=2000000` (2 tADA per payment)
Required header	`Idempotency-Key: verify:<submissionId>`
Free endpoints	`GET /health`, `GET /v1/price` (shows "verification costs 1 tADA per check")
Dev-only (no payment)	`POST /v1/verify-dry`; the engine refuses to count its verdicts as evidence
Buyer wallet	`W-VBUY` with env `VBUY_MNEMONIC_1`; facilitator URL in `FACILITATOR_URL`

3.6.2 Who pays whom

Three wallets are kept separate: `W-VBUY` pays the verifier, `W-SELL` receives verifier fees, and `W-PAY` pays workers (a plain transfer, not x402; see Chapter 7 of this guide). Money for one accepted task, in tADA (our estimate): 1 (verify) + about 0.17 (network fee) + 3 (worker payout) + about 0.2 (payout fee). A rejected attempt costs about 1.17, because the check is paid even when it fails.

Example: one paid check, step by step

Worker Anna uploads a photo for task `tsk_01` (store A, expected code K7Q-4XM). The engine asks the verifier; it answers 402 with the 1 tADA requirement. `W-VBUY` signs and retries. The verifier checks the photo; the code gate fails ("K7Q-4XN" instead of "K7Q-4XM"), so the verdict is `pass:false`. The payment still settles: the 1 tADA arrives at `W-SELL`, and the engine stores the transaction hash in its ledger with the verdict. Anna sees a hint to retake the photo; no payout is made. What the judges see: a Cardanoscan link for the check, even for the rejected photo.

In Shelfie: why pay-per-check suits the track

Every verification, pass or fail, is a visible, verifiable on-chain payment and a ledger line. It proves that one agent (the engine) pays another (the verifier) with no accounts. The cost is honest and stated: rejects cost money ("verification spend"). The alternative of calling the verifier as a function in the same program (rejected as ADR-02) would be cheaper but would remove exactly the proof the track asks for. The Masumi-escrow variant of x402 was also considered and postponed (4+ hours of work, only if time remains).

3.7 Pitfalls

1. **Paying for failed checks.** By design the fee is per check. Mitigate with clear photo guidance, a retry limit (`MAX_ATTEMPTS`) and free pre-checks (blur, size) before the paid call. Report the spend openly.
2. **Errors that should be free.** Invalid input (400/413/422) and an unavailable vision model (503) should return *before* settlement. Where exactly a handler error lands relative to settle in the template is a *verify Friday* item.
3. **Spend caps.** Always set `maxAmountPerPayment`. A bug or a hostile seller must not be able to drain the buyer wallet. Use throwaway wallets with only the test money needed (about 100 tADA each for `W-VBUY` and `W-PAY`).
4. **Double payment.** If a call times out after about a minute, the payment may still settle. Do *not* blindly sign again: first search `W-VBUY`'s history on Cardanoscan for a payment to `W-SELL` for this idempotency key, and only pay again if none exists.
5. **Idempotency.** The paid handler can run on retries; make side effects (calling a metered vision model) happen only once per key.
6. **Concurrency.** One wallet at a time; throughput is roughly one check per 20–60 s per wallet, so the worker page shows queue positions and the demo staggers uploads.
7. **Facilitator trust.** The facilitator reports whether a payment is valid and settled. A hosted one is a third party: if it is down, the seller answers 500 ("no supported payment kinds"); if it lies, the seller might give work for free or the buyer might be charged without service. Mitigation: it is non-custodial (cannot take funds), we can check hashes on the chain ourselves, and we can run a local facilitator as fallback.
8. **Secrets.** Mnemonics live only in untracked `.env` files; never in the repo or a browser.

3.8 x402 versus the alternatives

	x402 on Cardano	API key + subscription	Credit card checkout
Who can buy	Anyone with a wallet, including agents	Registered customers	Humans with a card
Sign-up needed	✗	✓	✓
Secret to protect	Wallet key (capped by spend limit)	API key (unbounded spend)	Card data
Smallest sensible price	Around 1 tADA on Cardano (chain minimum); cents elsewhere	Flat monthly fee	Limited by card fees
Pay per request	✓	Usually ✗	✗
Chargebacks / reversal	✗(settled on chain)	Contractual	✓
Public proof of payment	✓(tx hash)	✗	✗
Third party in the middle	Facilitator (non-custodial)	Billing provider	Bank, card network
Automation by software	Native, one extra round trip	Needs prior human setup	Hard
Typical settlement	20–60 s in our estimate	Monthly invoice	Days

The honest trade-off: x402 is slower per call (a payment confirmation, 20–60 s in our estimate) and has no refund button, while an API key is instant but needs a human relationship first.

★ Key takeaways

- HTTP is request and response; status codes classify the answer. `402 Payment Required` was reserved in the 1990s but never standardised.
- x402 gives 402 a meaning: the seller answers with price, asset, pay-to and network; the buyer signs and retries with `PAYMENT-SIGNATURE`; the seller returns the result plus a receipt.
- Three roles: buyer (signs), seller (holds no keys), facilitator (verifies and settles, non-custodial on Cardano).
- Verify checks, settle moves money. The paid handler runs between them and can run again on retries, so it must be idempotent.
- On Cardano: `@x402/cardano`, hosted preprod facilitator, headless mnemonic signer, `x402-express` template, `fetchWithPayment` with a per-payment cap. All of this is planned, not yet built.
- Shelfie: 1 tADA per check, paid pass or fail, cap 2 tADA, wallets `W-VBUY` → `W-SELL`.

? Check yourself

1. What is the difference between a 401, a 403 and a 402 response?
2. Name the three roles in x402 and say which one holds no private keys.
3. In which order do verify, the seller's work and settle happen, and when does money actually move?
4. Why does the buyer wrapper need a `maxAmountPerPayment`, and what value do we plan?
5. Why is a Shelfie rejected photo still paid for, and how can the team limit this cost?
6. A buyer call times out after one minute. What must it do before paying again?

Answers. (1) 401 = not logged in, 403 = logged in but not allowed, 402 = must pay first. (2) Buyer, seller, facilitator; the seller holds no keys. (3) Verify, then the seller's work, then settle; money moves at settle. (4) A safety fuse against overpaying or hostile prices; 2 000 000 lovelace. (5) The price is per check and the verifier does the work either way; limit via retry limits, free pre-checks, clear guidance. (6) Check the buyer wallet's on-chain history for a payment with this key; pay again only if none exists.

CHAPTER 4

AI agents, agentic commerce, Masumi and Sokosumi

Why this matters for us

The Cardano track is called *Agentic Commerce on Cardano*. Its whole point is an **AI agent** that does a useful job and gets paid for it on Cardano. To enter, we must be able to explain what an agent is, why it needs money, and how two tools called **Masumi** and **Sokosumi** let an agent be hired, paid and refunded. This chapter builds those ideas from zero, reads the official track brief line by line, and ends with how each piece appears in Shelfie. HTTP 402 and x402 are only mentioned briefly here (Chapter 3 covers them); what "on-chain" really means is in Chapter 5.

4.1 Large language models in one page

A **large language model** (LLM) is a program trained on huge amounts of text to predict what text comes next. If you give it a question, it writes the most plausible answer, one small piece (a **token**) at a time. Well-known examples are the models behind ChatGPT, Claude and Gemini. An LLM has no memory of its own between calls, it does not browse the web by itself, and it can be confidently wrong.

4.1.1 Prompts

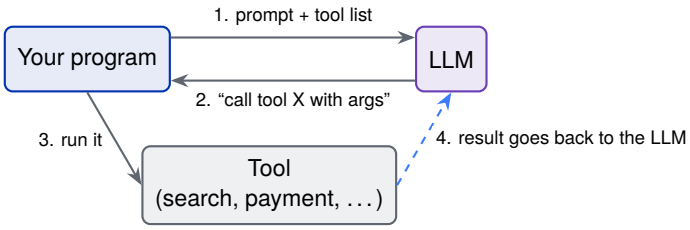
The text you send to the model is the **prompt**. It usually has two parts: a *system prompt* (the standing instructions: "You are a careful research assistant, always cite sources") and the *user message* (the actual request). The model's reply is the **completion**. Good prompts are specific, give the output format, and state what to do when information is missing.

Example: A tiny prompt

System: You extract prices from shelf photos. Answer only as JSON with the keys `product`, `price`, `code`. If a value is not visible, use `null`.
User: (a photo)
Reply: `{"product":"Oat drink 1 l","price":"1.89","code":"K7Q2"}`

4.1.2 Tool calling

On its own an LLM can only produce text. **Tool calling** (also called function calling) lets it *ask* for an action: the developer describes tools such as `search_web(query)` or `send_payment(address, amount)`; the model answers with a structured request like "call `search_web` with *bike shops Munich*"; the surrounding program runs the tool and feeds the result back; the model continues. The model never runs anything itself, the program around it does.

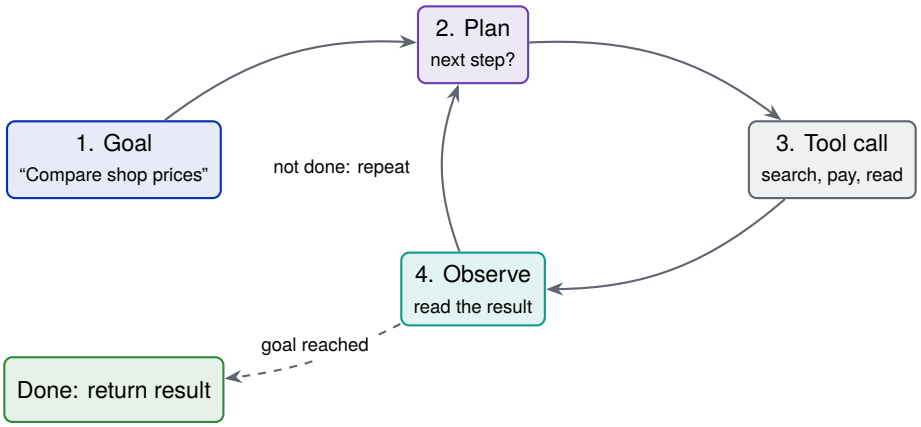


4.1.3 Vision models

A **vision model** is an LLM that can also take images as input. You send a photo plus a question and receive text. This is how a program can “read” a price tag, a receipt or a shop shelf. Vision models can misread blurry or small text, so a careful design lets the model only *read* and lets ordinary code *decide*. Shelfie follows this rule (Chapter 8).

4.2 From model to agent

An **AI agent** is a program that uses an LLM to pursue a *goal* over several steps, choosing and using tools on its own. Instead of one question and one answer, it runs a loop.



4.2.1 Agent, chatbot or script?

	Plain script	Chatbot	AI agent
Who decides the steps?	The programmer, fixed in advance	A human, one message at a time	The model, at run time
Uses tools?	Yes, hard-coded	Usually no	Yes, chosen by the model
Handles surprises?	No, it breaks	Answers in text only	Can retry or change plan
Loops until done?	Only if coded	No, waits for the user	Yes
Predictable?	Very	Medium	Least; needs guard rails
Example	Nightly price scraper	Support chat window	Research bot that searches, reads, pays for data and writes a report

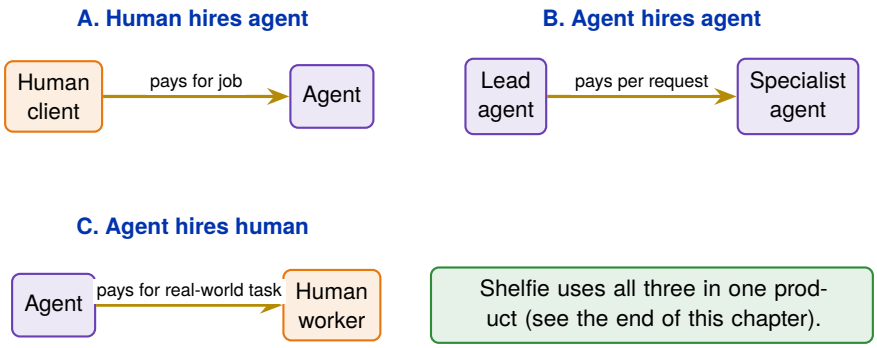
✗ Common pitfall: “Agent” is a loose word

In marketing almost anything is called an agent. For this guide: an agent has a goal, a loop, and tools with real effects (spending money, sending messages). The more real effects, the more guard rails it needs. The best agent design is often a mix: the model handles the fuzzy part, plain code handles the checks and the money.

4.3 Agentic commerce

Agentic commerce means software agents taking part in buying and selling: an agent *sells* a service (writes a report, answers a data query), an agent *buys* a service (pays for a dataset or a specialist), and sometimes an agent buys from or sells to humans. The track's own description is an agent, service or developer tool that does useful work and *charges for it on Cardano*.

4.3.1 Three patterns



Pattern C is the unusual one: a software agent can pay a person to do something it cannot, such as photographing a shelf in a physical shop. Cardano's brief explicitly mentions agents that "act in the real world by buying, publishing or booking".

4.3.2 Why an agent needs its own money, identity and receipts

A human pays with a bank card tied to a name and a monthly statement. An agent has none of that, and sharing a person's card with a program is dangerous. So an agent economy needs:

- **Its own wallet.** A cryptographic account the agent controls, funded with a small budget. No bank account or sign-up is needed.
- **An identity.** Others must know *which* agent they are dealing with, so that reputation and responsibility attach to something. Masumi stores agent registrations on-chain (below).
- **Receipts.** A payment that anyone can verify later: who paid whom, how much, for which job. On a blockchain every payment is automatically such a receipt.
- **Spending limits.** The agent should never be able to spend more than a cap per payment and per job.
- **Conditional payment.** For a job that takes time, money should be held until the work is delivered (*escrow*, explained below).

Everyday analogy: Company credit card with a limit

Give an intern a company card with a 50 EUR limit and receipts required. The intern can buy supplies without asking you each time, and the worst case is 50 EUR. A wallet with a spending cap works the same way for an agent.

4.3.3 Risks

Risk	What can go wrong	Typical countermeasure
Runaway spending	A loop keeps buying data; a bug or a prompt trick drains the wallet	Per-payment cap, small throw-away wallet, job budget
Fraud	A seller takes money and returns junk; a buyer claims non-delivery	Escrow with refund; automatic checks before release
Accountability	Nobody knows who is responsible for a bad action by an agent	Registered identity, recorded decisions and hashes
Prompt injection	Text on a web page tells the agent to pay someone	Code-level limits that the model cannot override
Bad output accepted as good	The LLM judges its own work too generously	Deterministic pass/fail rules, not model opinion

4.4 The track brief, explained

Source: the official prize page of the hackathon, copied into the team's research file of 29 October 2026. The table below restates it.

Item	What the brief says
Challenge	An agent, service or developer tool that does a useful job and *charges for it on Cardano*.
Prizes	Total 6,000 EUR: 1st **3,000 EUR**, 2nd **2,000 EUR**, 3rd **1,000 EUR**.
Two payment routes (may be combined)	**Per job: Masumi on Sokosumi.** The agent becomes a hireable "Coworker"; the payment waits in escrow until delivery, with a refund if nothing is delivered. **Per request: x402.** HTTP 402, no accounts or API keys; called "the fastest way to a first payment".
Hard requirement	A working agent or service that completes **at least one paid job or payment on Cardano preprod**, via Masumi or x402. (Preprod is the free test network; see Chapter 2.)
Extra to hand in	The **transaction hash** of that preprod payment; a video of at most 2.5 minutes saying who it is for, the problem, the **business case**, and one paid job from start to finish; a description of how it works, which Cardano tools it uses and how it could run in real life.
Judging criteria	Original use case; complete paid workflow; technical execution with Cardano tools; creativity; user experience; pitch and demo; real-world impact and continuity after the hackathon.
What Cardano especially wants to see	(1) A clear **customer and their journey**; (2) **payment tied to the result**: escrow, refunds, or paying only for work that passed a check; (3) **agents hiring other agents**, or agents acting in the real world; (4) a **live demo** with the transaction visible on a preprod explorer.
Judges	Bora Oben (Cardano) and Vladislav Kudrin (Cardano Foundation).
Timing	Hacking starts Fri 30 Oct, 09:00; submission deadline Sat 31 Oct, 15:00; live judging 15:00–17:00; winners 17:30.



Cardano track prize pool: 6,000 EUR

✗ Common pitfall: A completed task is not a payment

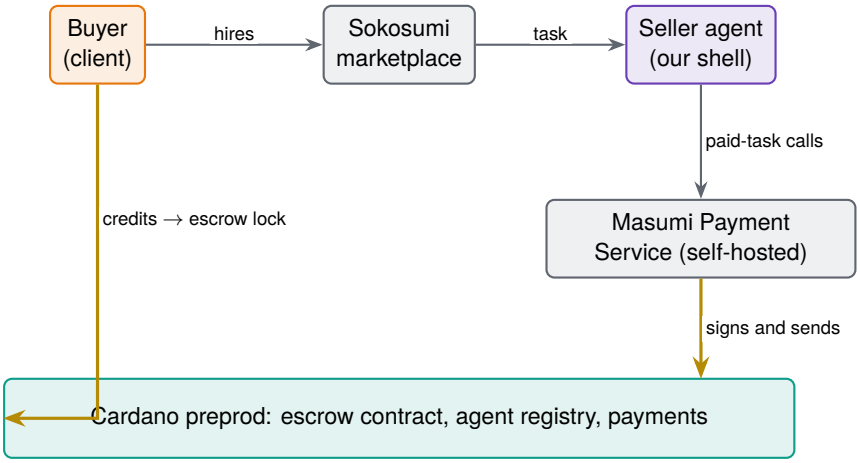
The Masumi guide warns that a Sokosumi task showing `COMPLETED` does *not* prove that a payment happened. We must show the seller's receipt and the transaction hash. Without the hash we have not met the hard requirement.

In Shelfie: Our route to the requirement

The team's plan treats x402 as the fast, low-risk way to qualify (one paid request) and Masumi escrow as the heavier second layer that earns the "pay on result" points. If the escrow is not working by Friday 22:00 it is frozen and shown as roadmap; the x402 payment still satisfies the requirement.

4.5 Masumi Network

Masumi is a network for agent payments and agent identity on Cardano. It gives agent developers the pieces that are hard to build alone: a way to get paid per job, hold payment in escrow, register an agent publicly, and keep tamper-proof records of what an agent did. Think of it as the payment and trust layer, while **Sokosumi** (next section) is the marketplace on top where people hire agents.



4.5.1 The Masumi Payment Service

The **Masumi Payment Service** (MPS) is the software that talks to the blockchain for an agent: it holds the seller wallet, registers the agent, watches for a buyer's escrow payment, submits result hashes and withdraws the earnings. The hackathon requires us to *run our own copy*; it is not a hosted service. According to the runbook:

- **Node.js** at least 24.22.1 and below 25 (Node 26 is not supported).
- **pnpm** version 10.30.2 (the package manager the project is pinned to).
- **PostgreSQL** 13 or newer (15 recommended) as its database; on a laptop with only the `psql` client, run Postgres in Docker.
- **Seeded wallets**: a seed script creates wallets and prints their secret recovery phrases once. Only the operator may see them; never record or commit them.
- A free **Blockfrost** preprod API key (the service that reads the chain) and secrets in a private `.env` file (database URL, encryption key of at least 32 characters, admin key).
- Default port 38127, with an admin page and API docs on the same address.

Test money comes from the Masumi dispenser (tADA and test dollar tokens). The runbook also mentions a quote of 1 test USDM for the sample task and a separate optional 5 ADA collateral from the dispenser; both are things to confirm on Friday.

4.5.2 Agent registry and decision logging

Before an agent can sell on Masumi it is **registered on-chain**. The registration creates a public record (an NFT, according to the research notes) that ties together the agent's identifier, its owner wallet (the seller verification key) and its payment settings. A buyer, a marketplace or a judge can look it up; nobody can quietly swap the identity. In the starter, a script registers the agent and you repeat a status read until it says `RegistrationConfirmed`; the confirmed identifier then goes into a state file.

Decision logging is the companion idea: instead of storing a whole report on the chain (expensive), the agent records a **hash** (a fixed-length fingerprint, Chapter 1) of its result. Anyone who later holds the report can recompute the hash and compare. If one character changed, the hashes differ. This gives an audit trail of "what this agent delivered" without putting the content on-chain.

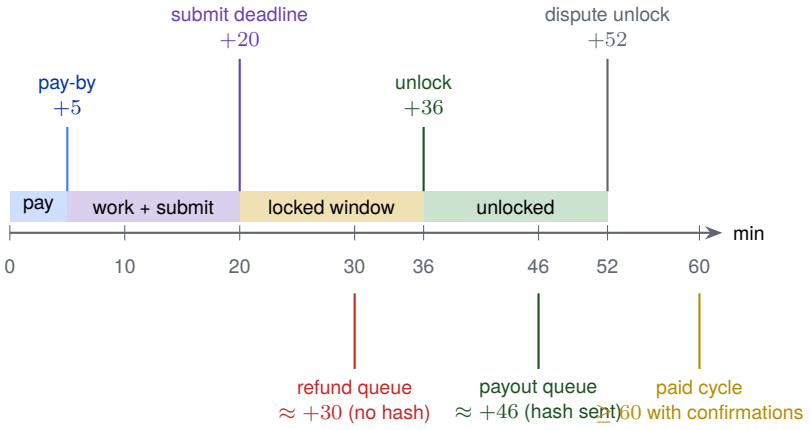
4.5.3 Escrow and its lifecycle

Escrow means a neutral holder keeps the buyer's money until the seller delivers. On Masumi the neutral holder is a smart contract on Cardano. The buyer locks funds (`FundsLocked`); the seller does the work and submits a result hash (`ResultSubmitted`); after a waiting period the seller can withdraw. If the seller submits nothing before the deadline, the buyer can get the money back.

Everyday analogy: Holding a deposit with a notary

You hire a photographer. You hand the fee to a notary. When the photos arrive, the notary pays the photographer; if they never arrive, the notary returns your money. Neither side has to trust the other, only the notary. Here the notary is code that nobody can bribe.

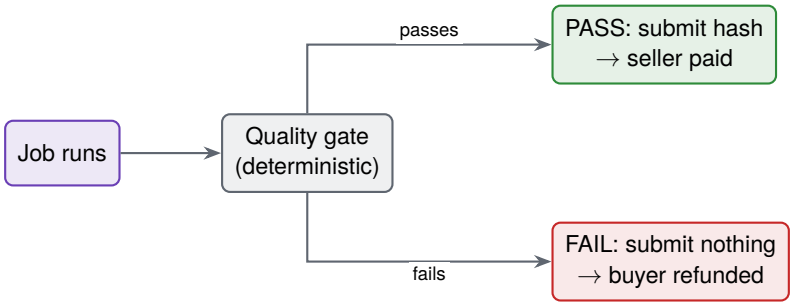
The contract uses several deadlines. The runbook gives these offsets (all *estimates* derived from reading the code, counted from the start of the job): pay by $+5$ min, submit the result by $+20$ min, unlock at $+36$ min, an external-dispute unlock at $+52$ min. Automatic seller withdrawal needs a result hash plus ten more minutes after unlock (about $+46$), and an automatic buyer refund applies if no hash exists ten minutes after the submit deadline (about $+30$). A complete paid cycle takes **at least 60 minutes** including on-chain confirmations.



All offsets are estimates from the runbook (counted from the start of the job). The exact reference point and whether the buyer-side refund runs automatically must be confirmed Friday with the Masumi engineer.

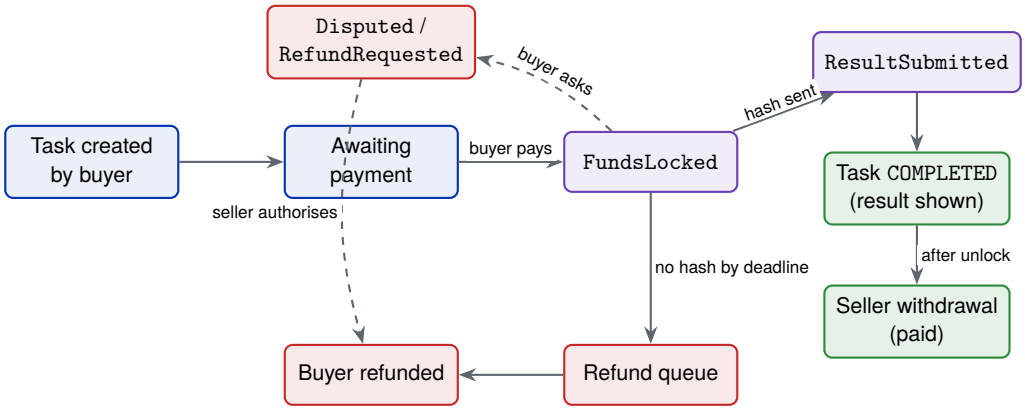
✗ Common pitfall: The crucial rule: any submitted hash pays the seller

The escrow cannot judge quality. If the seller submits *any* result hash, even for a text that says "failed", the contract treats the job as delivered and the seller can be paid. Marking the Sokosumi task as failed does *not* trigger a refund. To refund the buyer after a failed job, the seller must **withhold the result**: persist the failure locally, submit no hash, do not complete the task, and explain the reason in the task text. The buyer's no-result timeout then returns the money. The alternative, a manual refund request followed by the seller's authorisation, also exists but needs the buyer's side.



4.5.4 Seeing a Masumi job as a state machine

A **state machine** is a diagram of the states a thing can be in and the events that move it between them. This is a simplified view of one paid job. State names with a monospaced font appear in the runbook; the others are our plain-language labels.



Simplified: the real contract has more intermediate and error states (for example `Withdrawn`, pending confirmations, cooldowns).

4.5.5 The x402 path inside Masumi

Besides the Sokosumi credit route, the team's open questions list a variant where a headless agent pays *into* a Masumi escrow using x402, through a `/x402/start_job` path. We have not verified how it works; it is on the list of questions for the Masumi engineer on Friday. The design decision records note it as a 4+ hour option to attempt only if time remains. Treat it as unconfirmed.

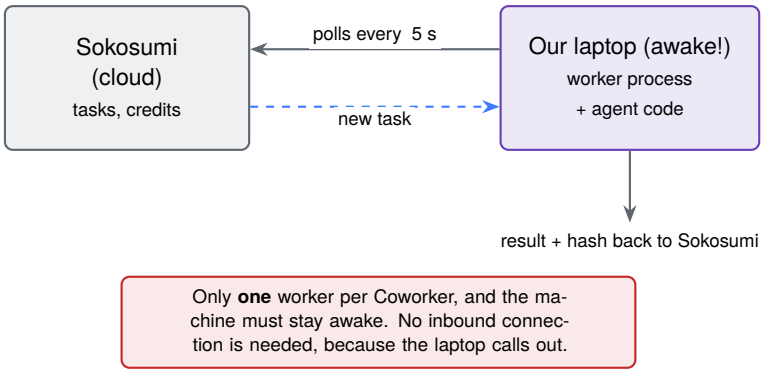
4.6 Sokosumi

Sokosumi is the agent marketplace in the Masumi world: a web app where people hire AI "coworkers" for tasks and pay in credits. It was launched by Serviceplan Group with NMKR; the public page lists eight coworkers (for example a research coworker and a coding coworker), mostly generic roles. For the hackathon there is a free test version at `preprod.sokosumi.com` and a shared

team space called the **TUM Workspace** that every participant must join.

4.6.1 Vendor, Coworker, worker

Concept	Meaning
Vendor	The company-like account that offers agents. One per account; it needs organisation membership first. In our plan it is our team's seller identity.
Coworker	The agent as it appears to customers: a name, a description, and the kinds of tasks it accepts. Ours is a *private* Coworker during development.
Worker process	The program running on a laptop that *polls* Sokosumi every few seconds for new tasks for the Coworker, runs the agent and posts the result. Start it with `npm run worker`.
Task	One request from a buyer to a Coworker. It moves through statuses such as `READY` and `COMPLETED`.



4.6.2 The `sokosumi` command-line tool

Each team member installs the official command-line tool, package `@masumi_network/sokosumi` version **1.0.4**, under Node 24. Typical first commands (preprod flag, as in the runbook):

```
sokosumi --preprod auth login
sokosumi --preprod auth whoami --json
sokosumi --preprod workspaces list --json
```

Logging in opens the browser. On Linux the tool stores its credentials in the operating system's secret store, so a keyring service (for example GNOME Keyring) must be running. A human login is different from the key the running agent uses (the runtime key is stored separately).

4.6.3 Credits versus on-chain payments

➤ Sokosumi credits are not x402

Buyers on Sokosumi pay with **credits**, an account balance inside the platform. When a paid Masumi job runs, the platform converts credits into the escrow lock on-chain. But if our lead agent hires *another* Sokosumi agent through the platform, it again pays in credits, which gives us no transaction hash we control. For the track we want a real on-chain payment between agents, so we use **x402** for that leg. The runbook states this directly: Sokosumi credits $\neq$ x402.

4.6.4 The starter and model keys

The organisers provide a starter repository called `tum-hackathon-quickstart`, pinned in the runbook to a specific commit. It contains the agent definition, the worker, the paid-task adapter and a settlement checker. Its agent needs access to an LLM provider, configured through environment variables: `ZAI_API_KEY`, `ZAI_BASE_URL`, `ZAI_MODEL` and optionally `ZAI_CONTEXT_WINDOW_TOKENS`. The

guide suggests Z.ai or Vercel AI Gateway; another provider means adapting the agent code. Two terminals are used: `npm start` (the agent runtime) and `npm run worker`, until the log says "Continuous worker running".

Example: From clone to first paid task (runbook order)

1. Install Node 24 and the Sokosumi CLI; sign up on preprod Sokosumi; join the TUM Workspace.
2. Create Vendor and private Coworker; connect it; import its key into the OS vault.
3. Run the agent and worker; create a smoke task; see `COMPLETED` (*no payment yet*, 5–15 min).
4. Start the Masumi Payment Service, fund the seller wallet from the dispenser (15–30 min), register the agent until `RegistrationConfirmed`.
5. Set `PAID_TASKS_ENABLED=true`, restart the worker. If registration is not ready it silently falls back to an *unpaid* task, so verify.
6. Run a fresh task: credits → `FundsLocked` → result hash → `ResultSubmitted` → `COMPLETED`, then fetch the receipt. Allow at least 60 min.
7. Paste the receipt's transaction hash into the preprod Cardanoscan explorer and save the URL.

4.7 What already exists: the competitive landscape

The team's research (public sources only, some pages unreachable) found this picture.

- **On the marketplace:** generic research, marketing and coding agents. The hackathon template and the track's own example (a research report for a shop) push every team towards a generic research agent, so this is the most crowded space. Likely empty (an inference, unverified): agents doing a checkable real-world task for a small customer, and agents that buy from other agents with a proof-of-result gate.
- **Past hackathons:** winners were infrastructure or a clear new commerce primitive (pay per tool call, getting past paywalls, guest checkout), showed a *real* transaction rather than a mock, and told a concrete story for a specific audience.
- **The x402 on Cardano path** is new: the Cardano Foundation contributed the Cardano support, and its hosted test facilitator has run only on preprod. Critics note there is little evidence of agents paying for real services in ADA at volume, so a visible real-commerce example is valuable.

Most teams will	How we can stand out (inference)
Build a generic research/summary agent on the template	Named customer with real pain and a price that fits
Show a "pay" button without a transaction hash	Outcome-gated payment with a recorded refund
Show only the happy path	A deterministic checker (rules) decides pass/fail
Pitch "the agent economy" in general	One agent hiring another with a spend cap, visible in a live ledger; explorer links in the demo

➤ The lesson we took

Lesson: escrow-with-refund works best when a *deterministic pass/fail gate* decides, not an LLM opinion.

4.8 How Shelfie uses each piece

In Shelfie: The lead agent: a Sokosumi Coworker with Masumi escrow

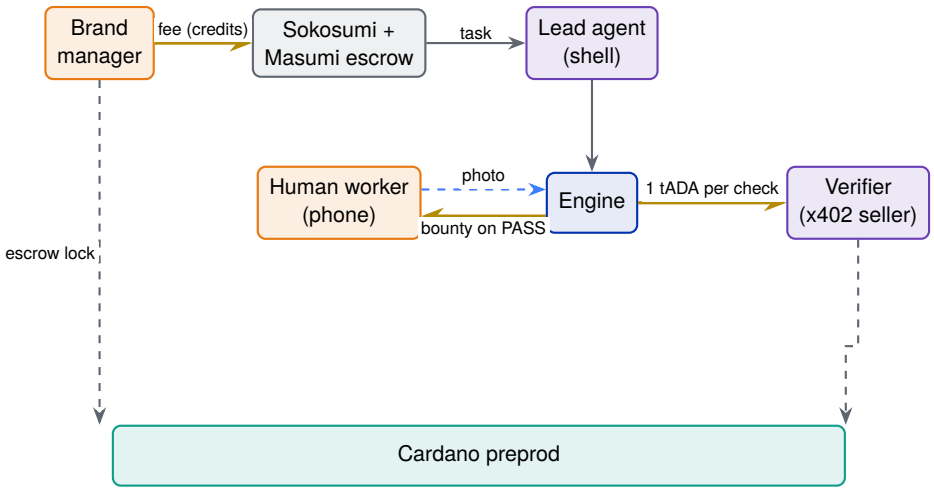
A brand manager hires Shelfie on Sokosumi ("check this product in N stores, at least K must pass"). The fee is locked in Masumi escrow. Our *shell* (a thin fork of the quickstart) runs the worker, receives the paid task and hands it to our engine. When the engine decides, the shell either submits the report hash (seller paid) or, on a failed K-of-N result, **withholds the result** so the brand is refunded after the timeout. The shell holds no business logic; it is one bridge file.

In Shelfie: The verifier agent: paid via x402

The verifier is a separate service that sells `POST /v1/verify` behind x402 at 1 tADA per check. Our engine pays it from a dedicated buyer wallet (with a per-payment spending cap) every time a worker submits a photo. Every check, pass or fail, is therefore a visible on-chain payment: agent pays agent. Sokosumi credits were rejected for this leg because they give no hash we control.

In Shelfie: Humans: paid by plain transfer

When the verifier passes a photo, the engine pays the human worker a bounty by an ordinary Cardano transfer from a payout wallet. Escrow per worker would take at least an hour and x402 would need a seller on every phone, so a plain transfer is the decision (with fallbacks if the library cannot build the transfer: to verify Friday).



Simplified overview: the photo actually goes from the worker page to the engine (Chapter 7 shows the exact flows). Pattern A = brand hires agent, B = engine/lead agent pays verifier, C = agent pays human.

Payment	Mechanism	Why
Brand → Shelfie	Sokosumi credits into Masumi escrow	Pay on result; refund if K-of-N fails
Engine → verifier	x402, 1 tADA per check	Agent-to-agent payment with a tx hash we control
Engine → worker	Plain Cardano transfer	Fast, cheap, no extra protocol

✗ Common pitfall: Be honest about the limits

Escrow settlement takes an hour or more, so on stage we show the live x402 and payout legs and present settlement and refund from earlier recorded runs, clearly labelled. No code is

built yet; everything above is the plan.

★ Key takeaways

- An AI agent is an LLM in a loop (goal, plan, tool call, observe, repeat) with tools that have real effects; it needs its own wallet, identity, receipts and spending limits.
- The Cardano track pays 3,000 / 2,000 / 1,000 EUR and has one hard requirement: at least one paid job or payment on **preprod**, via Masumi or x402, proven by a transaction hash.
- Masumi = payment and identity layer (self-hosted Payment Service, on-chain registry, escrow); Sokosumi = the marketplace where Coworkers are hired with credits.
- Escrow cannot judge quality: **any submitted result hash pays the seller**. A failed job must withhold the result so the buyer is refunded after the timeout.
- A paid Masumi cycle takes at least 60 minutes; plan for it and record refund and settlement runs before the demo.
- Sokosumi credits are not x402. Shelfie uses escrow for the brand, x402 for the verifier and plain transfers for humans.

? Check yourself

1. What are the four steps of the agent loop, and what makes an agent different from a script?
2. What is the single hard requirement of the Cardano track, and what must be handed in as proof?
3. In escrow, what happens if the seller submits a result hash for a failed job? How must a failed job be handled to refund the buyer?
4. Name the three things you need to run the Masumi Payment Service besides its code.
5. Why does Shelfie pay its verifier with x402 rather than with Sokosumi credits?
6. Why can the escrow settlement not be shown live on stage?

Answers. 1: goal/plan, tool call, observe, repeat; the model chooses the steps at run time. 2: at least one paid job or payment on Cardano preprod (Masumi or x402); the transaction hash. 3: the seller is paid; withhold the result (no hash, no completion) and let the no-result timeout refund. 4: Node 24.22.x with pnpm 10.30.2, a PostgreSQL database, seeded wallets (plus a Blockfrost preprod key). 5: credits give no transaction hash we control; x402 gives an on-chain payment between agents. 6: a paid cycle takes at least 60 minutes, so we show recorded runs with explorer links.

CHAPTER 5

On-chain versus off-chain

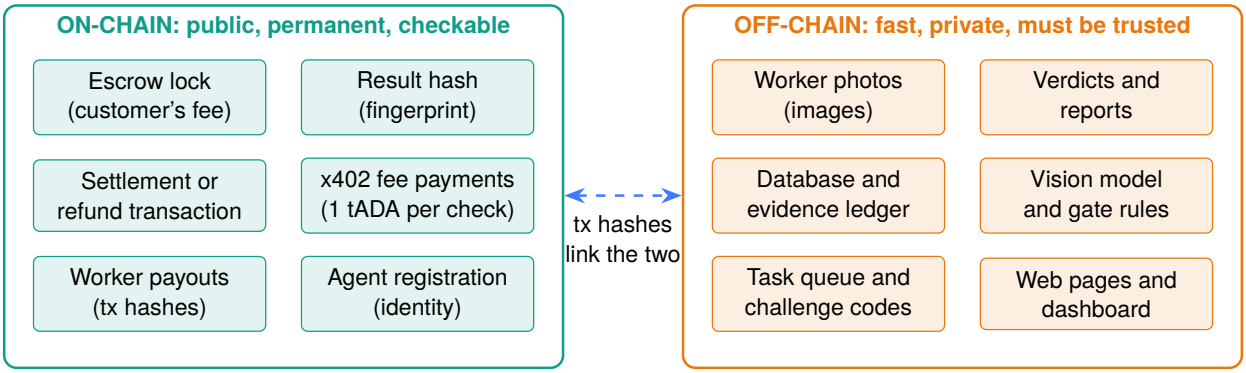
Why this matters for us

A common mistake in blockchain projects is to put everything on the chain, or nothing. Shelfie does neither on purpose. Photos, verdicts, the database and the vision model stay on our own servers; money movements and the escrow that protects the customer live on the Cardano chain. Judges will ask "what is actually on-chain?" and "who do I have to trust?". This chapter gives you the vocabulary and the honest answers, including the escrow timing facts that decide how we design the demo.

5.1 Two zones

➤ On-chain and off-chain

On-chain means recorded in the blockchain itself: every node stores it and anyone can read and check it. **Off-chain** means kept anywhere else: our servers, our laptops, a model provider's cloud. Off-chain data is controlled by whoever runs the machine.



5.1.1 Properties side by side

	On-chain	Off-chain
Who can read it	Everyone (public chain)	Only those with access
Can it be changed later	✗ (append-only history)	✓ (the owner can edit or delete)
Can a third party verify it without trusting us	✓	✗ (they must trust the operator)
Cost	Network fee per transaction (about 0.17 ADA on preprod)	Almost free
Speed	Seconds to minutes until confirmed; escrow flows take an hour or more	Milliseconds
Privacy	Poor: never put personal data or photos on a public chain	Good, if we protect it
Storage size	Tiny and expensive	Large and cheap

Everyday analogy: a courtroom ledger versus your notebook

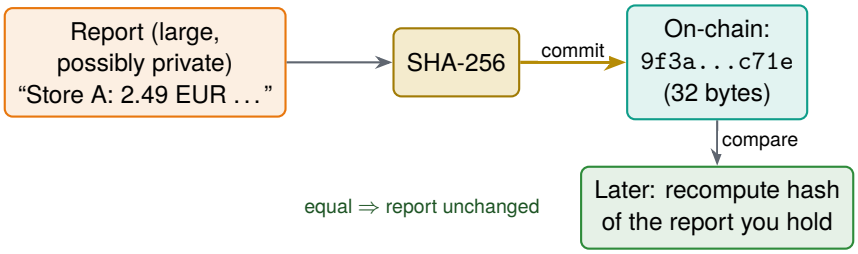
The chain is the official court ledger in a public hall. Everything written there is permanent, anyone may inspect it, but each entry costs a stamp fee and takes a while. Your notebook (off-chain) is fast, free and private, but nobody else can be sure you did not rewrite a page. Sensible people keep their drafts in the notebook and enter only the important, disputable facts in the ledger.

5.2 Hash commitments: proving without publishing

Often we want the permanence of the chain but not the size or privacy cost of the data. The standard tool is a **hash**: a function that turns any data into a short fixed-length fingerprint (for example 64 hexadecimal characters for SHA-256). Three properties matter:

- the same input always gives the same hash;
- changing even one character of the input changes the hash completely;
- you cannot recover the input from the hash.

A **hash commitment** puts only the hash on-chain. Later, anybody who holds the original data can recompute the hash and compare. If they match, the data is exactly what was committed at that time.



Example: a tampered report

Shelfie delivers a report saying "lowest price 2.49 EUR" and commits its hash `9f3a...` on-chain. Next week someone alters the file to say "2.19 EUR". The hash of the altered file is completely different, so the comparison with the on-chain value fails and the change is

exposed. The report itself never had to be public.

In Shelfie: where hashes appear

Our evidence ledger is a hash-chained, append-only file: each line stores the hash of the previous line, so editing history breaks the chain. Photos are identified by their SHA-256, which also catches the same photo being uploaded twice. In the Masumi flow the seller submits a *result hash* to the escrow; the full result text stays off-chain. Chapter 4 covers the Masumi side.

5.3 Escrow from zero

5.3.1 The problem

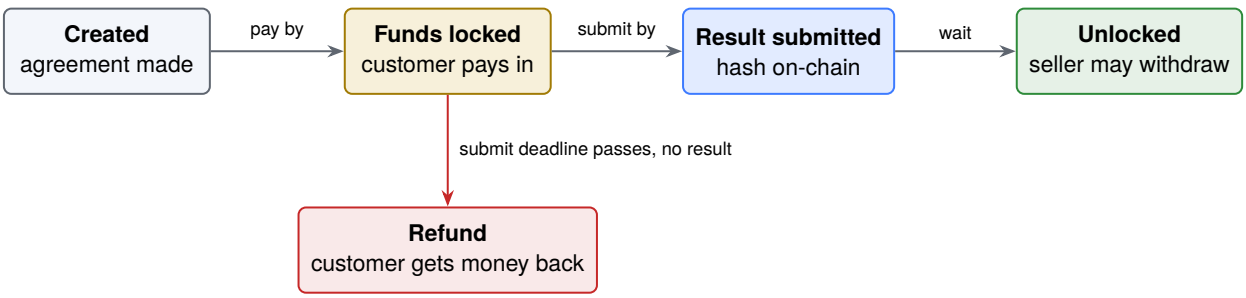
Customer and provider do not trust each other. If the customer pays first, the provider may disappear. If the provider works first, the customer may refuse to pay. An **escrow** solves this with a neutral holder: the customer deposits the money with the holder, the provider does the work, and the holder pays out only when the agreed condition is met.

Everyday analogy: notary and Treuhand

When you buy a flat in Germany, the buyer does not hand the price to the seller directly. A *Notar* (or a *Treuhandkonto*, a trust account) holds the money until the ownership is registered. Escrow is that, only the notary is replaced by a small program on the blockchain.

5.3.2 Smart-contract escrow

A **smart contract** is a program stored on the chain that holds funds and releases them only under coded rules, which nobody (not even the provider) can override. Such an escrow has a small state machine with deadlines (“timeouts”):



Three deadlines drive everything:

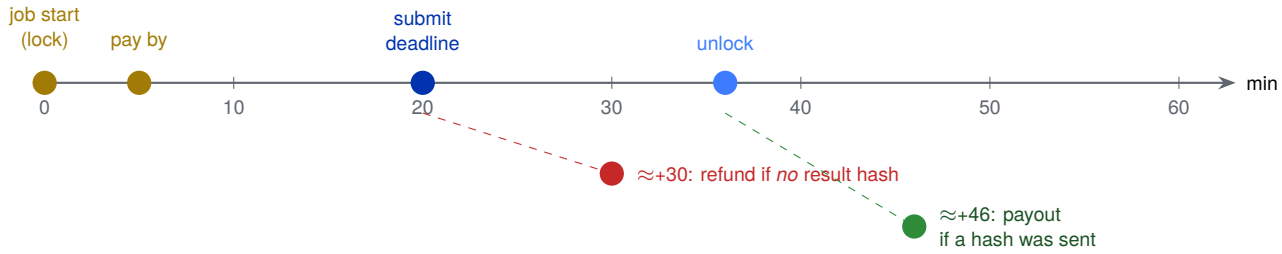
1. **Pay-by:** until when the customer must lock the funds.
2. **Submit-by:** until when the provider must submit a result (its hash).
3. **Unlock:** after a waiting period following the submission, the provider can withdraw the money. The waiting period gives the customer time to complain.

If the provider misses the submit deadline, the customer can get a **refund**. That refund path is what makes the customer comfortable paying before delivery.

5.3.3 The timing facts for Masumi (from the team's runbook)

These are estimates from the research notes (`investigacion/cardano/masumi-runbook.md`) and must be confirmed with the Masumi mentors on Friday.

Fact	Value
A full paid cycle (lock, result, settlement)	at least about 60 minutes
Provider payout eligible	about $+46$ min counted from the start of the job: unlock at $+36$ plus about 10 min queue (runbook inference; needs a result hash)
Refund eligible if no result hash	about $+30$ min from the start: submit deadline at $+20$ plus about 10 min (estimate; buyer automation to verify)
Official docs mention longer defaults	Up to about 12 hours in some documentation; the hackathon setup is shorter. Verify.

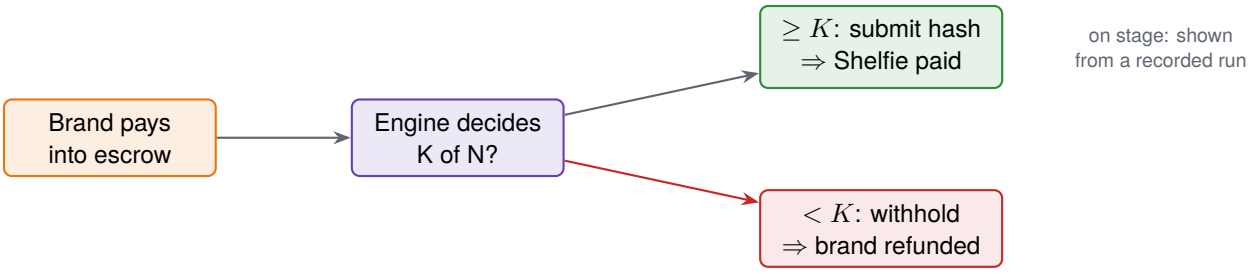


✗ Common pitfall: any result hash pays the seller

The escrow cannot judge quality. If the provider submits *any* result hash, even for a report saying "failed", the seller is paid after the unlock period. Marking the Sokosumi task as failed does *not* refund. To trigger the refund, the provider must **withhold** the result: submit no hash at all and let the deadline pass.

In Shelfie: the K-of-N rule and withholding

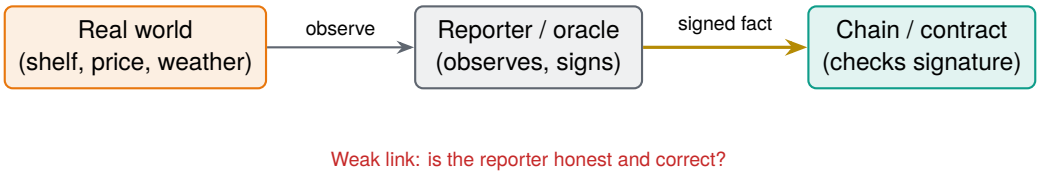
A brand hires Shelfie to check a product in N stores and requires at least K verified photos. The engine decides. If at least K pass, the shell submits the report hash and the seller is paid. If fewer than K pass, the shell submits *nothing* (ADR-11), so the brand's money flows back automatically after the timeout. Workers whose individual photos passed are still paid (ADR-09), which Shelfie bears as a stated cost.



This is why ADR-05 starts the first paid cycle on Friday afternoon, runs the failure/refund job on Friday night, and completes the last paid cycle by 13:00 on Saturday: the live stage can only show the verify and payout legs; settlement and refund are shown from recordings with explorer links, labelled as such.

5.4 Oracles and the oracle problem

A blockchain only knows what is inside it. It cannot look at a shelf, read a price on a website, or check the weather. An **oracle** is a bridge that brings real-world facts onto the chain. The **oracle problem** is that this bridge is a trust problem: if the chain accepts a fact, someone had to report it, and that someone could be wrong or lying. Decentralisation of the ledger does not decentralise the truth about the outside world.



➤ approaches to the problem

(1) Trust one named reporter and make it accountable (reputation, deposits, contracts). (2) Use signed data from the original source, so the contract verifies the signer, the request and the freshness instead of trusting a middleman. (3) Combine several independent reporters. None removes trust completely; they only make it explicit and checkable.

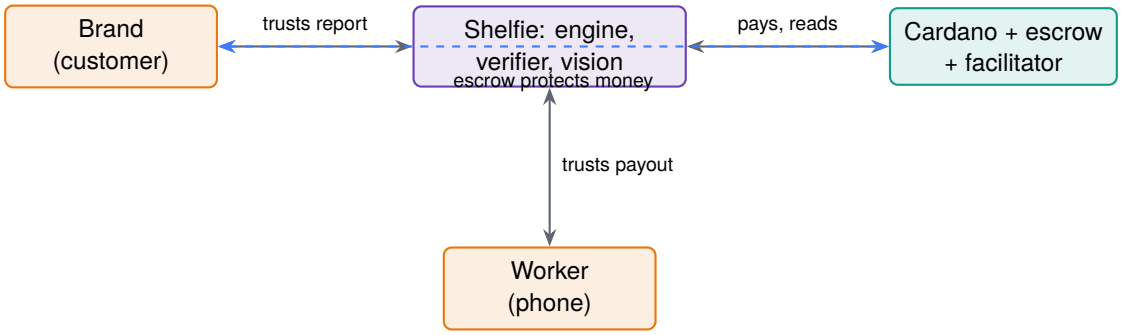
Example: the API3 hackathon track

This hackathon has another partner track, *API3 – Any API in your contract*. There the central feature must depend on authenticated data from AirnodeHub: a user brings the signed response on-chain and the contract verifies the expected signer, the intended request and the freshness before acting. Judges run a "tamper test" with one valid and four rejected responses (edited value, too old, wrong request, wrong key's signature). That is the oracle problem solved with signed data. We do not depend on it; entering API3 as a secondary track is undecided (`diseno/decisions.md`).

In Shelfie: Shelfie is a small oracle

The fact "this product was on this shelf at this time" is a real-world fact that the chain cannot see. Our verifier is the reporter. We do not claim to remove the trust; we make it small and inspectable: a random single-use challenge code in the photo shows freshness (not location, an honest limit stated in ADR-12), deterministic gate rules decide instead of a model's opinion, and an append-only evidence ledger plus payment hashes make each decision auditable.

5.5 Trust model: who must trust whom?



Party	Must trust	Why / protection	Residual risk
Brand (customer)	The verifier and engine to judge photos honestly; the chain	Escrow refunds if no result; rules are deterministic; ledger and hashes are auditable	We could still be wrong or biased; freshness is proven, location is not
Worker	The engine to pay verified photos	Payout appears as a tx hash on Cardanoscan; visible and permanent	A failed payout needs the queue and reconcile logic
Shelfie engine	The verifier's verdicts and the facilitator	Verifier is ours; payments are checkable on-chain	Vision model errors; provider outage (a flagged demo override exists)
Verifier seller	The facilitator to settle correctly	Facilitator is non-custodial; hashes can be checked on-chain	If down, no paid checks (fallback: local facilitator)
Everyone	Cardano preprod itself	Public ledger, many nodes	It is a test network; tADA has no value, and resets or outages are possible

5.6 What can go wrong: failure scenarios

Understanding the trust model is easiest by walking through bad days. The table lists what happens, what protects us, and what we still have to build or check.

Scenario	What the design does	Status / to verify
Brand pays but Shelfie never delivers	No result hash is submitted, so the escrow refunds the brand after the timeout	Timing is an estimate; verify with Masumi mentors
Job fails K-of-N but the shell submits a "failed" text	Seller is paid anyway (any hash pays)	Prevented by design: withhold the result
Verifier is paid but the vision model is down	Error returned before settlement where possible; engine retries with the same idempotency key	Exact order in the template to be verified Friday
Payout transaction times out	Queue marks it unknown, searches the chain for the metadata tag, pays again only if absent	Metadata support in the SDK is unverified
Worker uploads an old photo	Challenge code and timing checks reject it; the check still costs 1 tADA	Honest limit: freshness, not location
Hosted facilitator unreachable	Seller answers 500; fall back to a local facilitator (`npm run facilitator`)	Test before the demo
Preprod network is slow	Stage shows recorded settlement; the live part is verify and payout	Decided in ADR-05

5.6.1 Finality: when is "on-chain" really final?

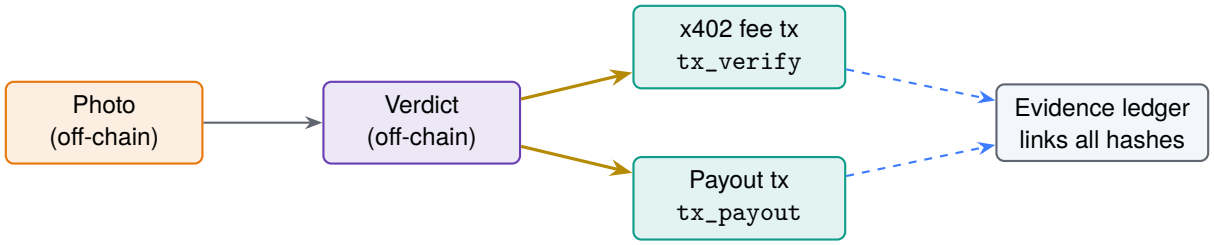
A transaction is not final the instant it is submitted. It is first accepted into a block and then becomes harder and harder to reverse as more blocks follow. The x402 default is one confirmation before the facilitator reports success; a faster mode with zero confirmations exists and hosted facilitators accept it, at the price of a small risk that the payment does not end up in the chain. For the demo this explains why one verify payment takes 20 to 60 seconds, and why a "timed out" call must be checked on the explorer before retrying.

Everyday analogy: a cheque versus cash in hand

Submitting a transaction is like handing over a cheque; the confirmation is the bank clearing it. You may deliver the goods when the bank says it is cleared, or earlier if you accept a little risk. Escrow adds a third option: the cheque is already deposited with the notary before you start work.

5.7 What is on-chain and off-chain in Shelfie

Item	On-chain	Off-chain	Notes
Brand's fee in escrow (lock)	✓		Masumi escrow; tx hash
Settlement or refund	✓		After the timing window; shown from a recording on stage
Result hash	✓		Fingerprint only; report text is off-chain
x402 verify fee, 1 tADA	✓		One transaction per check, pass or fail
Worker payout, 3 tADA (planned)	✓		Plain transfer, tx hash on Cardanoscan; metadata tag `shelfie:<payoutId>` planned (verify Friday)
Worker photos		✓	Large, personal; only SHA-256 hashes are used
Verdicts, gate results, reports		✓	Kept in the engine and ledger
Task queue, claims, challenge codes		✓	In-memory state plus event log on disk
Evidence ledger (`evidence.jsonl`)		✓	Hash-chained and append-only; links to tx hashes
Vision model and rules		✓	Provider is configuration (ADR-07)
Dashboard and worker page		✓	Served through an HTTPS tunnel



Each photo leaves a trail: off-chain evidence plus on-chain proof of payment

✗ Common pitfall: do not oversell "on-chain"

Do not tell judges that the verdicts are on a blockchain. The honest claim: money movements and the escrow are on-chain and independently checkable; the judgement itself is off-chain and is made auditable through rules, a hash-chained ledger and payment hashes. Also say clearly that settlement and refund on stage are from earlier recorded runs.

★ Key takeaways

- On-chain = public, permanent and checkable but costly and slow; off-chain = fast, private and cheap but requires trust in the operator.
- A hash commitment puts a fingerprint on-chain and keeps the data elsewhere; any later change is detectable.
- Escrow holds money until a condition is met; with deadlines (pay-by, submit-by, unlock) and a refund path it protects the customer without a human notary.
- Masumi timing (estimates): paid cycle at least 60 min, payout eligible at about +46

min and refund at about $+30$ min, both counted from the start of the job.

- Submitting any result hash pays the seller, so a failed job must withhold the result to trigger a refund.
- An oracle brings real-world facts on-chain, but trust moves rather than disappears. Shelfie is a small oracle for shelf facts and makes its trust explicit.
- Shelfie on-chain: escrow, x402 fees, payouts. Off-chain: photos, verdicts, database, ledger, vision model.

? Check yourself

1. Why should photos not be stored on a public blockchain?
2. What does a hash commitment prove, and what does it not reveal?
3. List the three deadlines of a timeout-based escrow and say what each protects.
4. A job fails the K-of-N rule. Which action leads to a refund and which action would wrongly pay the seller?
5. What is the oracle problem, and how is Shelfie an example of it?
6. Name two things Shelfie records on-chain and two that stay off-chain.

Answers. (1) Public, permanent, size-limited and expensive; privacy would be lost. (2) That data existing now matches the committed fingerprint, so later changes are detected; it does not reveal the data. (3) Pay-by (customer must fund), submit-by (provider must deliver), unlock (waiting period before the provider withdraws; time to complain). (4) Withholding the result triggers the refund; submitting any hash, even "failed", pays the seller. (5) The chain cannot see the real world, so someone must report facts and be trusted; Shelfie reports "product was on the shelf". (6) On-chain: escrow, x402 fee, payouts; off-chain: photos, verdicts, ledger, vision model.

Part III

Our idea: Shelfie

CHAPTER 6

Shelfie: the idea

Why this matters for us

The first five chapters gave you the tools: blockchain, Cardano, HTTP 402 and x402, agents with Masumi and Sokosumi, and the difference between on-chain and off-chain. This chapter is the first time we put them to work on *our* product. If you can explain Shelfie in two minutes to a stranger, to a judge, and to a friend who has never heard of blockchains, you have understood the project. Everything in chapters 7 to 9 (engine, verifier, hackathon day) builds on the story told here.

6.1 The problem: nobody can see the shelf

6.1.1 Retail execution in plain words

A small drinks brand sells its bottles to a supermarket chain. The contract is signed, the pallets are delivered to the warehouse. But does the bottle actually stand on the shelf in store number 7 in Munich? At which price? Is the shelf tag correct, or is the product hidden behind a competitor's? The work of making sure that a product arrives on the shelf as agreed is called **retail execution**. The people who do it for brands are called field-sales or merchandising staff, and many brands hire external companies for it.

Three words that you will meet again:

- **Retail execution**: making sure the agreed product, price and placement really appear in the store.
- **Field marketing** (or merchandising): sending people into stores to check, restock or promote products.
- **Mystery shopping**: a person visits a store, pretends to be a normal customer and reports what they see.

Everyday analogy: The restaurant delivery photo

You order food and the app says "delivered". You would like a photo of the parcel at your door, taken *now*, not last week. A brand ordering a shelf check has the same wish: "show me the shelf as it is today". Shelfie sells exactly that photo, and pays the photographer only if the photo is real.

6.1.2 What the options cost today (marked as inference)

Today a brand has three choices, none of them good:

Option	Problem	Status of the claim
Drive around yourself	An afternoon for 10 stores, and it does not scale.	Plain logic.
Field-marketing agency or mystery-shopping panel	Booked days ahead; paid per visit whether or not the photo is usable.	*Inference.* Price per visit and lead time still need two public sources.
Trust the chain's own sales data	It tells you what was sold, not what stands on the shelf.	Plain logic.

✗ Common pitfall: Do not state numbers we have not sourced

The team's notes carry the placeholder `[SOURCE NEEDED]` for the price of an agency visit, the typical lead time and the share of sales lost to missing products. In the pitch we only say "booked days ahead, paid per visit" unless we find sources. A made-up number can sink credibility with judges.

6.2 Our fictional customer: Katharina Brandl

Every pitch needs one concrete person. Ours is **fictional** and we say so whenever asked:

In Shelfie: Persona (invented for the story)

Katharina Brandl, field-sales manager at **Isarquell**, a small Bavarian craft-lemonade brand (invented, no real company). There are four people in sales. Last month Katharina sold a new 0.33 l bottle to a regional supermarket chain. Today she needs to know: *is it on the shelf in 10 Munich stores, and at which price, right now?*

Why a persona at all? The Cardano track judges explicitly want to see a *clear customer and journey*. A name, a job and a concrete question turn an abstract protocol into a story that people remember.

6.2.1 The job: "check product X in N stores"

Katharina does not want to learn about blockchains. She wants one sentence to work:

"Check Isarquell 0.33 l in 10 Munich stores."

She posts this job on Sokosumi (the marketplace from Chapter 4) to the *Shelfie lead agent*. The fee for the job is locked in Masumi escrow (Chapter 4). From her side that is all. What happens next is Shelfie's work, and it is the subject of the rest of this guide.

6.3 The three payment gates

Here is the heart of the product. Money moves in three places, and each place has a rule that must be satisfied first. The team's pitch calls them the **three payment gates**:

1. **Photo gate: pays the human.** The verifier agent checks one photo. If it passes, the person who took it receives a payout on Cardano preprod right away. If it fails, nothing is paid and the task reopens.
2. **Job gate: settles the escrow.** When all stores are done or the time window closes, the engine counts passed stores. If at least K of N passed, the report is delivered and the escrow settles.
3. **Fallback: refund.** If fewer than K passed, the report is withheld and the brand is refunded automatically.

✗ Common pitfall: Two kinds of "gates"

Do not confuse the three *payment gates* above with the seven *verifier checks* G1 to G7 (Chapter 8). The photo gate is *made of* G1 to G7. The pitch says "three gates"; the verifier spec says "G1 to G7".

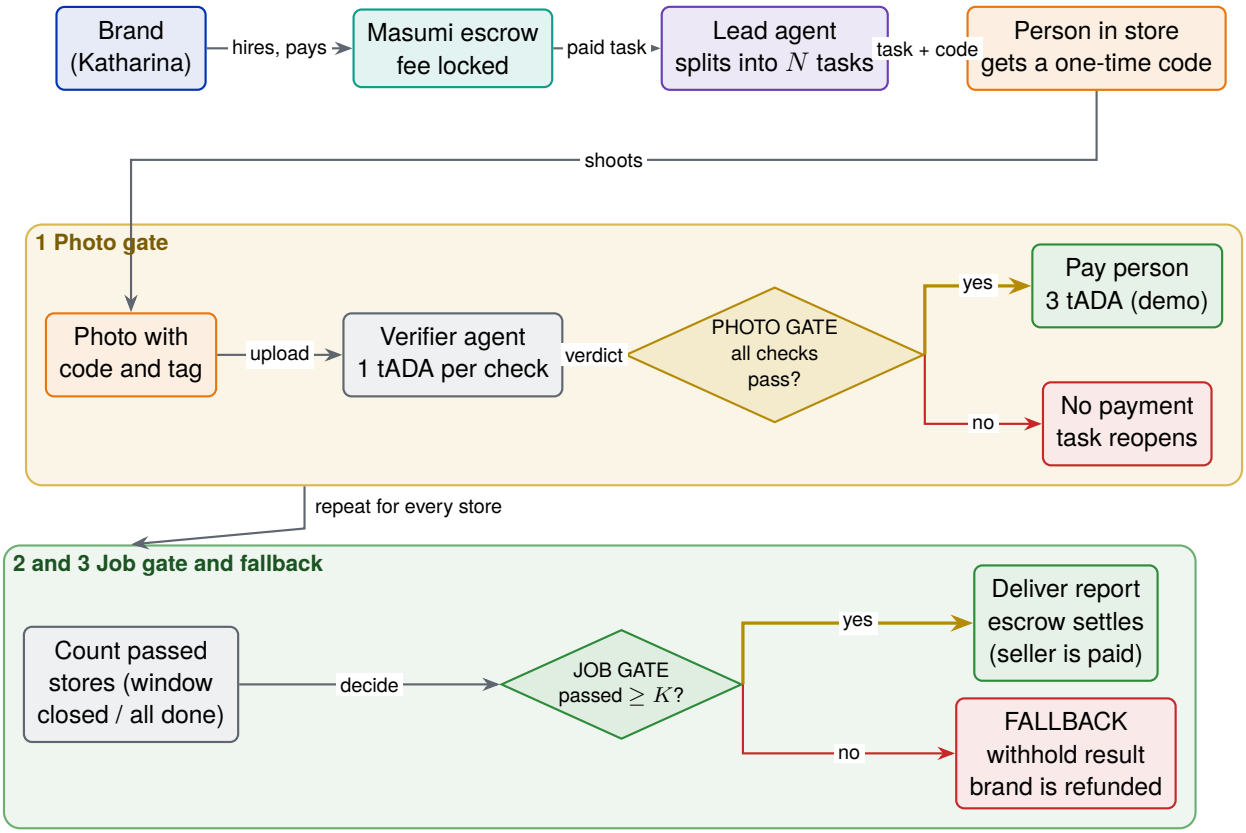


Figure 6.1: The three payment gates. Gate 1 pays one person per accepted photo. Gate 2 and the fallback decide the whole job: either the escrow settles or the brand gets the money back.

6.3.1 K of N, with numbers

The job gate uses a simple rule: the brand pays for the job if at least K stores pass out of N. We call this **K-of-N**. The default is

$$K = \lceil 0.6\, N \rceil$$

where $\lceil x \rceil$ means "round up to the next whole number". The 0.6 comes from the design files (`kDefault: ceil(0.6*n)`); an earlier draft said 0.7 and was corrected.

Example: Worked examples

- $N = 5$: $0.6 \cdot 5 = 3.0$, so $K = 3$. Three good photos are enough.
- $N = 10$: $0.6 \cdot 10 = 6.0$, so $K = 6$. Katharina's real job.
- $N = 7$: $0.6 \cdot 7 = 4.2$, rounded up $K = 5$. Rounding up makes the rule stricter, never looser.
- $N = 3$ (our smoke test): $0.6 \cdot 3 = 1.8$, so $K = 2$.

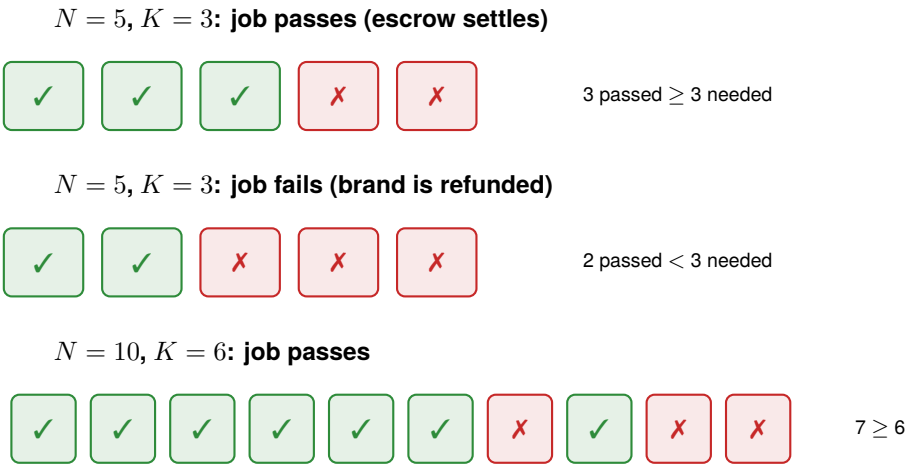


Figure 6.2: Each square is one store. Green means the verifier accepted the photo. Whether the job passes depends only on the count of green squares.

6.3.2 Who bears the cost of a failed job?

Workers whose photo was accepted are paid immediately, even if the job later fails the K-of-N rule and the brand is refunded (decision ADR-09 in the design files). Reason: paying at verification keeps the live "photo goes green, payout appears" moment, and workers should not wait an hour for money. Consequence: on a failed job *Shelfie* bears the cost of workers already paid. The ledger records this as `COST_BORNE`, and the pitch says it openly: "we pay workers for verified work, regardless of the brand's threshold."

➤ Why can a failed job still produce payouts?

The two decisions are independent. The *photo gate* asks: "is this photo true?" The *job gate* asks: "did the brand receive enough true photos?" A true photo deserves its payout even when the other stores failed.

6.4 The demo moment

Judges remember one moment, not twelve slides. Ours is designed in advance:

1. One teammate stands up holding a product with a printed price tag.
2. The worker page on the phone shows a **challenge code** such as `K7Q-4XM`. The phone that shoots cannot show its own screen, so the code is written on paper (or a printed card from our card pool is used) and held next to the tag.
3. The teammate takes the photo in the page's camera. The verifier highlights price and product; the row turns **green**.
4. A payout transaction appears on the dashboard and, a few seconds later, on Cardanoscan (the preprod explorer).
5. Then the **cheat attempt**: an old photo of the same tag, without the right code. The verifier rejects it live: red, reason "code missing", no payment. The verification fee is still spent (the verifier is paid per check), and we say so.
6. Optional: invite a judge to take a photo themselves.

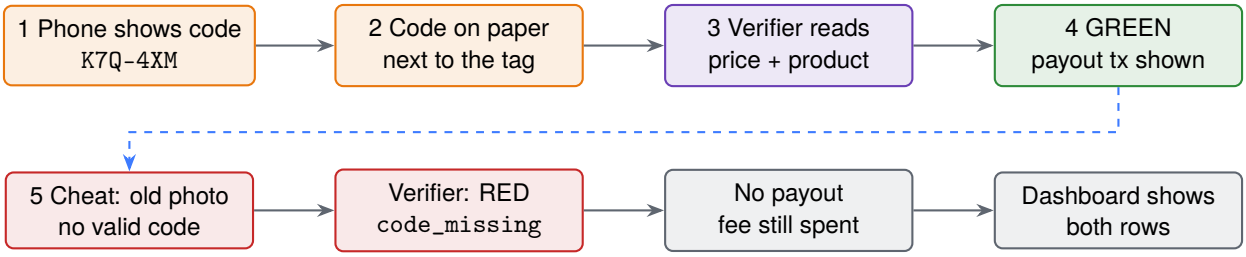


Figure 6.3: Live demo choreography: one honest photo, then one cheat attempt.

✗ Common pitfall: How the cheat photo gets in is still open

Over the HTTPS tunnel the worker page uses in-app capture and cannot pick a gallery photo. How the team submits the cheat photo live (LAN fallback page, operator-only upload route, or a pre-shot file through the dashboard) is an *open item* in the review report. Decide Friday, rehearse, and say which method was used.

6.5 Why it fits the judging rubric

The Cardano track description lists what the judges want to see. The table maps each wish to a feature. The right column is our reading (inference), not an official statement.

Judges want	Shelfie answers with
Clear customer and journey	Brand manager Katharina, one concrete job, a real industry (retail execution).
Payment tied to the result	Three gates: verifier pass pays the human; K-of-N settles the escrow; otherwise refund.
Agents hire agents, or act in the real world	The lead agent hires a verifier agent (x402) *and* humans in physical stores.
Live paid task with transaction in an explorer	A photo on stage turns green and the payout tx shows on Cardanoscan.
Originality	Agents as employers of people; a one-time code as the freshness proof; nothing similar seen on Sokosumi.
Impact and continuation	Retail audits first; later any "proof of the real world" task: deliveries, outages, accessibility checks.

The official requirement is modest: a working agent or service that completes at least one job or payment on Cardano **preprod**. Everything beyond it is about earning the points for the criteria above.

6.6 Business model (illustrative pricing)

✗ Common pitfall: These euro figures are illustrative

The EUR amounts below are *hypotheses for the pitch*. They are not measured and not what the demo moves. The demo uses tADA, which are test coins on preprod and have no market value. Never put a tADA amount next to a EUR amount as if they were equal.

Target: about **EUR 5 per verified store check, charged only when it passes**. One check involves five money items. Margin per check, for the bottom of the payout range and for the top:

Item per passed check	Low payout	High payout
Price to the brand (illustrative)	5.00	5.00
Payout to the person	−2.50	−3.00
Verifier cost (model + fee, estimate)	−0.10	−0.10
Network/escrow fee (about 5%, secondary source, verify)	−0.25	−0.25
Margin before support and disputes (EUR)	**2.15**	**1.65**

Hence the margin range of **EUR 1.65 to 2.15** that appears in the pitch. Two honesty notes: (1) verifier fees spent on *rejected* photos are not in this table, because rejected photos are not billed to the brand; (2) support, disputes and legal costs are not included.

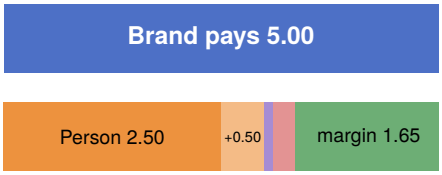


Figure 6.4: Where one illustrative EUR 5 goes (high-payout case). Orange: payout (2.50 solid, up to 3.00 with the lighter block); purple: verifier 0.10; red: fee 0.25; green: margin.

6.6.1 Who pays and why would they switch?

Small and mid-size consumer brands without a retail-execution team. They switch for *speed* (same day), *proof* (a code-bound photo with an audit trail) and *pay per passed check*. We have **not validated demand with interviews**; the hackathon proves a mechanism, not a market. Next step in the roadmap: five brand interviews.

6.7 Limits we admit

Judges respect honest limits more than hidden ones. The team has agreed on this list and says it plainly:

Limit	What is true
The code proves **freshness**, not **location**	A worker could photograph the same product in a different shop. Roadmap: store-owned QR codes or geofenced tasks.
The gate proves “fresh and readable”, not “genuine”	No claim of zero fraud; a fake made live is possible in theory. The cost of cheating should exceed the payout.
No identity checks, no reputation	Only claim time limits, attempt caps and a duplicate-image check exist. Workers are just a wallet address.
Humans need a wallet	The demo uses teammates’ preprod wallets. A production onboarding is roadmap.
Preprod only	No mainnet, no real money. The x402 facilitator is preprod-only today.
Settlement is shown from earlier runs	An escrow cycle takes at least 60 minutes, so on stage we show recorded lock, settlement and refund transactions and say so.
Legal questions open	Is per-task pay employment? Tax? Custody? Needs a legal opinion before mainnet.
Demo override exists	An admin-only manual override can rescue a stage demo; it is flagged red and never used in evidence runs (Chapter 8).

6.8 Alternatives we considered

The team ran two independent brainstorming rounds with 27 ideas and weighted scoring. Both put the same concept first.

Idea	Score	Verdict
Shelfie ("Eyes on Ground" / "Ground Truth")	4.25 of 5 and 7.95 of 10	Chosen. Real-world task, deterministic pass/fail, strong live moment.
ShopScout	3.35 of 5	Solid but generic; its "report quality" gate is a subjective opinion, which escrow handles badly. Its plumbing is reused by Shelfie.
Behörde Brief Decoder	7.65 of 10	Safest build; **Plan B**. Same architecture with an OCR worker instead of humans.
Termin-Sniper / CancelBot	–	Great business models but bot-protected portals or long latency, so mock-only demos.
Verified Findings	–	Easiest, least emotional.

➤ Plan B is not a separate project

Plan B reuses the engine from Chapter 7. If feedback or a workshop forces a change, the team swaps the *domain pack* (task template, verifier rules, report template) and keeps payments, agent shell and UI. A letter is uploaded, an OCR worker extracts deadline, amount and authority, rules check plausibility, and the report explains the letter. Trigger from the risk register: by Friday 17:00 the vision verifier cannot read printed tags reliably *and* the labelled override is not enough.

★ Key takeaways

- Shelfie sells **verified shelf checks**: a brand hires an agent, which hires people to photograph shelves; payment is tied to the photo being true.
- Persona: Katharina Brandl at Isarquell (**fictional**). Job: "check product X in N stores".
- Three payment gates: photo gate pays the human; job gate settles the escrow at K of N with $K = \lceil 0.6N \rceil$; fallback refunds the brand.
- Demo: one honest photo turns green and shows a payout tx; one cheat photo is rejected with no payment.
- EUR prices are **illustrative**; margin range EUR 1.65 to 2.15; the demo uses tADA on preprod.
- Honest limits: the code proves freshness, not location; no identity checks; settlement shown from recorded runs.

? Check yourself

1. Which of the three payment gates pays a human, and what triggers it?
2. Compute K for $N = 8$. Show the rounding.
3. A job with $N = 5$, $K = 3$ ends with 2 accepted photos. Who has been paid, and what happens to the brand's fee?
4. Why do we say the pricing is "illustrative"?
5. Give two limits of the freshness code that we admit openly.
6. What is Plan B and what does it reuse?

Answers. (1) The photo gate: when all verifier checks G1 to G7 pass, the payout is queued immediately. (2) $0.6 \cdot 8 = 4.8$, rounded up $K = 5$. (3) The two workers with accepted photos are paid at once; the brand's escrowed fee is refunded because 2 is less than 3; Shelfie bears the worker cost. (4) The EUR amounts are unsourced hypotheses; the demo moves tADA without market value. (5) It does not prove location, and a live fake is possible in theory (also: no identity checks). (6) Behörde Brief Decoder; it reuses the engine, payments, shell and UI, swapping only the domain pack.

CHAPTER 7

Architecture and domain model

Why this matters for us

Four people will write code in parallel from Friday 09:00 without meetings. That only works if everyone shares the same picture of the pieces, the names and the rules for money. This chapter is that picture, in plain language. It follows the design folder (`diseno/`), which is the canonical source when files disagree. **Nothing described here exists as code yet.** It is a plan, and the build window opens on Friday.

7.1 The big idea: a verified micro-task engine

Shelfie is not a "shelf app". Underneath, it is a general machine for one pattern:

Job → Tasks → Worker (human or agent) → Verifier → Settlement

A **job** is what the customer orders. It is split into small **tasks** (one per store). A **worker** does each task; the worker may be a human with a phone or another agent. A **verifier** applies rules to the result. **Settlement** is the money outcome of the whole job. Everything about shelves (the task template, the verification rules, the report layout) lives in a replaceable **domain pack**.

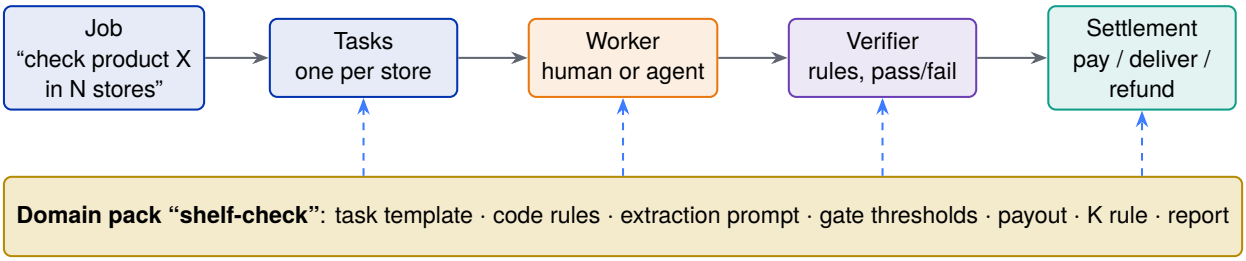


Figure 7.1: The engine pattern. The domain pack plugs shelf-specific content into the generic stages.

Everyday analogy: A pizza oven with changeable recipe cards

The oven (heat, timer, conveyor belt) stays the same. Only the recipe card says what goes in and when it is done. Our engine is the oven; the domain pack is the recipe card. If we swap the card, we bake a different dish (Plan B: decode a German letter) with the same oven.

7.1.1 Domain packs

A pack tells the engine: what a task looks like, who works on it (human or agent), what the verifier extracts, which rules apply, how much a task pays, which K rule decides the job, and how to render the report. Plan B shows how little changes:

Pack field	`shelf-check`	`letter-decoder` **(Plan B)**
Job input	product, EAN, stores, K	letter images, language
Tasks	one per store	one per page or field group
Worker	human, camera, challenge code	agent (OCR), no challenge
Extraction	product name, price, code	sender, date, amount, deadline
Rules	code, readable, product, price, timing	date and amount parse, plausible
Payout	bounty to the human	optional fee or 0
Settlement	K of N stores	all fields required ($K = N$)

To keep Plan B truly cheap, two things are in the engine from day one: workers of kind `human` and `agent`, and an *optional* challenge.

7.2 Components and owners

Four roles, A to D. The names of the four teammates are *not yet* assigned to letters; this is decided on Thursday evening.

Component	Folder	What it does	Owner
Lead agent shell	`apps/shell`	Fork of the Masumi starter (JavaScript). Sokosumi Coworker, paid-task adapter, bridge to the engine. Thin.	A
Engine	`apps/engine`	State machines, challenge codes, K-of-N decision, REST API, ledger writer. Loads one pack.	A
Verifier	`apps/verifier`	Sells `POST /v1/verify` behind x402; one vision call, then deterministic gates.	B + C
Payments	`packages/payments`	Wallet signers, x402 buyer, payout queue, reconcile.	B
Worker page	`apps/web /w`	Mobile page: list tasks, claim, show code, take photo, show verdict.	D
Brand dashboard	`apps/web /dashboard`	Progress per store, verdicts, ledger with explorer links, big-screen layout.	D
Evidence ledger	`packages/evidence`	Append-only, hash-chained list of every tx hash and verdict.	D
Domain pack	`packages/domain-shelf`	Thresholds, prompts, report template, labels.	C

➤ Role summary in one line each

A: shell and engine (Sokosumi/Masumi operator). **B**: payments, x402, wallets, tunnel. **C**: verifier gate, vision, domain pack, 20-photo evaluation. **D**: web pages, evidence ledger, README, pitch. On stage the roles shift: D pitches, B operates, A is the field shopper, C narrates the verifier.

7.2.1 Deployment: one hub laptop and one tunnel

During the demo everything runs on **one laptop**, the hub (ADR-14). One machine to keep awake, one set of secret files. Phones reach the engine through a public HTTPS **tunnel** (a tool such as a Cloudflare quick tunnel that gives your laptop a temporary public web address). Why HTTPS matters: phone browsers allow the in-page camera only on secure pages. The verifier and Masumi Payment Service stay private.

✗ Common pitfall: Never restart the tunnel mid-demo

Quick-tunnel addresses change on restart. The dashboard shows a QR code for the worker page; after any restart everyone must scan again. Fallback: laptop hotspot with plain HTTP, where the page falls back to a file picker (no in-page camera).

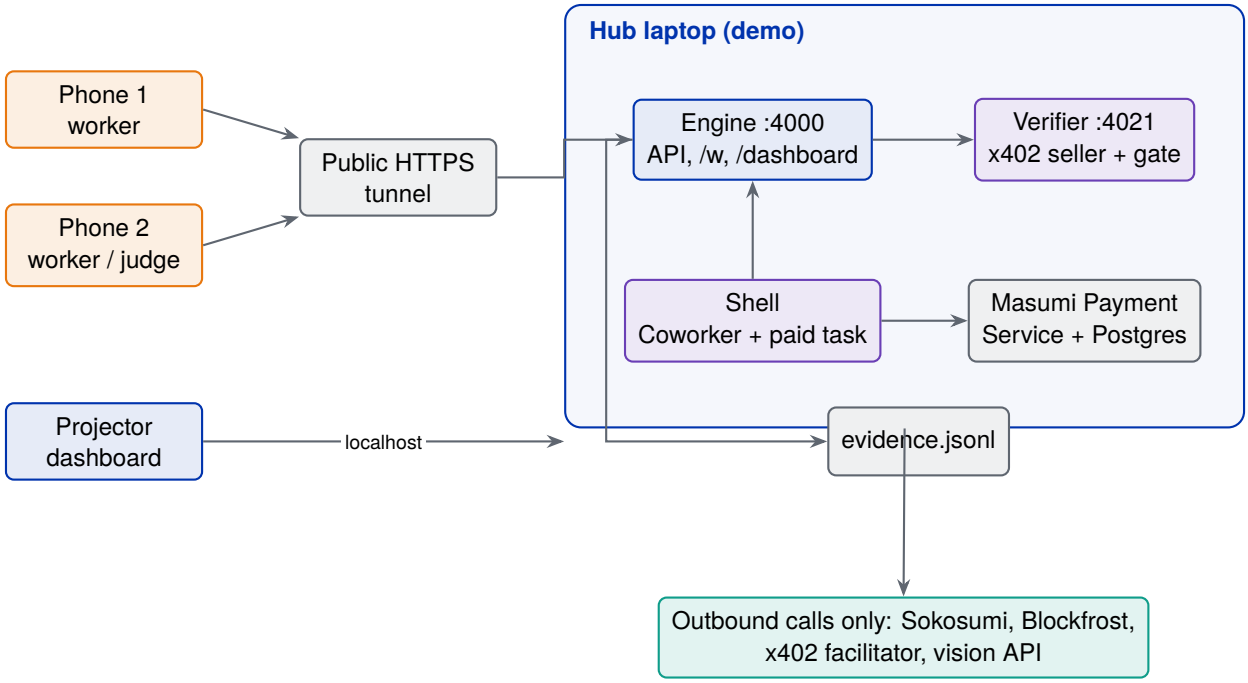


Figure 7.2: Demo deployment. Only the engine is exposed publicly; everything else calls out from the hub.

7.3 One task, end to end

Here is the happy path of one store, as a sequence diagram. Read it top to bottom; each arrow is a message.

➤ What the 402 steps do

The engine first calls the verifier *without* paying. The verifier answers "402 Payment Required, 1 tADA". The engine signs a payment from its wallet `W-VBUY` and retries with the proof. This is exactly the x402 flow from Chapter 3, with our verifier as the seller and our engine as the buyer. A spending cap prevents the buyer from paying more than 2 tADA per call.

When the photo fails, the picture changes: the verifier still answers 200 (it did its job), but the verdict is FAIL, so the engine enqueues *no payout*. The check was paid for, the photo was not. If attempts remain (default cap: 3), the task goes back to OPEN, and the next attempt needs a *new* code. After the repeat for all stores, the engine counts passes: $\geq K$ means the report is delivered and the escrow settles later; otherwise the shell submits *no* result hash and the escrow refunds the brand after the Masumi timeout (about $+30$ minutes counted from the start of the job, i.e. about 10 minutes after the submit deadline; an estimate to record Friday).

7.4 Entities and ID prefixes

The domain model lists nine things the engine stores. Each has an ID that begins with a short prefix, so you can recognise it in a log at a glance. All IDs are strings; money is always a decimal string of lovelace (1 ADA = 1,000,000 lovelace).

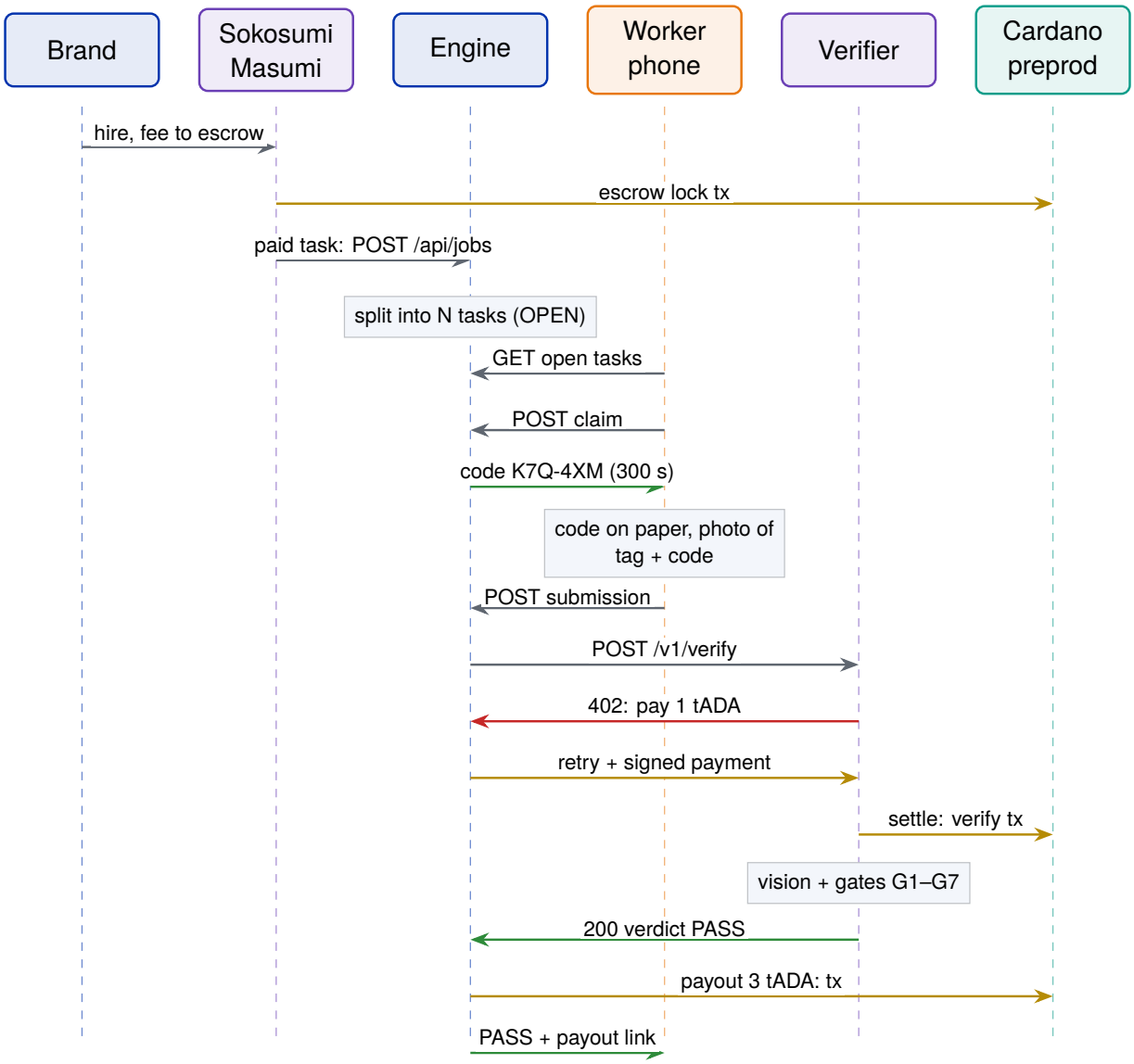


Figure 7.3: One store, happy path. Gold arrows are money; green arrows are good news; red is the 402 challenge.

Prefix	Entity	Meaning
`job_`	Job	One hired engagement (e.g. “Isarquell in 10 stores”). Knows N, K, window and decision.
`tsk_`	Task	One unit of work: one store. Has state, attempts, accepted submission.
`clm_`	Claim	A worker reserved a task for a short time (default 300 s) and received a code.
`sub_`	Submission	One photo upload for a claim, with server receive time and image hash.
`vdt_`	Verdict	The verifier’s answer: pass or fail, all gate results, extraction, verify tx hash. Immutable.
`pay_`	Payout	One payment to a worker for one task. Unique per task.
`stl_`	Settlement	The job-level outcome: DELIVER or WITHHOLD, escrow tx references.
`evd_`	Evidence entry	One line in the hash-chained ledger.

A `ChallengeCode` (not prefixed) belongs to a claim and is single use. It uses the 6-character format described in Chapter 8.

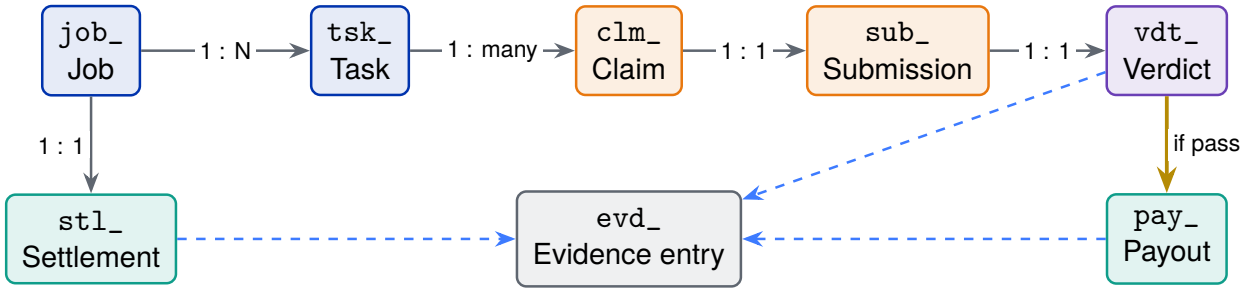


Figure 7.4: How the entities relate. Dashed arrows: everything on-chain or decisive is also written to the evidence ledger.

7.5 State machines

A **state machine** lists the states a thing can be in and the allowed moves between them. It prevents nonsense such as paying a task that was never verified.

7.5.1 Job states

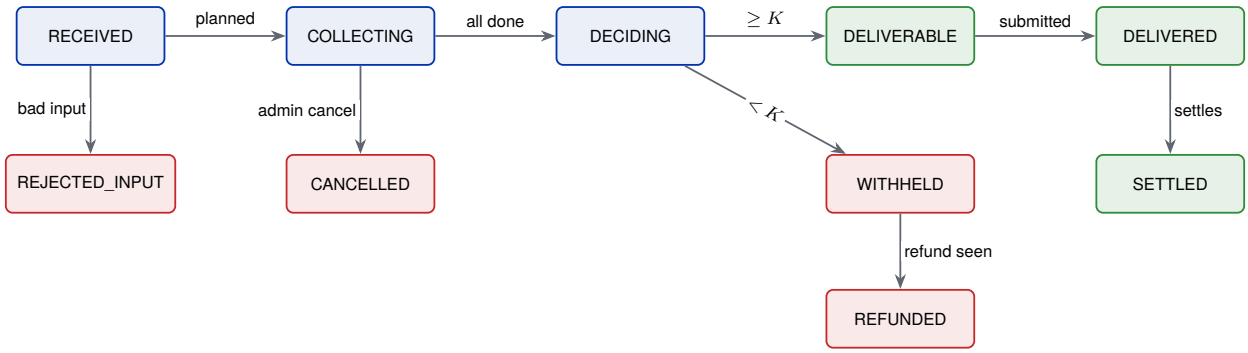


Figure 7.5: Job lifecycle: “all done” means the window closed or every task is finished; K is the pass threshold. SETTLED and REFUNDED are informational: the engine records what Masumi did but never waits for it.

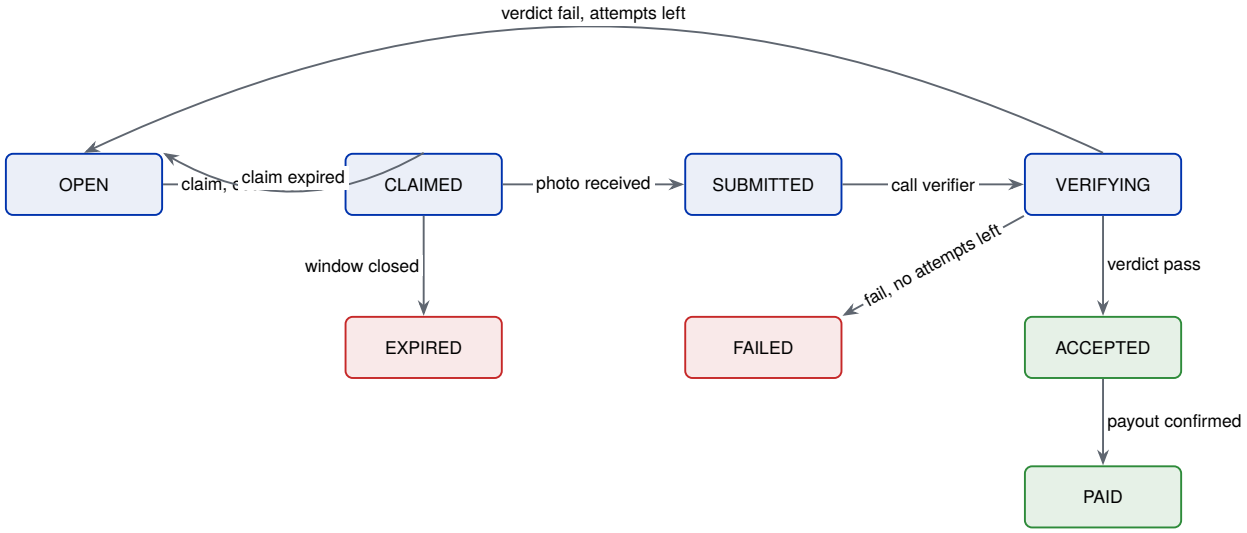


Figure 7.6: Task lifecycle. ACCEPTED counts toward K immediately, even if the payout is still in flight; the payout has its own machine.

7.5.2 Task states

In Shelfie: A rejected photo burns its code

A claim is single use: one submission per claim. If the photo fails, the code is spent. A retry means a new claim and a *new* code. This is the reason a cheater cannot try the same code twice.

7.5.3 Payout states

The payout machine is the most delicate, because it moves real value. States: `QUEUED` → `SENDING` → `SUBMITTED` → `CONFIRMED`. If the engine does not know whether a transaction went out (timeout, crash), the payout becomes `UNKNOWN` and must be *reconciled* against the chain before anything is sent again. The next section explains why.

7.6 Idempotency and the payout queue, from zero

7.6.1 What idempotent means

An operation is **idempotent** if doing it twice has the same effect as doing it once. Pressing the lift button twice calls the lift once. In a web system requests are repeated all the time: the network times out, the user double-taps, a program restarts and tries again. If the operation is "pay 3 tADA" and it is not idempotent, a retry means paying twice. Money makes retries dangerous.

An **idempotency key** is a label that identifies the *intention*, e.g. "pay the worker for task `tsk_07`". The system stores the key the first time. Any later request with the same key returns the original result instead of repeating the action.

Example: Why it prevents double payment

Task `tsk_07` passes. The engine creates a payout with the key `payout:tsk_07` and sends 3 tADA to Anna's address. The engine crashes at the worst moment: the transaction was sent, but the engine did not write down its hash.
Without idempotency and write-ahead logging: after restart the engine sees "task accepted,

no payout hash", concludes "not paid" and sends another 3 tADA. Anna received 6 tADA; Shelfie lost 3.
With our design: before calling the wallet, the engine *writes* "SENDING, attempt 1" to disk (this is called a write-ahead record). After the crash it sees SENDING with no hash and moves the payout to UNKNOWN. A reconcile step searches the wallet's history for a transaction carrying the metadata tag `shelfie:pay_03` to Anna's address. It finds one: the payout is marked SUBMITTED with that hash. No second payment. Only if the search *proves* no such transaction exists (after a waiting period of at least two minutes) may the payout return to QUEUED.

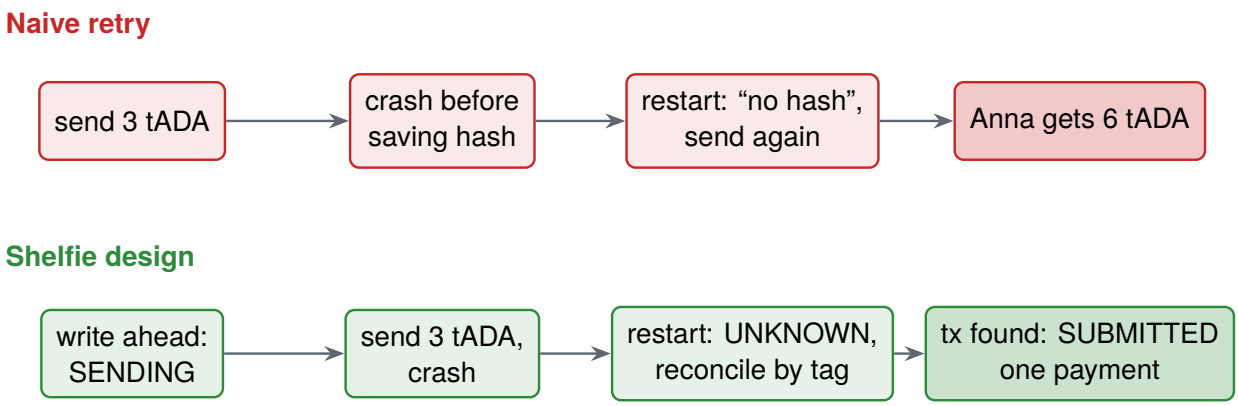


Figure 7.7: The same crash, two designs.

7.6.2 The keys we use

Operation	Key	On repeat
Create job	Sokosumi task id (`externalRef`)	Returns the existing job
Claim a task	(task, worker) while active	Same claim, same code
Submit a photo	claim id; image hash	One submission per claim; same hash in same job: `duplicate_image`
Verify	`verify:<submissionId>`	Cached verdict; vision model not called again
Payout	`payout:<taskId>`	Returns the existing payout
Payout tx	metadata tag `shelfie:<payoutId>`	Lets reconcile find an earlier attempt
Override	`override:<submissionId>`	One override per submission

7.6.3 The payout queue and one-wallet-no-concurrency

In Cardano's eUTXO model (Chapter 2), a wallet spends specific coins (UTxOs). If two payments from one wallet start at once, both may try to spend the same coin and one fails. The x402 runbook states the first UTxO acts as a nonce, and a timeout does not prove a payment failed. Design consequences:

- Each sending wallet has its **own serial queue with concurrency 1**: the next send starts only after the previous is confirmed or its change output is visible.
- Separate wallets per purpose, so a verify payment and a worker payout never compete.
- The worker page shows "queued for verification, position 3". Verification takes 20 to 60 s per check (an estimate), so a burst of five uploads takes minutes. The demo script staggers submissions.

7.7 Wallets and who pays whom

Four wallets, each with one purpose (plus the workers' own addresses). All are throwaway preprod wallets; secrets live only in untracked environment files.

Wallet	Who uses it	Purpose
`W-SELL`	verifier (receives)	Receives the 1 tADA verifier fees. The verifier holds no keys, only this address.
`W-VBUY`	engine (pays)	x402 buyer: pays the verifier, with a spend cap.
`W-PAY`	engine (pays)	Plain transfers of 3 tADA to workers.
`W-MPS`	Masumi Payment Service	Seller wallet for the escrowed job fee, independent of ours.

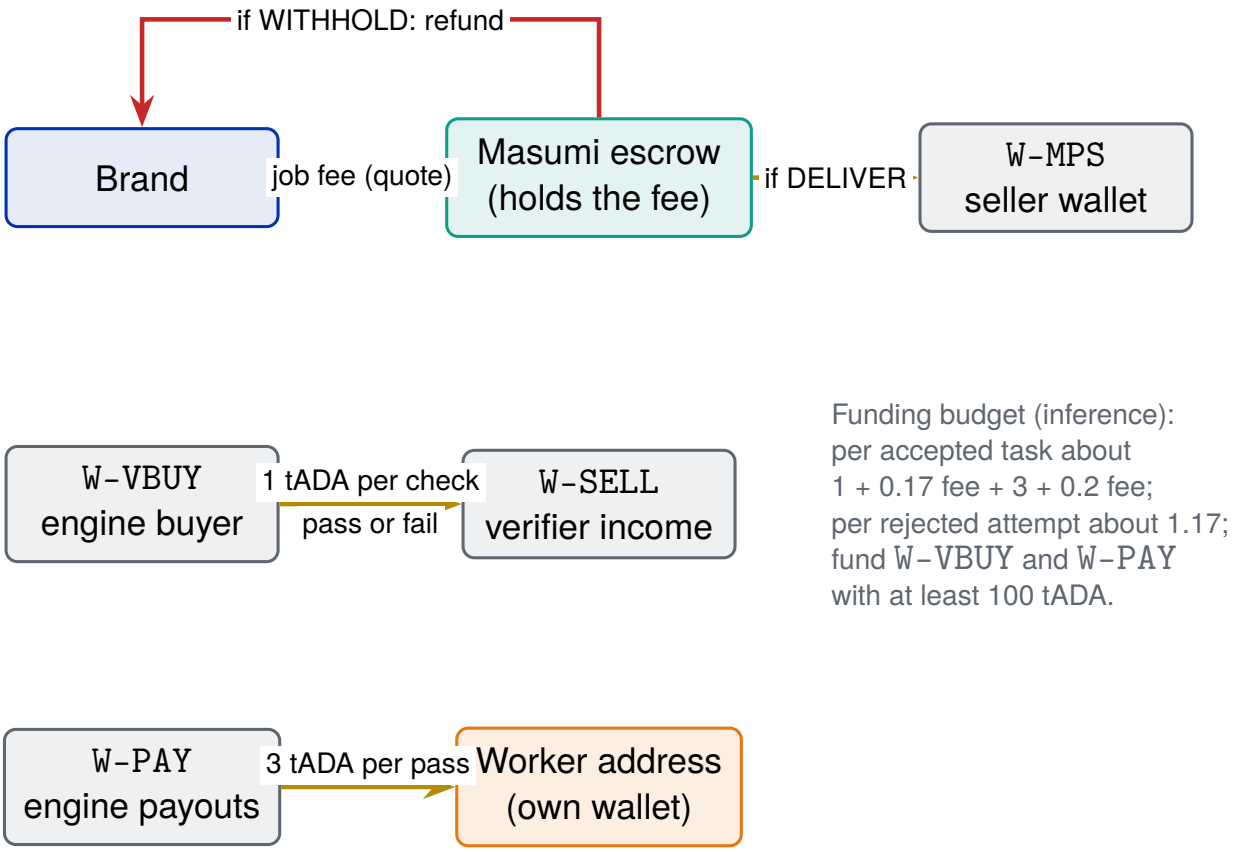


Figure 7.8: Money flow of one job. Gold arrows are payments. The escrow asset is a Masumi quote (about 1 test USDM, verify Friday); the other payments are tADA.

✗ Common pitfall: Do not mix up the two kinds of money

The brand's escrow fee is a Sokosumi/Masumi quote whose asset and amount are **still to be confirmed** (`[VERIFY_ASSET]`). The verifier fee and the payout are in tADA (ADR-03). Never convert between them in the pitch.

7.8 Key API endpoints

The engine exposes a small REST API on `http://localhost:4000/api`. IDs are prefixed strings, timestamps are ISO-8601 UTC, and errors share one shape: `{ "error": { "code": "claim_expired", "message": "..." } }`. A selection:

Endpoint	Caller	Purpose
`POST /api/jobs`	shell, admin	Create a job (idempotent on `externalRef`); returns tasks.
`GET /api/jobs/{id}`	web	Job with counts, decision, settlement.
`GET /api/jobs/{id}/decision`	shell	Long poll: PENDING, DELIVER (with report), WITHHOLD.
`POST /api/jobs/{id}/settlement-events`	shell	Report Masumi observations (lock, settle, refund).
`GET /api/tasks?status=OPEN`	worker page	List tasks a worker can take.
`POST /api/tasks/{id}/claim`	worker page	Reserve a task; returns the challenge code.
`POST /api/tasks/{id}/submissions`	worker page	Upload the photo (base64 JSON); returns 202.
`GET /api/submissions/{id}`	worker page	Poll verdict, payout state and tx link.
`GET /api/ledger`, `/ledger/stream`	dashboard	Evidence entries; live stream (SSE).
`GET /api/health`	dashboard	Wallet balances, verifier up/down, *fake* flags.
`POST /api/admin/.../override`	admin only	Demo override (off by default).
`POST /v1/verify` (verifier)	engine	Paid endpoint; 402 first, then verdict.

Three header-based roles exist: workers are open (a random `workerId` plus a payout address), the shell sends `X-Shell-Token`, admins send `X-Admin-Token`. The verifier's `POST /v1/verify` is protected only by x402. Typical status codes you will meet: 402 (payment needed, verifier only), 409 (state conflict such as `claim_expired`), 413 (image too large), 422 (valid format, bad content), 429 (rate limited).

In Shelfie: Fakes make parallel work possible

With `FAKE_PAYMENTS=1` the engine runs on fake payments and fake verification, so A, C and D can build against it from minute one. The dashboard then shows a red "FAKE PAYMENTS" banner, and an evidence run never counts a fake verdict.

7.9 Important decisions (ADRs) in plain language

An **ADR** (architecture decision record) is a short note: what we decided, which alternatives we rejected, what it costs us. The design folder has 15; these are the ones every teammate should know.

ADR	Decision	In plain words
01	Engine plus domain pack	Build one generic engine; shelf checks are one pack. Costs about an hour of structure, buys a cheap Plan B.
02	x402 for the verifier, 1 tADA per check	Agents paying agents is the point of the track. Every check leaves a public tx, pass or fail. 1 tADA is the floor; if the facilitator rejects it, use 2.
03	tADA, not tUSDM	Two incompatible tUSDM token variants exist; lovelace avoids the mix-up.
04	Plain transfer for worker payouts	Unverified: can the SDK send plain transfers? Fallbacks in order: SDK; Lucid/Mesh for the transfer only; worker address as x402 `payTo`; labelled manual payout.
05	Escrow job runs early	An escrow cycle takes at least 60 minutes, so start Friday and show recorded settlement on stage.
06	HTTPS tunnel over LAN	Only secure pages may use the in-page camera; venue Wi-Fi may isolate clients.
07	Vision model is configuration	The 20-photo test picks the model. The model reads; plain code decides; the model never sees the expected code or price.
08	Separate wallets, one queue each	Avoids coin contention; verification becomes slow but safe.
09	Workers paid at verifier pass	Keeps the live payout moment; Shelfie bears the risk if the job later fails.
10	Simple persistence	In-memory plus an append-only log replayed at start; fine for tens of records.
11	Withhold the result to refund	On a failed job the shell submits no hash, so Masumi's no-result refund applies. A "failed" result would pay the seller.
12	Code proves freshness, not location	GPS is spoofable and EXIF editable; store-owned QR codes are roadmap.
13	Override exists, off by default, always visible	A rescue for the stage, never silent.
14	One hub laptop	One machine to keep awake; a cold spare is prepared.
15	Thin shell fork	JavaScript stays only in the starter; logic is in testable TypeScript.

★ Key takeaways

- Shelfie is a **generic engine** (Job → Tasks → Worker → Verifier → Settlement); shelf checks are one swappable **domain pack**.
- Roles: A shell and engine, B payments and x402, C verifier and pack, D web, evidence, pitch. Everything runs on one hub laptop behind one HTTPS tunnel.
- IDs: `job_ tsk_ clm_ sub_ vdt_ pay_ stl_ evd_`. State machines forbid illegal moves.
- **Idempotency keys** and a write-ahead payout queue with on-chain reconcile prevent double payment.
- Wallets: `W-VBUY` pays the verifier (1 tADA per check), `W-SELL` receives, `W-PAY` pays workers (3 tADA), `W-MPS` is Masumi's seller wallet.
- Nothing is built yet; items marked "verify Friday" (SDK transfers, escrow timing, tunnel) are open.

? Check yourself

1. Name the five stages of the engine pattern.
2. What would you change to swap Shelfie for the letter decoder, and what stays?
3. What does the prefix `pay_` identify, and how many may exist per task?
4. Explain in two sentences how the write-ahead record and the metadata tag prevent a double payout after a crash.
5. Which wallet pays the verifier, and how much per check?

6. Why is the demo run on one hub laptop with an HTTPS tunnel?

Answers. (1) Job, Tasks, Worker, Verifier, Settlement. (2) Only the domain pack (and `PACK_ID`); shell, payments, engine core, verifier wrapper, ledger and web frame stay. (3) A payout; at most one per task (unique key `payout:<taskId>`). (4) The "SENDING" record is written before the wallet is called, so after a crash the engine knows a send may have happened and checks the chain for the tagged transaction first; it only resends if none exists. (5) `W-VBUY`; 1 tADA, pass or fail. (6) One machine to keep awake and configure; the tunnel gives phones HTTPS (needed for the in-page camera) and works on mobile data.

CHAPTER 8

The verifier

Why this matters for us

The verifier is the part of Shelfie that decides whether money moves. Everything the pitch promises ("paid only when the photo is true") rests on it. It is also the part judges will attack with questions about fraud. Understanding it deeply, including what it *cannot* prove, lets any teammate answer those questions calmly. The details come from `diseno/verifier-rules.md`. As everywhere in this guide, nothing is built yet.

8.1 Design principle: a deterministic gate around a probabilistic reader

Two kinds of components appear in the verifier:

- A **vision model** (an AI that looks at images) is *probabilistic*: it gives an answer that is usually right, sometimes wrong, and can differ between two runs.
- A **deterministic** rule is ordinary code: the same input always gives the same output, and you can explain why.

Shelfie's rule: **the model only reads; plain code decides.** The model reports what it sees (a code, a price, a product name). Then rules compare those readings with what we expect. Payment depends on rules, never on the model's opinion.

Everyday analogy: Court clerk and judge

The vision model is a clerk who reads out what is written on a document. The gates are the judge who applies the law to those words. If the clerk misreads a digit, the judge's explicit rules (does the code match? is the price plausible?) still catch most mistakes, and every decision can be explained afterwards.

Another design rule: **the model never sees the expected code or the expected price.** If it knew the answer, it might "confirm" what we hope to see (confirmation bias). It reports blindly and our code compares.

8.2 The challenge code

8.2.1 What problem it solves

A worker is paid for a photo. The cheapest cheat is to reuse a photo: an old one, a stock image, a screenshot, or a photo taken at home. How can the verifier know that the photo was taken *now* and *for this task*? Ask the worker to include something that did not exist before the task started. That is the **challenge code**.

➤ Challenge code

A random, single-use, short-lived 6-character code, issued when the worker claims a task. The worker must show it in the photo, next to the price tag. It proves *freshness* (the photo is newer

than the claim) and *task binding* (the photo belongs to this task). It does **not** prove location.

8.2.2 Format and alphabet

- 6 characters, displayed as `XXX-XXX`, for example `K7Q-4XM`, in a big monospace font.
- Alphabet: `ABCDEFGHJKMNPQRSTUVWXYZ23456789`. The characters **0, O, 1, I, L** are left out because they are easily confused in handwriting and photos (zero or letter O? one or letter L?). That leaves 31 characters.
- $31^6 = 887{,}503{,}681$ combinations, so guessing is hopeless (the design files say "about 30^6", which is the same order). Codes come from a cryptographically secure random generator, not from the task id, so they cannot be predicted.
- Time to live: 300 s by default. An expired claim returns the task to OPEN.
- A new claim attempt always gets a *new* code. A rejected photo burns its code.

8.2.3 Why the phone cannot show its own screen

The phone that takes the photo is busy being a camera. It cannot display the code and photograph itself at the same time. So the code has to be *physically* next to the price tag. Two modes exist, chosen in the pack config:

- `handwritten` (default): the worker reads the code on the phone, writes it on a piece of paper and holds it next to the tag.
- `card-pool`: printed cards with pre-generated codes; the engine hands out a card number. We use this for the stage demo for reliability. In the card-pool mode we print 20 large cards with high contrast.

A third mode, `none`, exists for Plan B where agents, not humans, do the work.

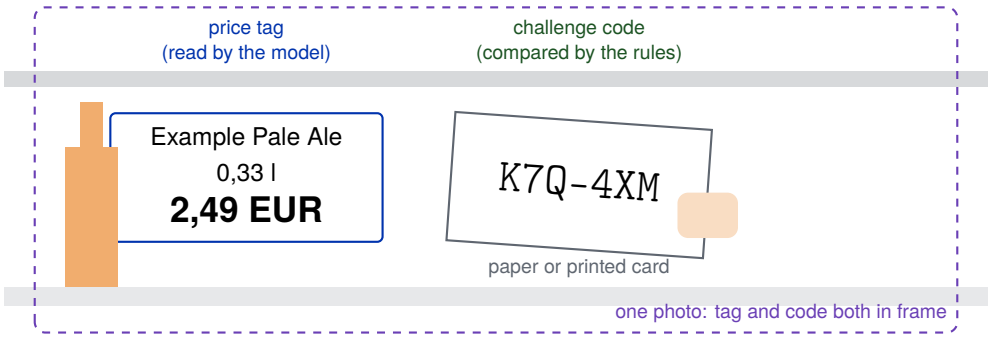


Figure 8.1: What a valid photo contains (schematic, not a real photo). The code is not shown on the phone; it is on paper or a card in the frame.

✗ Common pitfall: Freshness is not location

The code proves the photo is new and for this task. A worker could still photograph the same product in a different shop. We state this limit in the pitch (ADR-12). Roadmap: store-owned QR codes or geofenced tasks.

8.3 The verification pipeline and gates G1 to G7

The verifier receives a request (the photo plus the task context: expected code, product, time window) and produces one **verdict**. The verdict says **pass only if every gate passes**. There is no weighted score; scores exist only inside a gate, against that gate's own threshold. All gates are evaluated and reported, so the worker and the dashboard see every failure at once. The one exception: if G1 fails, the (costly) vision call is skipped.

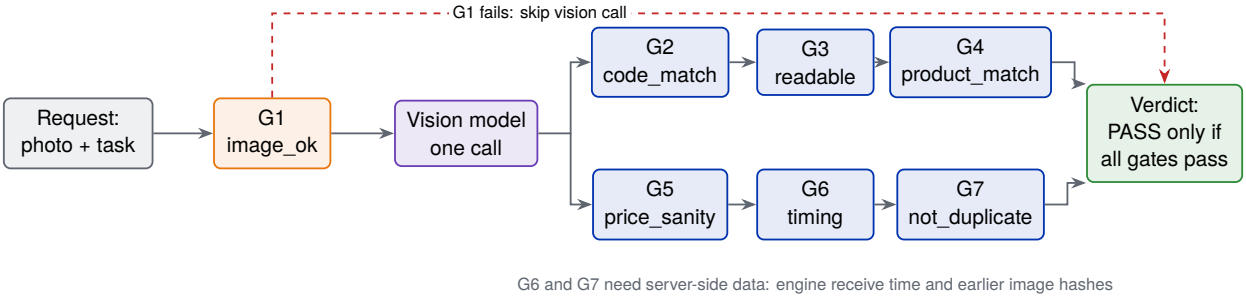


Figure 8.2: The verification pipeline. The vision call is the only impure step; the seven rules are pure functions of the stored extraction, so they can be tested with fixed JSON and no model.

8.3.1 The seven gates, each with a pass and a fail example

Use `Example Pale Ale 0.33l`, expected code `K7Q-4XM`, as the running example.

Gate	What it checks	Pass example	Fail example (reason code)
G1 `image_ok`	Decodable JPEG or PNG; at least 800×600 pixels; not too big; not a near-blank image.	1600×1200 JPEG, 400 KB.	640×480 image (`image_too_small`).
G2 `code_match`	Code read from the photo equals the expected code (edit distance at most 0) and read confidence at least 0.7.	Reads `K7Q-4XM`, confidence 0.93.	Reads `K7Q-4XN` (`code_mismatch`); no code visible (`code_missing`).
G3 `readable`	Price parses to a number (by our parser); currency EUR or absent; product text not empty; overall confidence at least 0.6.	Text `2,49 EUR` parses to 2.49.	Glare hides the price (`price_unreadable`).
G4 `product_match`	Valid EAN-13 barcode equals the task EAN, or name similarity to name or an alias at least 0.8.	Reads `Example Pale Ale 0,33 l`.	Reads another beer (`product_mismatch`).
G5 `price_sanity`	Between 0.20 and 60.00 EUR; within 60% of a reference price if the job gives one; at most 2 decimals.	2.49 EUR.	249.00 EUR (`price_out_of_range`).

Gate	What it checks	Pass example	Fail example (reason code)
G6 `timing`	Engine's receive time is at least 5 s after the claim and before the claim expires and before the task window ends.	Received 40 s after the claim.	Received after the 300 s TTL (`claim_expired`).
G7 `not_duplicate`	Image hash not seen in the same job; perceptual hash not too close to earlier accepted photos.	A new photo.	The same file submitted twice (`duplicate_image`).

A few details that often confuse people:

- **Edit distance** counts how many characters you must change to turn one string into another. `K7Q-4XM` to `K7Q-4XN` is distance 1. The default allows 0, because our alphabet has no confusable characters.
- **G6 uses the engine's clock**, never the phone's clock and never the photo's built-in EXIF date, because both are editable. EXIF time is only logged as advisory.
- A **perceptual hash** is a small fingerprint of what an image *looks like*, so that a slightly cropped or recompressed copy of a photo is still recognised as the same.
- The worker sees a plain-language message per failure, for example "We could not read the code K7Q-4XM in your photo. Hold the paper closer and try again."

Example: Cheat attempts we rehearse (all must be rejected)

1. An old camera-roll photo of the tag: no code in frame, G2 `code_missing`.
2. The right tag with a wrong code: G2 `code_mismatch`.
3. A photo of a different product with the right code: G4 `product_mismatch`.
4. The price covered or blurred: G3 `price_unreadable`.
5. The same photo submitted twice: G7 `duplicate_image`.

In every case the check was still paid to the verifier (1 tADA); the worker is paid nothing.

8.4 The vision model contract

One call per submission. The prompt lives in the domain pack. The model must return *only* JSON in a fixed shape (illustrative, from the design file):

```
{ "code": { "text": "K7Q-4XM", "confidence": 0.93 },
  "price": { "text": "2,49 EUR", "value": 2.49,
             "currency": "EUR", "confidence": 0.9 },
  "product": { "text": "Example Pale Ale 0,33 l", "confidence": 0.88 },
  "barcode": { "digits": "4000000000000", "confidence": 0.4 },
  "scene": { "isShelfOrPriceTag": true, "peopleVisible": false,
             "glare": "low", "blur": "none" },
  "advisory":{ "tamperSuspected": false, "looksLikeScreenPhoto": false },
  "overallConfidence": 0.88 }
```

Rules for the adapter, in plain words:

1. Unreadable fields are `null` with confidence 0, never guessed.
2. The JSON is parsed strictly; on a parse failure the call is retried once with a repair instruction, then the verifier answers "503 vision unavailable" (no verdict, and if possible no charge).
3. German number formats such as `2,49` are parsed by *our* parser. If the model's `value` and our parse disagree, ours wins.
4. Images are downscaled to at most 1600 pixels on the long side for cost and speed.
5. Which model and provider is used is *configuration* (ADR-07), chosen by the 20-photo test below, not by preference. Whether the shell's own language model can read images is unknown (verify Friday).

8.5 Thresholds

Every number is in `pack.config.json`, so tuning needs no code change.

Parameter	Start	How it is tuned
`code_match.maxEditDistance`	0	Raise to 1 only if handwritten codes fail but wrong-code attacks still fail.
`code_match.minConfidence`	0.7	Lower if too many false rejects, never below 0.5.
`readable.minConfidence`	0.6	Same.
`product_match.minTokenScore`	0.8	Add aliases first; lower the threshold last.
`price_sanity.minEur / maxEur`	0.20 / 60	Set per demo product.
`price_sanity.maxDeviationFromRef`	0.6	Only when the job gives a reference price.
`timing.minSecondsAfterClaim`	5	Fixed.
`image_ok.minWidth / minHeight`	800 / 600	Fixed.
Verify latency budget	p95 under 8 s	Faster model if exceeded (excludes the x402 step).

➤ False accept versus false reject

A **false accept** is a bad photo that passes (we pay for fraud). A **false reject** is a good photo that fails (an honest worker is annoyed). For a payment gate, false accepts are the safety failure. That is why the acceptance targets below demand *zero* false accepts but tolerate one false reject.

8.6 The 20-photo evaluation

Goal: choose the model and thresholds with evidence and obtain an honest number for the pitch. Owner C runs it on Friday as soon as the verifier answers (target: before milestone M2). Capture takes about 45 minutes with two people: printed price tags for a real product on our own shelf (or a fridge) plus a few from a shop with permission; **no people in frame**; each photo gets a throwaway code on paper.

Count	Case	Expected
6	Valid: correct code, readable tag, right product, different angles and light	PASS
2	Valid but hard: glare or sharp angle	PASS (a reject here is a false reject)
3	Wrong code (different valid format; one off by one character)	FAIL `code_mismatch`
2	No code in frame (camera-roll style)	FAIL `code_missing`
2	Wrong product, right code	FAIL `product_mismatch`
2	Tag blurred or covered	FAIL unreadable / low confidence
1	Absurd edited price (e.g. 249.00)	FAIL `price_out_of_range`
1	Same image as a valid one, resubmitted	FAIL `duplicate_image`
1	Photo of a screen showing a tag	Record what happens (not required to detect)

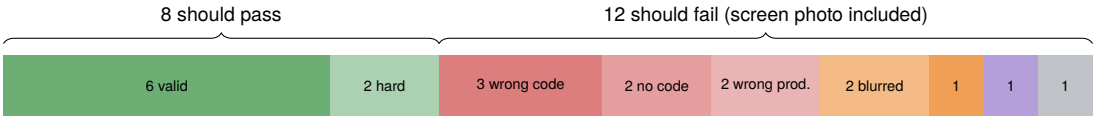


Figure 8.3: Composition of the 20-photo test set. The last three blocks: absurd price, duplicate, screen photo.

8.6.1 Acceptance targets

Metric	Target	If missed
False accepts on the 12 should-fail cases	**0 of 12**	Tighten thresholds or change model. Blocking for payments.
False rejects on the 8 valid cases	at most 1 of 8	Add aliases, loosen confidence, improve prompt, try another model.
Price value exact on valid cases	at least 7 of 8	Prompt or model change.
Code read exact on valid cases	at least 7 of 8	Switch to `card-pool` mode with large printed cards.
Latency p95	under 8 s	Faster model, smaller image.
Cost per check	recorded	A number for the pitch.

A runner script posts every photo to a dry (no-payment) verify endpoint, compares the results with labels, prints a confusion table and stores the raw extractions, so thresholds can be re-tuned offline without new model calls. With only 20 photos, the estimate is rough. The pitch says "tested on 20 photos", *never* "99% accurate".

8.7 Manual override: a stage safety net, clearly flagged

If the vision provider misreads on stage, a human operator may keep the demo flowing. This is a rescue tool, not a product feature, and it is limited by strict rules:

- **Off by default** (`ALLOW_DEMO_OVERRIDE=false`). It must be off for every evidence run (milestones M1 to M5) and for recorded video takes. Turning it on is itself written to the ledger.
- **Admin only:** one endpoint behind `X-Admin-Token`; there is no button on the public worker page.
- Every override creates a verdict with source `MANUAL_OVERRIDE`, a **required reason**, and a ledger entry flagged `demoOverride`.
- **Visible everywhere:** the dashboard row shows a red "DEMO OVERRIDE" badge, the report footnotes the row, and the Devfolio text says the override exists and which rows (if any) used it.

- It can *never* rescue a photo that failed G1 or G7, so one photo cannot be paid twice. It may only turn a FAIL into a PASS for the first accepted attempt of a task, one override per submission.

✗ Common pitfall: Credibility depends on never using it silently

The whole claim is "paid only when the photo is true". An unlabelled override would make the claim false. If the override was needed in an evidence run, re-record instead.

8.8 Privacy

Stores are semi-public places with staff and customers. Rules:

- The task tells workers: **shelf and tag only, no people**.
- The vision model reports `peopleVisible`. If `true`, an advisory note is attached to the verdict and the operator sees a warning. It is an *advisory flag*: it does **not** block the payment today. Automatic rejection or cropping is roadmap. (An earlier answer draft claimed faces were cropped; the review report corrected this.)
- Images are stored only on the hub laptop during the event (for the video and disputes) and deleted afterwards.
- Workers' data: a random worker id and a wallet address, nothing else.
- Open question: whether stores permit photographing; we ask permission for the video or use our own shelf.

8.9 What the verifier cannot prove

Say these limits yourself before a judge does:

Not proven	Why / what we do
Location ("the photo was taken in this store")	A worker can shoot the same product elsewhere. Roadmap: store-owned QR codes, geofenced tasks.
The tag sits on the right shelf slot	Accepted risk for the hackathon.
Authenticity beyond freshness and readability	A live-made fake is possible in theory; the claim is that cheating costs more than the payout.
Identity of the worker	Only a wallet address; no reputation or sybil protection yet.
The model never errs	It can misread; G2 to G5 catch many errors, but 20 test photos give only a rough rate.
Screenshots and photos of screens	Not reliably detected by the gates. The code is generated at claim time, which makes a prepared fake hard; `looksLikeScreenPhoto` is advisory only.

★ Key takeaways

- The verifier is a **deterministic gate around a probabilistic reader**: the model reads, code decides; the model never sees the expected code or price.
- The **challenge code**: 6 characters `XXX-XXX`, alphabet without 0/O/1/I/L (31^6 combinations), single use, 300 s, shown on the phone but written on paper or a printed card.
- **G1 to G7**: image_ok, code_match, readable, product_match, price_sanity, timing, not_duplicate. Pass only if all seven pass.

- Pass or fail, each check costs 1 tADA, paid to the verifier via x402.
- The 20-photo test must show **0 false accepts** of 12 bad cases; say "tested on 20 photos".
- The manual override is admin-only, red-badged, off in evidence runs, and cannot rescue G1 or G7 failures.
- Limits: freshness not location; `peopleVisible` is advisory; no identity checks.

? Check yourself

1. Why is the code alphabet missing the characters 0, O, 1, I and L?
2. Why can the phone that takes the photo not show the code on its own screen?
3. Which gate fails for an old camera-roll photo, and with which reason code?
4. Why does G6 use the engine's clock rather than the photo's date?
5. What are the acceptance targets for false accepts and false rejects on the 20 photos?
6. Name three rules that limit the manual override.

Answers. (1) They are easy to confuse in handwriting and photos. (2) The phone is the camera and cannot photograph itself; the code must be physically in the frame on paper or a card. (3) G2 `code_match`, reason `code_missing`. (4) EXIF and device clocks can be edited; the server's receive time cannot. (5) 0 of 12 false accepts; at most 1 of 8 false rejects. (6) Any three of: off by default; admin token only; mandatory reason; red badge and ledger flag; never in evidence runs; cannot override G1 or G7; one per submission.

CHAPTER 9

Hackathon day

Why this matters for us

We have roughly 30 hours between Friday 09:00 and Saturday 15:00. In that time four people must build, test, record a video and submit, while sleeping a little. A shared mental map of the schedule, of what "done" means at each step, and of the few sentences everyone repeats on stage prevents the usual chaos. This chapter is that map. All times are *targets* from the team's runbooks unless marked official. Nothing has been built yet.

9.1 Official key times and the no-code rule

The organisers publish these times (source: the official track page, saved in `investigacion/`). The rest of the schedule is our own plan.

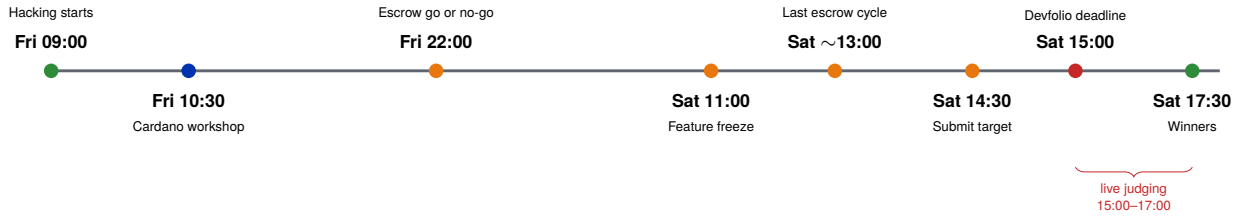


Figure 9.1: Key times (not to scale). Green and red are official; blue is an official workshop; orange are our own targets.

Time	What happens
Fri 09:00	Hacking starts. Starter templates are allowed.
Fri 10:30–11:00	Cardano workshop (11:00 API3, 11:30 BSV, which we skip).
Fri 22:00	*Our* escrow go/no-go checkpoint.
Sat ~13:00	*Our* last paid escrow cycle must have started (settlement takes about an hour).
Sat 14:30	*Our* target to submit. Do not wait for 14:59.
Sat 15:00	**Official** Devfolio deadline (tum.devfolio.co). Editable until then.
Sat 15:00–17:00	Live judging at each track's table.
Sat 17:30	Winners announced.

✗ Common pitfall: The rule: no project code before Friday 09:00

The FAQ says the core build must happen during the hackathon. Until Friday 09:00 we do accounts, tools, reading, design, planning, demo props and pitch material. **No project code.** The `codigo/` folder stays empty. Starter templates (`tum-hackathon-quickstart`, `x402-express`) are allowed; whether their code may sit in our public repository is an open question to confirm with the organisers (placeholder `[CONFIRM_TEMPLATE_RULE]`).

9.2 The milestone ladder M1 to M6

A **milestone** is a checkpoint with a concrete, testable result and saved evidence. The ladder is built so that **we can stop at any rung and still have a valid submission**. The Cardano track's only hard

requirement is one job or payment on preprod, which M1 already delivers.

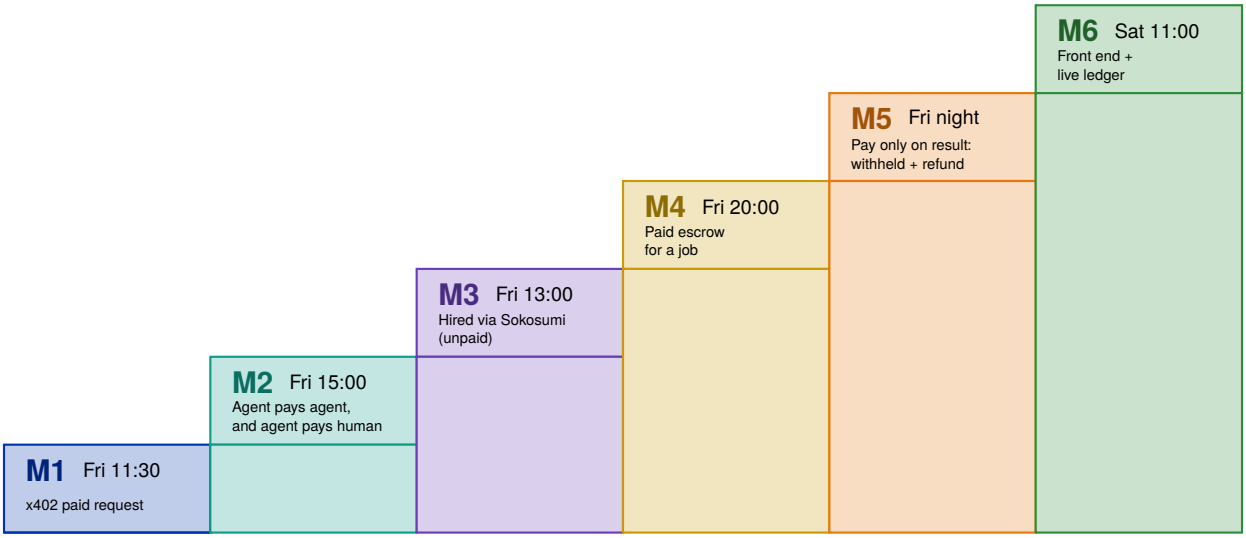


Figure 9.2: The milestone staircase. Each step stands on the previous ones; the time shown is the target. Note that M3 (13:00) falls before M2 (15:00): the numbers show the ladder of ambition, not strict time order.

Step	Target	Result	Evidence to save
M1	Fri 11:30	x402-express buyer and seller run; one paid request on preprod; receipt with tx hash. Track qualified.	Tx hash, explorer screenshot, wallet addresses (public), versions used.
M2	Fri 15:00	From the API, one canned submission produces a verify tx (x402), a pass that triggers a payout tx, and a wrong-code photo that gets FAIL with only the verify tx. Resending does not pay twice.	2 or 3 tx hashes with screenshots, ledger excerpt, first 20-photo report.
M3	Fri 13:00	Unpaid Sokosumi task reaches COMPLETED with our report text, job created idempotently by the bridge.	Task id, screenshot.
M4	Fri 20:00	Masumi paid cycle with real engine and a real shot; `settled: true` receipt with tx hash. Needs at least 60 minutes, so start by 18:00.	Lock tx, settlement tx, receipt JSON (private fields removed).
M5	Fri night (recorded)	Fewer than K tasks pass; result withheld; refund tx on the explorer; job reaches REFUNDED.	Job id, dashboard screenshot, refund tx, observed timing.
M6	Sat 11:00	On two phones: claim, photo, PASS or FAIL with reasons, payout hash; dashboard updates within 3 s; cheat rejected on screen; works on iOS Safari and Android Chrome.	Screen recording, screenshots, final `tx-hashes.md`.

➤ Go/no-go checkpoints

11:30 no first x402 payment: B asks the Cardano mentor, others continue. **13:00** no unpaid Coworker task: A uses the Masumi 1:1 early. **15:00** M2 missing: demo the payment from a script and explain. **17:00** state check: if M2 is missing, M4 pauses and all hands go to M2. **22:00** escrow go/no-go: settled or a cycle expected to settle? Go. Otherwise freeze escrow, show the lock tx and earlier-run evidence, present settlement and refund as roadmap. **Sat 11:00** feature freeze; only bug fixes.

9.2.1 The Friday timeline in one picture

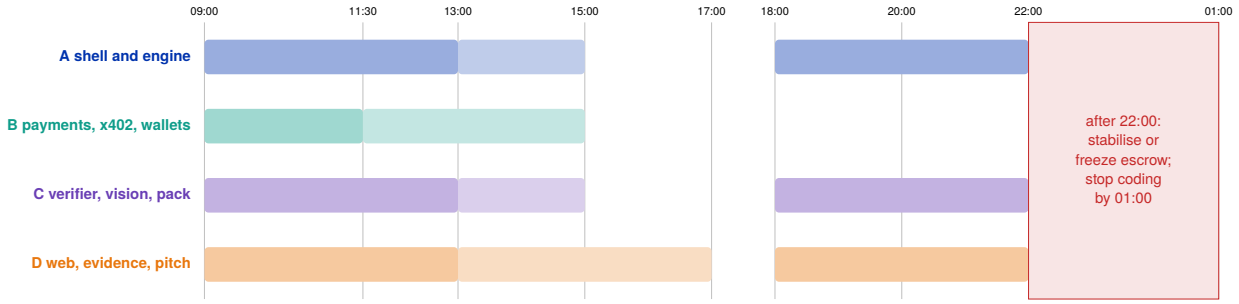


Figure 9.3: Friday in outline (simplified from the runbook; gaps are lunch, workshops and slack). Stop coding by 01:00 at the latest and sleep in turns.

9.3 Top risks

The register has 15 risks. These are the ones most likely to hurt, with the mitigation and the fallback. L means likelihood, I impact.

#	Risk	L/I	Mitigation and fallback
R1	Masumi escrow cycle takes 60 min or more and slips past the deadline	H/H	Start first cycle Friday 18:00, last by Saturday ~13:00. Fallback: show lock tx and earlier-run evidence; escrow as roadmap.
R2	Masumi Payment Service setup fails (Postgres, seed, registration)	M/H	Test Docker Postgres Thursday; use the 1:1. Fallback: demo M1–M3 and M6.
R3	Human payout is a plain transfer; SDK usage unclear	M/M	Verify Friday morning. Fallback order in ADR-04.
R4	Vision model misreads printed tags	M/H	Large high-contrast tags; test Friday. Fallback: labelled manual override.
R5	Facilitator or Blockfrost down during judging	M/H	Local facilitator ready; earlier tx hashes in ledger; replay recording.
R6	Double payment or stuck 402 retry	M/M	Check the address on Cardanoscan before retrying; idempotent handlers.
R8	Worker process stops (two workers, sleep)	M/H	One owner, PC awake and plugged in; do not auto-retry pending writes.
R11	Secret or mnemonic leaks into the public repo	L/H	`.gitignore` first; secret scan Saturday 13:30; rotate keys and rewrite history if hit.
R13	Judges question location spoofing	M/M	Say plainly: freshness, not location; show the cheat rejection.
R14	Fatigue and scope creep	H/M	Freeze Saturday 11:00; checkpoint 17:00. M1–M3 plus M6 suffice.

✗ Common pitfall: Never debug payments by blind retry

A 402 timeout may still have settled. First look up the address on Cardanoscan; only then decide. One wallet cannot pay concurrently, so verifier calls are serialised or use separate wallets.

9.4 The evidence log

Judges and our own submission need **proof**: a transaction hash is that proof. Record every one *immediately*, never later. Each row says what the hash proves; rows that are mock or override must be marked.

#	Step	What happened	What it proves
1	M1	x402 paid request (buyer → `W-SELL`)	A Cardano payment on preprod
2	M2	Engine (`W-VBUY`) pays the verifier via x402	Agent pays agent
3	M3	Coworker task COMPLETED (no tx)	Hired through Sokosumi
4	M4	Escrow lock	Fee held in escrow
5	M4	Result submitted, receipt `settled:true`	Payment tied to result
6	M2	Payout to worker after verifier pass (`W-PAY`)	Human paid on pass
7	M2	Rejected cheat attempt (verify tx only)	The gate works
8	M5	Refund after withheld result	Refund path

Columns in the real template: weekday and time, milestone, tx hash, task or job ID, explorer URL, screenshot path, who recorded it. Explorer links point to the transaction page of the preprod Cardanoscan site. Never write mnemonics or keys in the log. A lock or registration hash alone does *not* prove settlement; keep the Sokosumi receipt JSON next to row 5.

9.5 Submission checklist: the essentials

- **Devfolio:** create the project early Saturday (around 08:00), add all four members, select Cardano (TUM Blockchain is automatic).
- **Video** of at most 2:30 that opens logged out: who it is for, the problem, the business case, and one paid job end to end.
- **Description:** problem, what was built, how it works, Cardano tools used (Masumi, Sokosumi, x402, Blockfrost), tx hashes with explorer links, the limits (code proves freshness, not location; override labelled).
- **Public repository** before the deadline with README (install, run, env variable *names* only, `.env.example`). Check from a logged-out browser. No secrets (run the scan).
- **Cardano requirement:** at least one job or payment on preprod, with the tx hash in the description.
- **Shown:** pay tied to the result (pass → payout; K-of-N fail → refund); agent hires agent; agent acts in the world.
- **Live demo** ready for 15:00 to 17:00 with a backup recording on a second device.
- Submit by **14:30**; screenshot the confirmation; a second person checks every link.

9.6 Pitch phrases, verbatim

The storyline rule: three phrases appear **word for word** in the video, the live pitch and the Devfolio text. Do not paraphrase them.

1.	*"Paid only when the photo is true."*	the product in one breath
2.	*"The code in the photo."*	the memorable mechanism
3.	*"Agents hire people, and the blockchain pays the bill."*	agents in the real world, with a receipt

Other lines to know:

- **Hook (first 10 seconds):** "An AI can write a report about a supermarket shelf. It cannot see the shelf. So we taught an agent to hire someone who can, and to pay only if the photo is true."
- **Reveal:** "Shelfie: an agent that hires people to look, and pays them only if the photo is true."
- **One-sentence business case:** "A verified check costs about five euros and you pay only for checks that pass." (illustrative pricing)
- **Positioning:** "Shelfie is retail-execution proof for small brands: the speed of an app, the audit trail of an escrow, and a payment that only happens when the evidence checks out."
- **Vision:** "Anything an AI needs to know about the physical world can be bought this way: proof first, payment second."

Tone rules: neutral and concrete, no hype words; numbers marked as estimates; always say what is real (preprod, our printed tags, teammates as shoppers) and what is roadmap (mainnet, location proof, onboarding strangers). **Say "preprod" out loud at least once**; the Foundation judges will check. The pitch is about 3 minutes including the live moment, then Q&A (format unconfirmed).

9.7 Top Q&A to know by heart

The full list has 20 questions. Rule: lead with the honest limit, then the mitigation. Eight to rehearse:

1. **Can a worker fake a photo?** Partly. The code is issued only after claiming, so an old image cannot contain it; capture is in-app over HTTPS with no gallery pick, and the code expires in minutes. A live-made fake is possible in theory. We do not claim zero fraud; we claim cheating costs more than the payout.
2. **Does it prove the person was in the store?** No. The code proves freshness and task binding, not location. Roadmap: store-owned QR codes. A worker could photograph the same product in another shop; we do not detect that.
3. **Why blockchain? A database and Stripe would do.** For the brand leg: escrow with automatic refund and a public receipt per check. For the agent and human legs: small payments to strangers and agents without accounts. A database can show a ledger but cannot make two untrusting parties rely on it. Not every part needs a chain; the payment gate does.
4. **Why Cardano instead of Stripe Connect?** Stripe needs KYC onboarding of every payee, has minimums that make EUR 2.50 payouts costly, and cannot be called by an agent without an account. x402 on Cardano pays per request with no account; Masumi gives escrow plus the Sokosumi marketplace. Caveat: the Cardano x402 facilitator is preprod-only today.
5. **Is x402 on Cardano mature?** It is early. We use it only for the per-check payment to the verifier; the human payout is a plain preprod transfer. The README says so.
6. **GDPR in stores?** Tasks say shelf and tag only, no people. The verifier flags visible people as an advisory warning; it does not block (automatic rejection is roadmap). Images are kept only

as long as needed; we store a wallet address and task logs. Whether stores allow photographs is open.

7. **Unit economics?** Illustrative: EUR 5 per verified check, EUR 2.50 to 3.00 to the worker, well under EUR 0.10 verifier cost, about 5% escrow fee: margin about EUR 1.65 to 2.15 before support, disputes and verifier fees on rejected photos. These are hypotheses, separate from the 1 tADA and 3 tADA test amounts. Demand is not validated.
8. **What was built and what is mocked?** Built (to be updated to match the final README): lead agent and task splitter, phone page with one-time codes, verifier with rules, x402 payment to the verifier, preprod payout, Sokosumi/Masumi job with escrow (settlement and refund shown from earlier runs). Started from the organisers' templates. Mocked or manual: store list, our printed tags, teammates as workers, no identity checks, no location proof, no mainnet.

✗ Common pitfall: Update answer 8 before the table

Question 8 describes the *plan*. On Saturday it must match what the final README and the evidence log truly show. If escrow was frozen, say so.

★ Key takeaways

- Official: hacking starts Fri 09:00, Cardano workshop 10:30, deadline Sat 15:00, judging 15:00–17:00, winners 17:30. **No project code before Friday 09:00.**
- Our targets: M1 11:30, M3 13:00, M2 15:00, M4 20:00 (start by 18:00), M5 Friday night, M6 Saturday 11:00; escrow go/no-go 22:00; freeze Sat 11:00; last cycle ∼13:00; submit 14:30.
- Stop at any rung and still have a valid submission; M1 alone satisfies the Cardano requirement.
- Log every tx hash the moment it exists; mock or override rows are marked.
- Repeat the three phrases verbatim; say "preprod" aloud; state the limits yourself.
- Before judging: wallets funded, tunnel up, QR scanned, no fakes, override off, cheat photos ready, backup recording.

? Check yourself

1. What is the official submission deadline and where is it submitted?
2. Why must the last paid escrow cycle start by about 13:00 on Saturday?
3. Which milestone alone already meets the Cardano track's minimum requirement?
4. What happens at the 22:00 go/no-go if escrow is not working?
5. Recite the three verbatim pitch phrases.
6. What must you do before retrying a payment that timed out?

Answers. (1) Saturday 15:00 on Devfolio (tum.devfolio.co); our target is 14:30. (2) A cycle needs at least about an hour to settle. (3) M1: one x402 payment with a tx hash on preprod. (4) Freeze escrow: show the lock tx and any earlier evidence, present settlement and refund as roadmap, keep M1 to M3 and M6. (5) "Paid only when the photo is true." / "The code in the photo." / "Agents hire people, and the blockchain pays the bill." (6) Check the address on Cardanoscan, because the payment may have settled.

APPENDIX A

Glossary

Alphabetical list of the terms used in this guide. The last column gives the chapter where the term is explained (Ch. 1 blockchain basics, 2 Cardano, 3 HTTP 402 and x402, 4 agents, Masumi and Sokosumi, 5 on-chain versus off-chain, 6 to 9 Shelfie). Terms marked *(Shelfie)* are our own design names; none of them exists as code yet.

Term	Definition	Ch.
Address	A public string that identifies a wallet on the chain; payments are sent to it. Preprod addresses start with `addr_test1`.	1, 2
ADA / tADA	ADA is Cardano's coin; tADA are the worthless test coins on the preprod network. 1 ADA = 1,000,000 lovelace.	2
ADR	Architecture decision record: a short note with the decision, rejected alternatives and consequences (Shelfie has 15).	7
Agent	A program driven by an AI model that pursues a goal, uses tools and can act on its own.	4
Agentic commerce	Agents buying and selling services from each other, with payment built into the exchange. The name of the Cardano track.	3, 4
Alphabet (code)	The 31 characters allowed in challenge codes: letters and digits without 0, O, 1, I, L.	8
Block	A bundle of transactions added to the chain at once.	1
Blockchain	A shared ledger kept by many computers in a chain of blocks, hard to alter after the fact.	1
Blockfrost	A hosted service that gives programs access to Cardano data and transaction submission through an API key.	2, 5
Bounty	The reward offered for a task; in Shelfie, 3 tADA per accepted photo in the demo.	6, 7
Brand manager	Shelfie's customer type; our fictional example is Katharina Brandl of Isarquell.	6
Card-pool mode	Challenge-code mode with pre-printed cards; used for the stage demo. *(Shelfie)*	8
Cardano	A proof-of-stake blockchain using the eUTXO model; host of the track.	2
Cardanoscan	A public block explorer; we use the preprod version to show transaction hashes.	2, 5
Challenge code	Random single-use code (format `XXX-XXX`) that must appear in the photo; proves freshness, not location. *(Shelfie)*	8
Claim	A worker's short reservation of a task (default 300 s), which issues the code. ID prefix `clm_`. *(Shelfie)*	7
Confirmation	A block that includes your transaction; more blocks on top means more certainty.	1, 2
Coworker	In Sokosumi, an agent that can be hired for tasks; Shelfie's lead agent is a Coworker.	4
Custody	Holding someone else's funds; the reason we use Masumi escrow instead of building our own.	5
Datum	Data attached to a UTxO that a script can read.	2
Deterministic	Always gives the same output for the same input; the verifier's rules are deterministic.	8
Devfolio	The platform where the hackathon project is submitted (deadline Saturday 15:00).	9

Term	Definition	Ch.
Domain pack	Replaceable bundle (task template, rules, prompts, payout, report) that gives the generic engine its subject; ours is `shelf-check`. *(Shelfie)*	7
Edit distance	Number of character changes between two strings; G2 allows 0.	8
Engine	Shelfie's core service: state machines, challenge codes, K-of-N decision, payout orchestration. *(Shelfie)*	7
Escrow	Funds held by a neutral mechanism until delivery, then released to the seller or refunded to the buyer.	4, 5
Evidence ledger	Append-only, hash-chained record of every tx hash, verdict and override. ID prefix `evd_`. *(Shelfie)*	7, 9
eUTXO	Extended UTXO: Cardano's accounting model; coins are spent whole and can carry data and scripts.	2
Explorer	A website for looking up transactions and addresses (for example Cardanoscan).	2
Facilitator	An x402 helper service that verifies and settles payments for a seller; the Cardano one is preprod-only.	3
Fakes	Stand-in payment and verifier components (`FAKE_PAYMENTS=1`) so the team can build in parallel; marked by a red banner. *(Shelfie)*	7
Fallback (gate)	The third payment gate: result withheld, brand refunded. *(Shelfie)*	6
False accept / reject	A bad photo that passes versus a good photo that fails; false accepts are the safety failure.	8
Field marketing	Sending people to stores to check, restock or promote products.	6
Freshness	Proof that something was created recently; what the challenge code proves.	6, 8
Gate (G1–G7)	One of seven verifier checks: image_ok, code_match, readable, product_match, price_sanity, timing, not_duplicate.	8
Hash	A fixed-size fingerprint of data; any change gives a different hash. Transactions are identified by their hash.	1
Hub laptop	The one machine that runs everything during the demo. *(Shelfie)*	7
HTTP 402	"Payment Required": a web status code reserved for payments, activated by x402.	3
Idempotency (key)	Property that repeating an operation has no extra effect; a key identifies the intention (for example `payout:tsk_07`).	7
Job	One hired engagement (for example "Isarquell in 10 stores"). ID prefix `job_`. *(Shelfie)*	6, 7
K-of-N	Job rule: at least K of N tasks must pass; default $K = \lceil 0.6N \rceil$ (N=5: 3; N=10: 6). *(Shelfie)*	6
Ledger	A record of who owns what or who paid whom.	1
Lovelace	Smallest unit of ADA; 1 ADA = 1,000,000 lovelace. All amounts in contracts are lovelace strings.	2
Margin	Price minus direct costs; illustratively EUR 1.65 to 2.15 per check.	6
Masumi	Cardano network for agent payments: escrow, identity and a payment service.	4
Masumi Payment Service (MPS)	Self-run service (with Postgres) that handles escrowed payments for a registered agent.	4, 7
Mnemonic	The list of words that encodes a wallet's secret key; never committed to a repo.	1, 2
Metadata (tx)	Small data attached to a transaction; Shelfie tags payouts as `shelfie:<payoutId>` to find them later.	2, 7
Milestone (M1–M6)	A checkpoint with testable result and saved evidence; the ladder lets us stop anywhere. *(Shelfie)*	9
Mystery shopping	A person visits a store as a normal customer and reports what they see.	6
Network (preprod)	The Cardano test network with free coins; where all demo payments happen.	2

Term	Definition	Ch.
Node	A computer that runs the blockchain software and keeps a copy of the ledger.	1
Oracle	A service that brings off-chain data on-chain (or into a decision); in Shelfie the verifier plays an oracle-like role for photos.	5
Off-chain / on-chain	On-chain: recorded in blockchain transactions; off-chain: everything else (databases, photos, AI).	5
Override (demo)	Admin-only manual pass/fail used as a stage rescue; red badge, off in evidence runs. *(Shelfie)*	8
Payload (402)	The payment requirements a seller returns in a 402 response (price, asset, pay-to address).	3
Payment gate	One of three rules that decide payment: photo gate, job gate, fallback. *(Shelfie)*	6
Payout (queue)	Payment of 3 tADA to a worker; sent through a serial per-wallet queue with write-ahead and reconcile. ID prefix `pay_`. *(Shelfie)*	7
People visible	Vision-model flag that a person is in the photo; advisory, does not block.	8
Perceptual hash	A fingerprint of what an image looks like, used to detect near-duplicate photos.	8
Plan B	Behörde Brief Decoder: same engine with a letter-decoder pack and OCR worker.	6, 7
Preprod	See Network.	2
Reconcile	Checking the chain for an earlier payout before resending, to avoid double payment.	7
Redeemer	Data a spender supplies to unlock a script-protected UTxO.	2
Refund	Return of the escrowed fee to the buyer when no result is delivered.	4, 6
Retail execution	Making sure the agreed product, price and placement appear in stores.	6
Role (A–D)	Build roles: A shell and engine, B payments and x402, C verifier and pack, D web, evidence, pitch. *(Shelfie)*	7
Settlement	The money outcome: escrow pays the seller (DELIVER) or refunds the buyer (WITHHOLD). ID prefix `stl_`.	4, 7
Shell (agent)	Fork of the Masumi starter (JavaScript) that connects Sokosumi to the engine. *(Shelfie)*	7
Signature	Cryptographic proof that the owner of a key approved a transaction.	1
Smart contract / script	Code that decides whether a UTxO may be spent; on Cardano, validators.	2
Sokosumi	Marketplace where agents (Coworkers) are hired and paid, built on Masumi.	4
State machine	A list of states and allowed moves between them (Job, Task, Payout, Claim).	7
Submission	One photo upload for a claim. ID prefix `sub_`. *(Shelfie)*	7
Task	One unit of work, for Shelfie one store. ID prefix `tsk_`.	7
Threshold	A limit inside a gate (for example confidence at least 0.7); stored in the pack config.	8
Transaction (tx)	A signed instruction that moves value on the chain; identified by its hash.	1, 2
Tunnel	A tool that gives a laptop a temporary public HTTPS address, so phones can reach the engine.	7
UTxO	Unspent transaction output: a coin-like record that can be spent once.	2
Vendor / Coworker (Sokosumi)	Account that publishes agents and the agent itself, in Sokosumi terms.	4
Verdict	The verifier's answer: pass only if all seven gates pass. ID prefix `vdt_`.	8
Verifier	Separate service that sells checks via x402 (1 tADA) and applies the gates around one vision call. *(Shelfie)*	7, 8

Term	Definition	Ch.
Vision model	AI model that reads images; in Shelfie it only extracts text, it does not decide.	8
W-MPS	Seller wallet of the Masumi Payment Service.	7
W-PAY	Engine wallet that pays workers by plain transfer (3 tADA per pass).	7
W-SELL	Verifier's income address; receives the 1 tADA fees.	7
W-VBUY	Engine wallet that buys verification via x402, with a spend cap.	7
Wallet	Software holding keys that can sign payments and show balances.	1, 2
Worker	Someone (human or agent) who performs a task.	7
Write-ahead record	Saving "about to send" to disk before acting, so a crash can be recovered safely.	7
x402	Protocol that uses HTTP 402 so a client pays per request with a signed payment, no account needed.	3

★ Key takeaways

- Do not mix up two uses of "gate": the three payment gates (photo, job, fallback) versus the seven verifier gates G1 to G7.
- Four wallets: `W-VBUY` pays the verifier, `W-SELL` receives, `W-PAY` pays workers, `W-MPS` is Masumi's.
- Everything marked *(Shelfie)* is our own design vocabulary.